



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Dipartimento di Ingegneria "Enzo Ferrari"

Corso di Laurea Triennale in Ingegneria Informatica (L-08)

<Titolo dell'Elaborato>

Tutor:

Prof. Mirco Marchetti

Candidato:

Alessandro Marchesini

Matricola 187179

ANNO ACCADEMICO 2025/2026

<Dedica, opzionale. Cancella tutto in questa pagina per saltarla>

Indice

1	Introduzione	1
1.1	Contesto generale	1
1.2	Obiettivo dell'Elaborato	1
2	Contesto di Riferimento e Stato dell'Arte	2
2.1	Telecomunicazioni	2
2.1.1	Telecomunicazioni terrestri	3
2.1.2	Telecomunicazioni satellitari	4
2.1.3	Analisi comparativa e necessità di monitoraggio intelligente	5
2.2	Sistemi NIDS basati su Machine Learning	7
2.2.1	NIDS in Machine Learning per le telecomunicazioni terrestri	8
2.2.2	NIDS in Machine Learning per le telecomunicazioni satellitari	9
2.3	Adattamento di Dominio e Scarsità dei Dati	12
2.3.1	Il fenomeno del domain shift nei sistemi NIDS	13
2.3.2	Adattamento di dominio supervisionato per la mitigazione dello sbilanciamento	14
2.4	Sintesi e Posizionamento del Lavoro	16
3	Formulazione Teorica e Framework Metodologico	18
3.1	Modello del Sistema e Spazi di Rappresentazione	18
3.1.1	Formalizzazione dei Domini e Composizione Asimmetrica del Traffico	20
3.1.2	Allineamento dello Spazio delle Feature e Preprocessing Astratto	25
3.2	Tassonomia delle Architetture di Classificazione e Valutazione Multi-Modello	28
3.2.1	Modello di Riferimento (Baseline) e Modelli Aggregati	30
3.2.2	Decomposizione Specialistica Biclasse	35
3.2.3	Spazio delle Famiglie di Decisione Alberiformi ed Ensembling	39
3.3	Framework di Adattamento, Granularità e Stabilità Statistica	44

3.3.1	Incertezza Stocastica e Valutazione Multi-Seed	46
3.3.2	Granularità della Risposta: Formulazione Binaria vs. Multiclasse	51
3.4	Framework Analitico, Spiegabilità e Significatività Statistica	54
3.4.1	Caratterizzazione Densometrica e Proiezioni Latenti	56
3.4.2	Spiegabilità e Attribuzione delle Feature	60
3.4.3	Metriche, Soglia e Significatività dei Confronti	63
3.5	Sintesi del Capitolo e Raccordo con la Campagna Sperimentale	68
4	Implementazione del Framework Metodologico	69
4.1	Ambiente Sperimentale e Riproducibilità	70
4.1.1	Repository e Organizzazione del Codice	70
4.1.2	Stack Tecnologico e Dipendenze	73
4.1.3	Gestione della Riproducibilità Stocastica	74
4.2	Caratterizzazione dei Dataset e Composizione dei Benchmark	75
4.2.1	Dominio Sorgente: UNSW-NB15	75
4.2.2	Domini di Destinazione: Segmento Satellitare/Terrestre STIN	76
4.2.3	Parametrizzazione e Istanziamento dei Benchmark	78
4.3	Allineamento delle Feature e Pipeline di Preprocessing	84
4.3.1	Feature Grezze e Criteri di Filtraggio Applicati	87
4.3.2	Implementazione dell'Operatore di Mappatura	89
4.3.3	Verifica dell'Omissione della Scalatura Esplicita	95
4.4	Implementazione dei Classificatori	96
4.4.1	Algoritmi di Base e Iperparametri	96
4.4.2	Istanziamento dei Modelli Aggregati	99
4.4.3	Istanziamento dei Modelli Biclasse	102
4.5	Protocollo di Addestramento e Valutazione	104
4.5.1	Orchestrazione del Ciclo di Addestramento Multi-Seed	105
4.5.2	Calcolo delle Metriche e Costruzione della Matrice di Valutazione	108
4.5.3	Implementazione del Test di Welch	117
4.6	Strumenti di Analisi Diagnostica e Spiegabilità	123

4.6.1	Standardizzazione Diagnostica, PCA e KDE	125
4.6.2	Implementazione di SHAP e TreeSHAP	132
4.6.3	Pipeline di Aggregazione SHAP–KDE	140
4.7	Sintesi della Pipeline Implementativa	143
5	Risultati Sperimentali e Analisi	149
5.1	Organizzazione dell’Analisi dei Risultati	150
5.1.1	Criteri di Presentazione e Confronto	150
5.1.2	Configurazioni Sperimentali e Criteri di Aggregazione	150
5.2	Risultati dei Modelli Aggregati	150
5.2.1	Prestazioni del Modello Nativo Sorgente	150
5.2.2	Degradazione delle Prestazioni in Presenza di Domain Shift	150
5.2.3	Prestazioni dei Modelli Ibridi Aggregati	150
5.2.4	Confronto tra Baseline INJECTION, Modello Sorgente e Oracle	150
5.3	Risultati dei Modelli Biclasse	150
5.3.1	Rilevazione delle Singole Famiglie di Attacco nel Dominio Sorgente	150
5.3.2	Rilevazione delle Singole Famiglie di Attacco nei Domini Target	150
5.3.3	Confronto tra Modelli Specialistici Sorgente e Ibridi	150
5.4	Analisi della Variabilità Multi-Seed	150
5.4.1	Distribuzione delle Prestazioni sui Seed	150
5.4.2	Media e Varianza delle Metriche di Prestazione	150
5.4.3	Stabilità delle Prestazioni tra le Configurazioni	150
5.5	Analisi della Significatività Statistica	150
5.5.1	Confronto della <i>F1-Score</i> con la Baseline INJECTION	150
5.5.2	Risultati del Test di Welch	150
5.5.3	Interpretazione dei Confronti Significativi	150
5.6	Analisi Diagnostica del Domain Shift	150
5.6.1	Distribuzione Geometrica dei Benchmark mediante PCA	150
5.6.2	Interpretazione delle Feature mediante SHAP	150
5.6.3	Analisi Densometrica delle Feature Selezionate	150

5.6.4	Raccordo tra Evidenze SHAP e Distribuzioni KDE	150
5.7	Sintesi Comparativa dei Risultati	150
5.7.1	Confronto Complessivo tra le Configurazioni	150
5.7.2	Evidenze Sperimentali sul Domain Shift	150
5.7.3	Evidenze Sperimentali sull'Efficacia dell'Iniezione Few-Shot	150
5.7.4	Sintesi delle Evidenze Quantitative e Diagnostiche	150
Ringraziamenti		151
Riferimenti		152

Elenco delle Figure

2.1	Architettura di rete integrata Terrestre-Spaziale: rappresentazione della superficie d'attacco e posizionamento dei punti di monitoraggio <i>Network Intrusion Detection System</i> (NIDS).	6
2.2	Confronto schematico tra la valutazione intra-dominio (Scenario A) e l'impatto del domain shift sul trasferimento delle prestazioni tra il contesto terrestre e quello satellitare (Scenario B).	12
3.1	Pipeline logica astratta: allineamento dello spazio delle feature $\mathcal{X}$, isolamento preventivo Train/Test anti- <i>data leakage</i> , sintesi parametrica con ratio ρ senza reinserimento (con ancoraggio alla classe minoritaria) e istanziazione logica dei benchmark. I blocchi tratteggiati indicano le elaborazioni logiche, mentre i blocchi a linea continua rappresentano gli insiemi di dati e le categorie di output.	23
3.2	Flusso di mappatura e omogeneizzazione dello spazio delle <i>feature</i> : gli operatori ψ_S e ψ_T , definiti separatamente per ciascun dominio grezzo, convergono verso lo stesso spazio condiviso $\mathcal{X}$ attraverso i tre criteri comuni di filtraggio e armonizzazione descritti nel testo.	27
3.3	Flusso concettuale del protocollo di addestramento, inferenza e disaggregazione delle prestazioni per i modelli aggregati ($\mathcal{M}_{\text{base}}, \mathcal{M}_S, \mathcal{M}_H^{(T)}$). I modelli, congelati post-ottimizzazione, vengono valutati sia sull'intero benchmark multi-classe per misurare la generalizzazione di dominio, sia sulle singole partizioni per famiglia d'attacco $\mathcal{D}_a^{(\text{test})}$ per l'analisi di disaggregazione puntuale.	35
3.4	Flusso concettuale del protocollo di valutazione incrociata (<i>cross-evaluation</i>). Il modello specialistico $\mathcal{M}_{S,a}$, addestrato esclusivamente sulla singola minaccia a , rimane invariato e viene proiettato in fase di inferenza sull'intero benchmark eterogeneo $\mathcal{D}^{(\text{test})}$	39

3.5	Confronto logico-procedurale tra gli schemi di aggregazione. A sinistra, il paradigma <i>Bagging</i> esegue un addestramento parallelo su campionamenti bootstrap indipendenti. A destra, il paradigma <i>Gradient Boosting</i> adotta un approccio sequenziale in cui ciascun albero mira a correggere i residui iterativi generati dall'insieme precedente.	44
3.6	Diagramma di flusso del protocollo di isolamento e valutazione stocastica. La stabilità della partizione dei dati è forzata top-down dall'unico seed deterministico s_0 , delegando la totalità della varianza osservata tra le matrici H_k all'inizializzazione stocastica dettata dall'insieme dei seed $\mathcal{S}$ interni agli stimatori.	47
3.7	Rappresentazione bidimensionale nello spazio delle caratteristiche $\mathcal{X}$ dell'impatto del <i>domain shift</i> (vettore di traslazione $\boldsymbol{\eta}$) a seconda della granularità del modello. A sinistra, la maggiore ampiezza spaziale del modello aggregato garantisce tolleranza generalista; a destra, l'ottimizzazione specialistica sul singolo supporto malevolo determina rigidità sotto perturbazione.	53
3.8	Mappatura della probabilità a posteriori $f(\mathbf{x})$ sulla regola decisionale binaria $\hat{y}(\tau)$ in funzione della soglia τ	65
3.9	Rappresentazione schematica dell'Area Sottesa alla Curva (AUC) per le analisi grafiche ROC (a) e Precision-Recall (b).	67
4.1	Rappresentazione schematica della gerarchia delle directory del progetto e delle risorse generate a runtime.	72
4.2	Diagramma di flusso del processo di estrazione, pulizia, campionamento vincolato ($\rho = 10$) e partizionamento stratificato 80/20 per la generazione dei benchmark fisici aggregati e biclasse.	84
4.3	Diagramma di flusso della pipeline di preprocessing e armonizzazione tra il dominio sorgente $\mathcal{D}_S$ e i domini target $\mathcal{D}_T$, dall'estrazione delle feature grezze alla generazione dello spazio coordinato finale a 16 dimensioni.	86
4.4	Architettura del flusso dati per l'addestramento dei Modelli Aggregati. Le percentuali indicano l'origine logica dei pacchetti che compongono i <i>training set</i> , forzati strutturalmente al vincolo $\rho_{\text{ben/mal}} = 10/1$	101

4.5	Flusso operativo dell'orchestrazione multi-seed e isolamento degli artefatti persistiti.	106
4.6	Pipeline a due stadi per il calcolo delle metriche di classificazione, tracciamento su file system e aggregazione statistica multi-seed.	115
4.7	Pipeline di trasformazione dei dati per l'esecuzione del Test t di Welch, dal recupero delle metriche sui singoli seed alla persistenza degli esiti statistici.	119
4.8	Schema concettuale della Standardizzazione Diagnostica Dipendente dal Sorgente: le statistiche di centralità e variabilità (μ_S, σ_S) e gli autovettori della PCA ($\mathbf{W}_S$) vengono stimati esclusivamente sul benchmark sorgente S e congelati. La trasformazione dei benchmark target T_k avviene per applicazione diretta di tali parametri, preservando lo scostamento distributivo reale (<i>domain shift</i>).	127
4.9	Architettura della pipeline di spiegabilità SHAP per i modelli di classificazione. Il diagramma illustra le quattro fasi sequenziali disposte in blocchi verticalmente allineati con connessioni di flusso circolare (effetto Pacman) sul margine destro/sinistro tra le diverse fasi.	136
4.10	Diagramma architetturale del raccordo <i>Human-in-the-Loop</i> tra l'analisi diagnostica (SHAP) e la valutazione densometrica (KDE). L'intervento manuale (Fase 2) funge da disaccoppiamento metodologico tra le due pipeline automatizzate.	141
4.11	Architettura software end-to-end e flusso di esecuzione unificato: dall'ingestione dei dataset grezzi, passando per l'armonizzazione delle feature e il ciclo di addestramento multi-seed, fino alla biforcazione tra ramo inferenziale (valutazione prestazionale) e ramo diagnostico (PCA, KDE e TreeSHAP).	147

Elenco dei Codici

4.1	Routine di campionamento per l'istanziamento dei benchmark.	79
4.2	Routine di bilanciamento stratificato per l'istanziamento dei benchmark.	80
4.3	Trasposizione operativa dell'operatore di mappatura ψ_k tramite vettorializzazione pandas per il dominio UNSW-NB15.	93
4.4	Trasposizione operativa dell'operatore di mappatura ψ_k tramite vettorializzazione pandas per il dominio STIN.	94
4.5	Estrazione procedurale dell'etichetta di attacco per la generazione dinamica della nomenclatura del modello.	104
4.6	Gestione difensiva dei casi limite e calcolo delle metriche in <code>calculate_metrics()</code> .111	
4.7	Estrazione dei campioni ed esecuzione del Test t di Welch.	121
4.8	Standardizzazione e adattamento della PCA sul benchmark sorgente.	128
4.9	Applicazione della trasformazione diagnostica ancorata al benchmark sorgente. . .	128
4.10	Adattamento dello standardizzatore sulle feature utilizzate per la KDE.	129
4.11	Trasformazione diagnostica feature-per-feature ancorata ai parametri dello scaler sorgente.	130
4.12	Generazione delle stime KDE ed impostazione della scala logaritmica simmetrica su asse y.	131
4.13	Campionamento stratificato del test set per l'analisi SHAP.	134
4.14	Inizializzazione dinamica dello spiegatore SHAP con catena di fallback.	135
4.15	Calcolo, normalizzazione dimensionale e rendering dei valori SHAP.	138
4.16	Configurazione delle feature selezionate per l'analisi KDE.	142

1. Introduzione

1.1 Contesto generale

1.2 Obiettivo dell'Elaborato

2. Contesto di Riferimento e Stato dell'Arte

Il presente capitolo delinea il quadro concettuale, metodologico e bibliografico all'interno del quale si posiziona il lavoro di ricerca, ponendo le basi per l'analisi dei sistemi di rilevamento delle intrusioni in ambienti di rete complessi ed eterogenei. L'obiettivo primario è chiarire come la natura del canale trasmissivo e le relative dinamiche fisiche influenzino la rappresentazione vettoriale dei dati, condizionando la stabilità e la capacità di generalizzazione dei modelli di sicurezza basati su Machine Learning.

La trattazione si articola come segue. La Sezione 2.1 analizza le caratteristiche strutturali, le vulnerabilità e i profili di rischio che differenziano le infrastrutture terrestri dai segmenti spaziali, evidenziando la necessità di paradigmi di monitoraggio attivo. La Sezione 2.2 esamina i principi operativi dei *Network Intrusion Detection Systems* (NIDS) guidati dall'apprendimento automatico, fornendo una disamina critica della letteratura principale e identificando i limiti dei modelli attuali nel gestire transizioni tra domini differenti. Successivamente, la Sezione 2.3 formalizza rigorosamente i fenomeni di disallineamento distributivo (*domain shift*) e di scarsità dei dati etichettati, definendo le strategie di adattamento di dominio supervisionato. Infine, la Sezione 2.4 sintetizza i gap metodologici emersi dallo stato dell'arte e colloca puntualmente i contributi innovativi e l'approccio proposto da questo lavoro di tesi nel panorama scientifico di riferimento.

2.1 Telecomunicazioni

L'evoluzione delle infrastrutture di rete ha elevato i sistemi di telecomunicazione a componenti abilitanti fondamentali per la società moderna, la cui resilienza rappresenta un prerequisito imprescindibile per la protezione delle infrastrutture critiche e la continuità dei servizi essenziali [33]. Con l'affermazione di un paradigma trasmissivo interamente digitale, la convergenza di supporti fisici elettrici, ottici ed elettromagnetici su topologie geografiche complesse ha determinato un ampliamento esponenziale della superficie d'attacco¹.

¹La superficie d'attacco definisce l'insieme complessivo dei vettori di ingresso, delle vulnerabilità software, delle interfacce di rete e dei nodi fisici attraverso cui un agente malevolo può tentare di accedere, degradare o alterare un

La compromissione o la degradazione intenzionale di un canale trasmissivo non si limita a generare una temporanea interruzione della connettività, ma è in grado di innescare fallimenti sistemici a cascata, inficiando l'integrità dei flussi e paralizzando processi decisionali in tempo reale [32]. In un panorama di minacce caratterizzato dall'azione di attori malevoli ad alta persistenza², la difesa dei vettori di trasmissione non può prescindere da una progettazione orientata alla sicurezza nativa (*security-by-design*).

Tale paradigma mira a garantire i requisiti fondamentali di riservatezza, integrità, disponibilità e autenticità dei flussi informativi. L'impostazione di un'efficace strategia difensiva richiede pertanto una disamina puntuale dei vincoli strutturali e dei profili di rischio che caratterizzano i diversi contesti trasmissivi, tracciando una distinzione rigorosa tra le peculiarità delle reti di superficie e le criticità emergenti dei canali orbitali.

2.1.1 Telecomunicazioni terrestri

In questo quadro, la conformazione delle infrastrutture di comunicazione terrestri evidenzia un profondo compromesso tra prestazioni trasmissive e complessità delle contromisure di sicurezza. Da un lato, la combinazione di mezzi guidati ad elevata ampiezza di banda — quali le fibre ottiche — e di vettori radio ad alta frequenza garantisce latenze di propagazione estremamente ridotte, rendendo tali reti il supporto d'elezione per le applicazioni in tempo reale [33]. Dall'altro, la natura distribuita e fisicamente accessibile dei nodi e degli apparati di instradamento intermedi espone l'infrastruttura a un ampio spettro di minacce, agevolando sia intercettazioni sulla tratta fisica sia attacchi logici di tipo *Man-in-the-Middle* (MitM) e *Denial of Service* (DoS) [32].

Sebbene i protocolli crittografici a livello di rete e trasporto (quali IPsec e TLS³) garantiscano la riservatezza e l'integrità dei dati contro l'ascolto passivo, essi si dimostrano intrinsecamente insufficienti a proteggere l'infrastruttura nella sua interezza. Tali meccanismi non ostacolano infatti

sistema di comunicazione.

²Con minacce ad alta persistenza (*Advanced Persistent Threats*, APT) si identificano campagne d'attacco mirate e prolungate nel tempo, condotte da organizzazioni strutturate che impiegano tecniche avanzate per infiltrarsi e mantenere un accesso stealth non autorizzato all'infrastruttura.

³IPsec (*Internet Protocol Security*) opera a livello di rete (Livello 3 del modello ISO/OSI) cifrando l'intero pacchetto IP o il relativo payload, mentre TLS (*Transport Layer Security*) agisce a livello di trasporto/sessione (Livelli 4-5 ISO/OSI) per proteggere il canale sopra protocolli orientati alla connessione come il TCP.

l'analisi del traffico e dei metadati⁴ condotta da osservatori esterni, né risultano efficaci nel contrastare aggressioni volumetriche di tipo DoS o manomissioni strutturali della catena trasmissiva. Si rende quindi indispensabile l'integrazione di sistemi di monitoraggio attivo, come i *Network Intrusion Detection Systems* (NIDS), capaci di analizzare continuamente i flussi di rete per identificare tempestivamente deviazioni dal comportamento nominale prima che si traducano in disservizi sistemici [15].

Ciononostante, la sola protezione del segmento di superficie si rivela insufficiente laddove si richieda la copertura di regioni orograficamente complesse, l'interconnessione di nodi isolati o una ridondanza critica per la continuità operativa. Tali vincoli impongono l'estensione dell'architettura trasmissiva verso il segmento spaziale, introducendo nuove dinamiche di canale e ulteriori sfide di sicurezza.

2.1.2 Telecomunicazioni satellitari

In tale contesto di integrazione, l'impiego di piattaforme orbitali rappresenta una soluzione architetturealmente imprescindibile per garantire una copertura geografica globale, introducendo tuttavia severe sfide operative e di sicurezza. La capacità di superare i vincoli orografici rende i collegamenti spaziali il vettore primario per la connettività in contesti isolati o d'emergenza, garantendo un'essenziale ridondanza per le infrastrutture terrestri [17]. Tuttavia, la natura *broadcast* del canale radio e la vasta impronta a terra (*footprint*) dei fasci di copertura espongono il segnale a continui rischi di intercettazione passiva e a tentativi di *jamming* volti alla saturazione dello spettro [19].

A tali vulnerabilità si sovrappongono i vincoli fisici e trasmissivi specifici delle diverse orbite. Se da un lato i collegamenti in orbita geostazionaria (GEO) sono penalizzati da un elevato ritardo di propagazione (con tempi di *Round Trip Time*, RTT, di circa 500 ms) che degrada le prestazioni dei meccanismi di controllo della congestione del TCP, dall'altro le costellazioni in orbita bassa (LEO) introducono elevate dinamiche di spostamento Doppler (*doppler shift*, con scostamenti fino a ± 100 kHz nelle bande Ku/Ka) e frequenti disconnessioni di *handoff* dovute al ridotto tempo di visibilità dei satelliti (5–10 minuti) [17].

⁴L'analisi del traffico (*traffic analysis*) consiste nell'estrazione di informazioni sensibili tramite l'osservazione dei metadati non cifrati (es. indirizzi IP di sorgente e destinazione, dimensione dei pacchetti, frequenza dei flussi e distribuzioni temporali degli interarrivi), anche in presenza di un payload completamente cifrato.

Inoltre, la potenziale iniezione non autorizzata di pacchetti o comandi di controllo verso il segmento spaziale evidenzia la chiara inadeguatezza della sola cifratura passiva. In conformità con le linee guida di difesa in profondità e monitoraggio continuo per i segmenti di terra e di bordo [19], risulta indispensabile combinare contromisure a livello fisico — quali tecniche di modulazione a spettro espanso a protezione della trasmissione (*TRANSEC*)⁵ — con sistemi di rilevamento delle intrusioni (NIDS) a livello logico, capaci di identificare tempestivamente deviazioni comportamentali del traffico prima che compromettano la disponibilità dell'intero collegamento.

2.1.3 Analisi comparativa e necessità di monitoraggio intelligente

In questa prospettiva di sintesi, l'analisi comparativa tra le architetture di telecomunicazione terrestri e satellitari evidenzia una fondamentale complementarietà strutturale, affiancata da sostanziali divergenze operative nei profili di prestazione, copertura e superficie di vulnerabilità. Se le infrastrutture terrestri garantiscono elevati valori di *throughput* e latenze di propagazione ridotte grazie ai mezzi guidati, esse risultano vincolate da una rigida topologia e da un'alta densità di nodi esposti ad aggressioni sia fisiche sia logiche [33]. Al contrario, i segmenti spaziali offrono una connettività pervasiva e affrancata da barriere orografiche, ma introducono criticità legate ai ritardi di propagazione, alle attenuazioni atmosferiche e alla vulnerabilità intrinseca del canale etere a intercettazioni e interferenze intenzionali (*jamming*) [17].

La convergenza di questi ambienti eterogenei nei moderni ecosistemi di comunicazione definisce un perimetro difensivo complesso (schematizzato nella Figura 2.1). In tale contesto, le contromisure di sicurezza tradizionali — quali la sola cifratura del dato o l'impiego di firewall statici — si dimostrano intrinsecamente insufficienti a contrastare vettori d'attacco dinamici e tecniche di evasione avanzate [32]. Ciò impone la transizione verso paradigmi di monitoraggio attivo e resiliente, in grado di rilevare tempestivamente anomalie di flusso lungo l'intero percorso trasmissivo.

Per rispondere a questa crescente complessità, si rende indispensabile l'integrazione di architetture *Network Intrusion Detection Systems* (NIDS) concepite per l'ispezione continua e non invasiva del traffico trasversale alle tratte terrestri e orbitali. Tuttavia, la massiva volumetria dei flussi e

⁵Le contromisure *TRANSEC* (*Transmission Security*) agiscono a livello fisico mediante modulazioni a spettro espanso, quali *DSSS* (*Direct Sequence Spread Spectrum*) o *FHSS* (*Frequency Hopping Spread Spectrum*), per rendere il segnale indistinguibile dal rumore di fondo o garantirne la resilienza contro interferenze intenzionali.

le fluttuazioni stocastiche del canale radio rendono i tradizionali NIDS basati su firme (*signature-based*⁶) inadeguati nei confronti di attacchi inediti (*zero-day*) o di alterazioni comportamentali sottili.

In tale scenario, l'adozione di modelli di Machine Learning si rivela determinante per profilare la dinamica del traffico nominale e identificare in tempo reale deviazioni marginali riconducibili ad attività malevole [15]. L'apprendimento automatico consente di automatizzare l'analisi predittiva e la correlazione di sicurezza multi-dominio, fornendo un presidio difensivo adattivo sia per le tratte di superficie sia per quelle satellitari. Ai fini di un'adeguata contestualizzazione del ruolo di tali strumenti nell'infrastruttura di rete, la sezione seguente introduce la formalizzazione teorica e metodologica dei NIDS e dei modelli di Machine Learning che ne costituiscono il motore analitico.

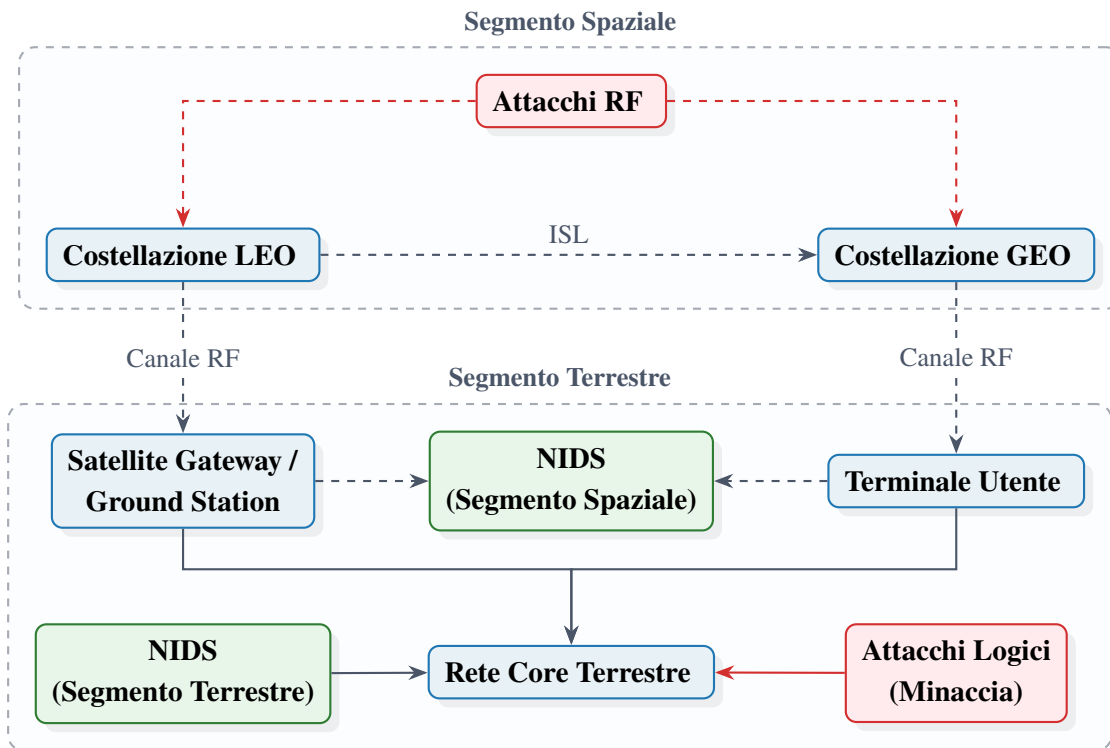


Figura 2.1: Architettura di rete integrata Terrestre-Spaziale: rappresentazione della superficie d'attacco e posizionamento dei punti di monitoraggio *Network Intrusion Detection System* (NIDS).

⁶I sistemi NIDS basati su firme, quali ad esempio Snort o Suricata, eseguono un'ispezione deterministica del traffico confrontando il contenuto dei pacchetti con un database di regole e pattern malevoli predefiniti, risultando ciechi di fronte a variazioni di formato, traffico cifrato o minacce non ancora catalogate.

2.2 Sistemi NIDS basati su Machine Learning

Muovendo da tali premesse, l'evoluzione delle minacce informatiche all'interno delle moderne infrastrutture di rete ha confermato l'inadeguatezza delle sole difese perimetrali e dei controlli d'accesso, rendendo indispensabile l'adozione di sistemi di rilevamento delle intrusioni basati su rete (*Network Intrusion Detection Systems*, NIDS) per il monitoraggio profondo e continuo del traffico logico. A differenza delle architetture *Host-based* (HIDS), progettate per analizzare i log locali e le chiamate di sistema del singolo dispositivo, un NIDS opera nei punti snodo dell'infrastruttura, ispezionando i flussi di dati per identificare tentativi di accesso non autorizzato, violazioni delle politiche di sicurezza o compromissioni dell'integrità del canale [12].

Tradizionalmente, la rilevazione delle intrusioni nei NIDS si articola in due macro-approcci: l'ispezione basata sulle firme (*Signature-based Detection*) e la rilevazione basata sulle anomalie (*Anomaly-based Detection*). Sebbene il primo garantisca un'elevata accuratezza nel riconoscere minacce note confrontando il traffico con un database di pattern malevoli predefiniti, esso mostra un'intrinseca inefficacia di fronte ad attacchi inediti (*Zero-Day*) e a varianti strutturali concepite per eludere i controlli [15]. Tale limite ha orientato la ricerca verso modelli analitici capaci di apprendere la profilazione comportamentale del traffico nominale.

Per superare la rigidità dei meccanismi deterministici basati su firme, l'integrazione delle tecniche di Machine Learning nei NIDS ha introdotto un paradigma adattivo e altamente generalizzabile, in grado di modellare la complessa dinamica del traffico trasmissivo moderno. Attraverso la formalizzazione di *feature* statistiche e sintattiche rilevanti⁷ — quali la frequenza dei pacchetti, la dimensione degli *header* e la distribuzione dei tempi di interarrivo —, un NIDS basato su Machine Learning costruisce una rappresentazione statistica del flusso informativo per rilevare deviazioni microscopiche riconducibili ad attività malevole senza la necessità di una firma preesistente [12, 15].

In questo contesto, la scelta metodologica di privilegiare classificatori di tipo supervisionato rispetto a soluzioni non supervisionate offre un vantaggio determinante in termini di affidabilità

⁷Nei NIDS moderni, la caratterizzazione del traffico poggia sull'estrazione di attributi di flusso (es. formato NetFlow/IPFIX), tra cui la dimensione degli *header*, la frequenza dei pacchetti, la dimensione delle finestre TCP e la distribuzione temporale degli interarrivi.

operativa⁸. Laddove l'apprendimento non supervisionato risente di un'elevata sensibilità alle fluttuazioni fisiologiche della rete, i modelli supervisionati permettono di tracciare confini di decisione precisi e stabili, ottimizzando la separazione tra flussi leciti e malevoli sia in formulazioni di classificazione binaria sia in configurazioni multiclasse per la caratterizzazione delle specifiche famiglie di minaccia [16].

2.2.1 NIDS in Machine Learning per le telecomunicazioni terrestri

L'applicazione dei *Network Intrusion Detection Systems* (NIDS) basati su Machine Learning alle infrastrutture di telecomunicazione terrestri risponde alla necessità imperativa di proteggere canali ad elevata ampiezza di banda e ad altissima densità di traffico. Nelle reti di terra ad alta velocità, i vettori d'attacco si manifestano prevalentemente attraverso aggressioni volumetriche di tipo *Distributed Denial of Service* (DDoS) o mediante attività di scansione coordinata (*port scanning*) finalizzate alla mappatura delle vulnerabilità del sistema [12, 15]. I modelli analitici sviluppati in letteratura si dividono principalmente tra approcci non supervisionati e classificatori supervisionati, ciascuno caratterizzato da specifici compromessi tra flessibilità applicativa e stabilità operativa [16]. Tuttavia, l'analisi dello stato dell'arte evidenzia un limite metodologico diffuso: la quasi totalità della letteratura valuta tali modelli esclusivamente all'interno del medesimo contesto di addestramento, sottovalutando il severo degrado prestazionale causato dalla variazione delle condizioni del canale e della distribuzione dei dati.

Un primo filone di ricerca nell'ambito del monitoraggio non presidiato poggia sull'analisi fondamentale di Sommer e Paxson [31], incentrata sullo studio critico dei limiti dell'anomaly detection basata su Machine Learning non supervisionato in reti di grande scala. Sebbene questo paradigma offra il vantaggio teorico di identificare deviazioni dal profilo di traffico nominale senza richiedere dataset etichettati — promettendo una potenziale copertura contro minacce inedite —, lo studio ne evidenzia i severi vincoli operativi. In particolare, l'assenza di confini di decisione guidati da etichette rende i modelli non supervisionati estremamente sensibili al rumore di fondo e alle fluttuazioni stocastiche del traffico, traducendosi in un elevato e inaccettabile tasso di falsi allarmi (*False Positive Rate*, FPR). Per superare tali criticità analitiche, la nostra trattazione privilegia

⁸L'apprendimento supervisionato consente di contenere drasticamente il tasso di falsi allarmi (*False Positive Rate*, FPR), che costituisce una delle principali criticità applicative dei NIDS in ambienti ad alta variabilità stocastica.

l'apprendimento supervisionato, al fine di garantire confini di separazione stabili, interpretabili e misurabili di fronte al disallineamento distributivo.

Spostando l'attenzione sui modelli guidati da dati etichettati, un primo contributo pionieristico per i classificatori supervisionati terrestri è rappresentato dallo studio di Tavallae et al. [34]. Gli autori hanno formalizzato le metodologie di *benchmarking* per la sicurezza di rete attraverso la bonifica dei bias di ridondanza con il dataset NSL-KDD⁹. Nonostante l'importanza storica di tale lavoro nel validare l'accuratezza dei modelli supervisionati, il nostro approccio se ne discosta nettamente: la valutazione condotta da Tavallae et al. rimane circoscritta a un dominio sintetico e stazionario, senza indagare il comportamento dei classificatori di fronte a fenomeni di *domain shift*¹⁰.

Successivamente, Sharafaldin et al. [30] hanno esteso la caratterizzazione dei NIDS supervisionati a flussi ad alta dimensionalità e a vettori d'attacco complessi tramite l'introduzione del dataset CIC-IDS2017. Sebbene questo studio costituisca uno standard di riferimento per la modellazione del traffico moderno, la nostra proposta segna un chiaro elemento di discontinuità: l'analisi di Sharafaldin et al. valuta le prestazioni focalizzandosi su una risposta prevalentemente multiclasse all'interno di un singolo contesto trasmissivo locale. Il nostro lavoro mette invece a confronto diretto la granularità binaria e multiclasse sotto scostamenti distributivi eterogenei, valutando come la struttura della risposta incida sulla stabilità dei confini discriminativi durante il trasferimento cross-dominio.

2.2.2 NIDS in Machine Learning per le telecomunicazioni satellitari

L'impiego dei *Network Intrusion Detection Systems* (NIDS) basati su Machine Learning nelle comunicazioni satellitari deve misurarsi con vincoli operativi e dinamiche di canale radicalmente divergenti rispetto al contesto terrestre. Le tratte orbitali sono penalizzate da elevati tempi di latenza di propagazione, capacità di canale fortemente asimmetriche e da una marcata suscettibilità

⁹Il dataset NSL-KDD ha risolto i problemi di duplicazione dei record presenti nell'originale KDD'99, i quali sbilanciavano le metriche di valutazione e alteravano la capacità dei classificatori di apprendere pattern di attacco rari.

¹⁰Con *domain shift* (o scostamento distributivo) si intende la variazione della distribuzione di probabilità congiunta dei dati tra il dominio di addestramento e quello di test ($P_S(X,Y) \neq P_T(X,Y)$), la quale determina un drastico decadimento della capacità di generalizzazione del modello.

al rumore atmosferico e ai fenomeni di *fading* stocastico del segnale [17]. In questo scenario, ai tradizionali vettori d'attacco logici si sovrappongono minacce specifiche del canale radio spaziale, quali le interferenze intenzionali di tipo *jamming* e l'iniezione non autorizzata di telecomandi malevoli (*command injection*) [19]. In ambienti caratterizzati da tale variabilità, la scelta di adottare classificatori supervisionati si rivela essenziale per contenere il tasso di falsi allarmi (*False Positive Rate*, FPR), a cui i modelli non supervisionati risultano strutturalmente vulnerabili quando esposti alle fluttuazioni stocastiche della rete [15, 17]. Tuttavia, la barriera primaria allo sviluppo di NIDS nativi per lo spazio risiede nella severa indisponibilità di dataset pubblici e realistici contenenti tracce etichettate di traffico satellitare malevolo [2].

Nell'ambito della letteratura focalizzata sulle reti integrate spazio-terra, un punto di riferimento di primaria importanza è costituito dallo studio di Wan et al. [36], incentrato su un NIDS distribuito basato su *Federated Learning* per la protezione delle infrastrutture satellitari. Il pregio principale di questo contributo risiede nell'adozione di un paradigma di addestramento cooperativo che tutela la riservatezza dei dati locali e riduce l'ampiezza di banda richiesta sui collegamenti spaziali. Ciononostante, il nostro approccio si diversifica in modo sostanziale da tale impostazione: lo studio di Wan et al. presuppone un'architettura federata attiva che richiede continui cicli di aggregazione e sincronizzazione dei parametri tra i nodi della costellazione. Al contrario, la nostra soluzione valuta la trasferibilità diretta (*zero-shot*¹¹) di un modello addestrato nel dominio terrestre verso l'ambiente satellitare, affrontando il *domain shift* senza gravare sui collegamenti orbitali con l'onere computazionale e comunicativo di un protocollo distribuito.

Infine, un ulteriore filone di ricerca sulle reti integrate spazio-aria-terra (SAGIN¹²) è rappresentato dal lavoro di Alam e Song [1], focalizzato sull'orchestrazione dinamica delle risorse mediante l'impiego combinato di *Graph Neural Networks* (GNN) e *Deep Reinforcement Learning* (DRL). Il merito fondamentale di tale studio risiede nella capacità di adattarsi alla topologia fortemente dinamica dei nodi spaziali per l'ottimizzazione delle prestazioni di rete. Tuttavia, il nostro lavoro si distingue nettamente da questa prospettiva: la trattazione di Alam e Song è orientata all'allocazione

¹¹Nel contesto considerato, l'approccio *zero-shot* indica la valutazione diretta delle prestazioni di un classificatore, addestrato esclusivamente su dati di origine terrestre, nei confronti del traffico proveniente dal canale satellitare, senza applicare riaddestramenti o adattamenti specifici (*fine-tuning*).

¹²*Space-Air-Ground Integrated Networks*: architetture di rete tridimensionali ed eterogenee che integrano la segmentazione spaziale (satelliti LEO/MEO/GEO), aerea (droni, HAPS) e terrestre.

delle risorse infrastrutturali e richiede modelli decisionali complessi caratterizzati da elevati costi computazionali e frequenti cicli di riaddestramento. La nostra proposta si concentra invece sulla caratterizzazione della sicurezza e sul rilevamento delle intrusioni tramite un'architettura di classificazione alleggerita, valutandone la robustezza discriminativa di fronte alle distorsioni di canale senza ricorrere a complessi processi di riapprendimento in tempo reale.

Tabella 2.1: Sintesi e confronto della letteratura principale sui NIDS nei domini terrestre e satellitare.

Rif.	Dominio	Approccio	Punto di Forza	Limiti e Gap Metodologico
[31]	Terrestre	ML Unsupervised	Rilevamento di minacce <i>zero-day</i> senza dataset etichettati.	Elevato tasso di falsi allarmi (FPR) dovuto alle fluttuazioni stocastiche di rete.
[34]	Terrestre	ML Supervised	Benchmarking rigido e bonifica dei bias di ridondanza (NSL-KDD).	Analisi circoscritta a dati sintetici e stazionari; assenza di valutazione su <i>domain shift</i> .
[30]	Terrestre	ML Supervised	Caratterizzazione di flussi ad alta dimensionalità (CIC-IDS2017).	Ristretto alla sola risposta multiclasse; omette la valutazione binaria e cross-dominio.
[36]	Spazio-Terra	Federated Learning	Addestramento distribuito e cooperativo con tutela della privacy.	Elevato <i>overhead</i> per sincronizzazione continua dei parametri su tratte orbitali.
[1]	SAGIN	GNN + DRL	Adattamento dinamico alle topologie spaziali ad alta variabilità.	Elevato carico computazionale; orientato alle risorse anziché alla rilevazione.

L'analisi dello stato dell'arte, i cui contributi fondamentali e i relativi elementi di discontinuità sono sintetizzati nella Tabella 2.1, evidenzia come la progettazione di NIDS basati su Machine Learning per infrastrutture integrate debba superare l'ipotesi di stazionarietà¹³ e omogeneità prevalente nella letteratura attuale.

Come illustrato nello schema concettuale di Figura 2.2, l'estensione dei modelli dal contesto terrestre a quello satellitare richiede di formalizzare le problematiche di degrado prestazionale causate dal disallineamento distributivo del traffico (*domain shift*) e dalla severa indisponibilità di dati etichettati nel segmento spaziale. Tali sfide metodologiche costituiscono l'oggetto della trattazione dettagliata sviluppata nella sezione seguente.

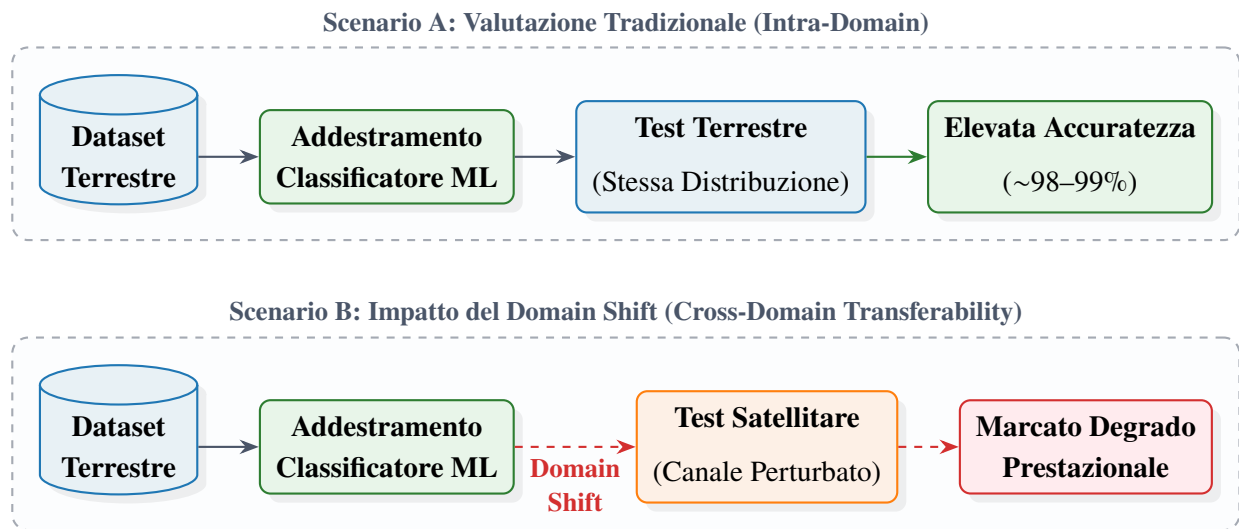


Figura 2.2: Confronto schematico tra la valutazione intra-dominio (Scenario A) e l'impatto del domain shift sul trasferimento delle prestazioni tra il contesto terrestre e quello satellitare (Scenario B).

2.3 Adattamento di Dominio e Scarsità dei Dati

A completamento dell'analisi delle criticità connesse all'integrazione di sistemi di sicurezza basati sull'apprendimento automatico nei moderni ecosistemi di telecomunicazione, risulta indispensabile

¹³L'ipotesi di stazionarietà assume che le proprietà statistiche dei flussi di rete (es. media, varianza e distribuzione dei pacchetti) rimangano invarianti nel tempo e tra domini differenti, condizione regolarmente smentita nella transizione tra reti terrestri e spaziali.

esaminare le problematiche metodologiche e teoriche legate alla variazione dei contesti operativi e alla disponibilità delle informazioni. La transizione da architetture trasmissive omogenee a scenari eterogenei impone infatti di superare i limiti strutturali associati non solo alla natura fisica dei canali, ma anche alla portabilità dei modelli analitici e alla disomogeneità statistica dei dataset di riferimento.

2.3.1 Il fenomeno del domain shift nei sistemi NIDS

L'efficacia dei modelli di Machine Learning applicati ai Network Intrusion Detection Systems (NIDS) poggia fondamentalmente sull'assunto teorico che i dati di addestramento e quelli operativi siano indipendenti e identicamente distribuiti (*i.i.d.*). Tuttavia, nelle infrastrutture di rete reali la natura dinamica e fortemente variabile del traffico invalida frequentemente tale ipotesi, introducendo un marcato scostamento distributivo [24]. In questo scenario, l'adozione di paradigmi di *transfer learning* diventa essenziale per mitigare il decadimento prestazionale dei classificatori. Tale approccio permette, infatti, di trasferire la conoscenza estratta dal dominio di origine eludendo il severo onere computazionale e logistico imposto dalla raccolta ex novo di dati etichettati e dal riaddestramento integrale dei modelli.

In termini formali, un dominio $\mathcal{D}$ è definito dalla coppia costituita da uno spazio delle *feature* $\mathcal{X}$ ¹⁴ e da una distribuzione di probabilità marginale $P(X)$, tale per cui $\mathcal{D} = \{\mathcal{X}, P(X)\}$. Assumendo uno spazio delle *feature* condiviso ($\mathcal{X}_S = \mathcal{X}_T = \mathcal{X}$), dati un dominio di origine (*source*) $\mathcal{D}_S = \{\mathcal{X}, P_S(X)\}$ e un dominio di destinazione (*target*) $\mathcal{D}_T = \{\mathcal{X}, P_T(X)\}$, il fenomeno del *domain shift* si verifica quando le distribuzioni marginali differiscono tra i contesti, ossia $P_S(X) \neq P_T(X)$ (*covariate shift*). In alternativa, o in concomitanza, esso si manifesta quando muta la distribuzione condizionata delle classi $Y \in \mathcal{Y}$ rispetto alle osservazioni, ovvero $P_S(Y|X) \neq P_T(Y|X)$ (*concept drift*) [24].

Nelle telecomunicazioni, questa discrepanza matematica si traduce in una tangibile alterazione dei profili di traffico. La transizione dal canale terrestre a quello satellitare introduce, infatti, elevati tempi di latenza di propagazione¹⁵, fluttuazioni stocastiche della larghezza di banda e severi effetti di

¹⁴Nel contesto dei NIDS basati su flussi di rete, lo spazio $\mathcal{X}$ codifica attributi statistici del traffico, quali la dimensione media dei pacchetti, la durata dei flussi e i tempi di interarrivo.

¹⁵Nei collegamenti satellitari geostazionari (GEO), il solo ritardo di propagazione *one-way* si attesta tipicamente sui 250-280 ms, alterando radicalmente le dinamiche temporali del protocollo TCP rispetto alle reti terrestri.

attenuazione atmosferica. Tali dinamiche fisiche modificano pesantemente la distribuzione dei tempi di interarrivo dei pacchetti e l'architettura dei flussi trasmessi, determinando una distorsione diretta nella rappresentazione vettoriale delle *feature* estratte dal sensore NIDS [17, 2]. Geometricamente, tale scostamento trasla i vettori di test rispetto agli iperpiani di separazione appresi durante la fase di *training* terrestre, provocando un grave collasso della capacità discriminativa del classificatore e un incremento incontrollato dei falsi allarmi [8].

In letteratura, il problema dell'adattamento di dominio (*domain adaptation*) viene affrontato prevalentemente ricercando uno spazio latente invariante o minimizzando metriche di distanza statistica (es. *Maximum Mean Discrepancy*) tra le distribuzioni. In questo ambito, il contributo di Wilson e Cook [38] fornisce un'esaustiva analisi teorica dei metodi di *unsupervised domain adaptation*, finalizzati a riallineare le distribuzioni marginali senza ricorrere a etichette nel dominio *target*. Tuttavia, sebbene efficaci nell'abbattere il *bias* in scenari standard, questi metodi sono calibrati su ambienti di rete intrinsecamente omogenei. Di conseguenza, il nostro approccio si differenzia posizionandosi in un'area ancora poco esplorata: la gestione della profonda asimmetria strutturale e della drastica corruzione dei profili di traffico indotte dal trasferimento diretto della sorveglianza dal canale terrestre a quello orbitale.

2.3.2 Adattamento di dominio supervisionato per la mitigazione dello sbilanciamento

Sul piano metodologico, la risoluzione del *domain shift* deve misurarsi con il severo sbilanciamento informativo tra gli ambienti di origine e destinazione. Nelle applicazioni reali di Network Intrusion Detection (NIDS), infatti, la cronica scarsità di campioni etichettati appartenenti al dominio *target* costituisce l'ostacolo primario all'addestramento di modelli di classificazione nativi. Tale vincolo configura scenari di apprendimento semi-supervisionato o a pochi campioni (*few-shot learning*), nei quali la disponibilità di osservazioni etichettate nel contesto di destinazione è estremamente ridotta o del tutto assente. Come formalizzato nel paradigma del *transfer learning* [24], in presenza di un set limitato di dati nel dominio *target*, è possibile trasferire la conoscenza strutturata appresa dal dominio di origine per preservare le proprietà di generalizzazione del classificatore.

Nelle comunicazioni satellitari, tale criticità risulta ulteriormente esacerbata dalla marcata

assenza di tracce di traffico etichettate relative al segmento spaziale. Tale carenza è imputabile a stringenti vincoli di riservatezza industriale e militare, agli elevati costi operativi di monitoraggio in orbita e alla natura intrinsecamente critica delle infrastrutture spaziali, fattori che rendono arduo il reperimento di dataset operativi rappresentativi [17, 2]. Sebbene parte della letteratura proponga l'impiego di modelli puramente non supervisionati per prescindere dalle etichette, tali approcci — basati sulla sola stima della densità di probabilità — mostrano un'intrinseca fragilità nei contesti ad alto rumore stocastico¹⁶. Essi tendono infatti a scambiare le fisiologiche fluttuazioni dinamiche del canale radio per attività malevole [19, 15].

Per preservare la capacità di definire confini di decisione netti tipica dei classificatori supervisionati, una strategia rigorosa consiste nell'adottare paradigmi di *supervised domain adaptation* basati sull'ottimizzazione congiunta del rischio empirico (*joint empirical risk minimization*) [6, 22].

In termini formali, sia $\mathcal{D}_S = \{(x_i^S, y_i^S)\}_{i=1}^{N_S}$ il dataset etichettato di origine terrestre e $\mathcal{D}_T^{sub} = \{(x_j^T, y_j^T)\}_{j=1}^{N_T}$ un sottoinsieme estremamente ridotto di campioni etichettati del segmento satellitare ($N_T \ll N_S$), con $y_i^S, y_j^T \in \mathcal{Y}$. L'addestramento del modello $f(\cdot; \theta)$ viene condotto minimizzando una funzione di perdita (*loss function*) congiunta $\mathcal{L}(\theta)$:

$$\mathcal{L}(\theta) = \frac{1}{N_S} \sum_{i=1}^{N_S} \ell(f(x_i^S; \theta), y_i^S) + \alpha \frac{1}{N_T} \sum_{j=1}^{N_T} \ell(f(x_j^T; \theta), y_j^T) \quad (2.1)$$

dove $\ell(\cdot)$ rappresenta la funzione di perdita istantanea (es. *cross-entropy*) e $\alpha > 0$ è un iperparametro di ponderazione della perdita¹⁷.

L'ottimizzazione dell'Equazione (2.1) permette di riallineare i confini di separazione e di inglobare la variabilità stocastica del canale di destinazione senza perdere la ricchezza informativa sugli attacchi complessi appresa dal dominio terrestre [22]. Tale approccio compensa l'impatto del disallineamento distributivo, garantendo un'elevata capacità di generalizzazione del classificatore sia nella discriminazione binaria sia nella caratterizzazione multiclasse delle minacce.

¹⁶Nelle tratte satellitari, le variazioni di canale (es. attenuazione da pioggia, effetto Doppler e ritardi stocastici) alterano la distribuzione statistica delle *feature* di rete, inducendo elevati tassi di falsi allarmi nei modelli basati su anomalie puramente non supervisionati.

¹⁷L'iperparametro α compensa la sproporzione quantitativa tra N_S e N_T , evitando che il gradiente calcolato sui dati terrestri sovrasti il segnale di adattamento proveniente dal dominio satellitare.

2.4 Sintesi e Posizionamento del Lavoro

L'analisi approfondita della letteratura scientifica condotta nel presente capitolo evidenzia come la convergenza tra le infrastrutture di comunicazione terrestri e i segmenti trasmissivi orbitali abbia ridisegnato il perimetro di sicurezza delle reti moderne. Se da un lato l'integrazione di sistemi di monitoraggio attivo basati su Machine Learning (NIDS) rappresenta la soluzione principale per contrastare vettori d'attacco dinamici e minacce *zero-day* [15, 12], dall'altro l'esame dello stato dell'arte mette in luce una marcata frammentazione metodologica, strutturata su due paradigmi tra loro disallineati:

1. **I NIDS per reti terrestri**, pur garantendo elevatissimi indici prestazionali su dataset di riferimento ad alta dimensionalità come NSL-KDD [34] e CIC-IDS2017 [30], vengono valutati quasi esclusivamente in condizioni stazionarie e intra-dominio. Tale impostazione ignora sistematicamente l'effetto che le variazioni della distribuzione dei dati (*domain shift*) producono sulla stabilità dei confini di decisione del classificatore.
2. **I NIDS per ambienti satellitari e SAGIN**, orientati ad affrontare la natura intrinsecamente dinamica del canale radio spaziale, ricorrono prevalentemente ad architetture complesse basate su *Federated Learning* [36] o all'orchestrazione con *Deep Reinforcement Learning* e Graph Neural Networks [1]. Pur offrendo soluzioni di protezione cooperativa e gestione delle risorse, tali approcci impongono continui cicli di aggregazione, riaddestramento e sincronizzazione dei parametri, generando un *overhead* comunicativo e computazionale¹⁸ difficilmente sostenibile su nodi orbitali con severe limitazioni di risorsa.

Tale scenario rivela un chiaro *Research Gap*: la letteratura scientifica manca di un'indagine sistematica e quantitativa sia sulla **trasferibilità diretta (*zero-shot cross-domain*)** dei modelli di Machine Learning addestrati su flussi terrestri verso il canale satellitare, sia sulle strategie trasversali di ***Supervised Domain Adaptation*** concepite per operare in contesti di estrema scarsità di campioni etichettati [17, 2]. L'assenza di dataset pubblici etichettati contenenti intrusioni reali registrate su

¹⁸Sui collegamenti satellitari, caratterizzati da elevati tempi di *Round Trip Time* (RTT) e da ampiezze di banda asimmetriche, lo scambio continuo di gradienti o pesi sinaptici tipico dei protocolli federati può saturare la capacità della tratta, incrementando i tempi di vulnerabilità del sistema.

tratte orbitali ha storicamente ostacolato questo filone di ricerca, favorendo l'impiego di tecniche di *unsupervised domain adaptation* [38] che tuttavia presuppongono un disallineamento distributivo contenuto e risentono dell'elevato rumore stocastico del canale radio spaziale.

In questo contesto, il presente lavoro di tesi si propone di colmare le lacune metodologiche riscontrate in letteratura, fornendo un quadro di analisi per la valutazione della robustezza discriminativa e della portabilità dei sistemi NIDS nel passaggio da ambienti di rete terrestri a segmenti satellitari.

Nello specifico, l'opera intende offrire i seguenti contributi principali:

- **Analisi del Domain Shift Cross-Domain:** Caratterizzazione teorica e quantitativa delle deformazioni indotte dal canale trasmissivo spaziale sullo spazio delle *feature* statistiche e sulla capacità di generalizzazione dei modelli terrestri.
- **Strategie di Adattamento di Dominio:** Valutazione di tecniche di *domain adaptation* per la mitigazione dello scostamento distributivo in contesti caratterizzati da forte scarsità di dati etichettati.
- **Studio sulla Granularità di Classificazione:** Indagine comparativa sull'impatto che differenti livelli di dettaglio nella risposta del classificatore (binaria e multiclasse) esercitano sulla stabilità delle prestazioni e sulla gestione dei falsi allarmi.
- **Valutazione di Architetture Difensive Efficienti:** Analisi di soluzioni di monitoraggio adeguate ai vincoli operativi delle infrastrutture satellitari, in grado di garantire stabilità discriminativa senza introdurre elevati costi computazionali o comunicativi.

Definito il quadro della letteratura e identificati i principali gap metodologici, il capitolo successivo introdurrà la formalizzazione teorica e la cornice metodologica generale adottata in questo lavoro.

3. Formulazione Teorica e Framework Metodologico

Il presente capitolo definisce l'impalcatura teorica e il framework metodologico alla base dell'intera indagine sperimentale, fornendo gli strumenti formali necessari per la modellazione, l'analisi e l'adattamento dei sistemi di *Network Intrusion Detection* (NIDS) in contesti operativi eterogenei. L'obiettivo primario risiede nell'instaurazione di un paradigma valutativo rigoroso, strutturalmente svincolato dalle specificità implementative dei singoli dataset, che consenta di quantificare oggettivamente i fenomeni di traslazione di dominio (*domain shift*) e di validare le performance di generalizzazione isolando i reali contributi algoritmici dalla varianza stocastica.

La trattazione si articola secondo una progressione logica suddivisa in quattro sezioni principali:

- La **Sezione 3.1** formalizza il modello astratto del sistema, definendo rigorosamente gli spazi vettoriali di rappresentazione, gli operatori di proiezione per l'armonizzazione delle *feature* e le regole architetturali di partizionamento dei domini.
- La **Sezione 3.2** delinea la tassonomia delle architetture di classificazione adottate e le relative strategie di *transfer learning*, uniformando le metriche di confronto mediante una formulazione omogenea basata sulla stima della probabilità a posteriori.
- La **Sezione 3.3** stabilisce il protocollo valutativo per la stabilità statistica, affrontando la gestione dell'incertezza stocastica tramite valutazione *multi-seed* e definendo gli indici morfologici di granularità dello spazio decisionale.
- Infine, la **Sezione 3.4** completa l'architettura metodologica descrivendo l'apparato diagnostico e di *explainability*, integrando strumenti di stima di densità e metriche di rilevanza fondate sulla teoria dei giochi cooperativi per la caratterizzazione e l'interpretazione causale delle divergenze distributive.

3.1 Modello del Sistema e Spazi di Rappresentazione

La presente sezione formalizza l'impalcatura teorica e matematica del framework analitico, definendo in modo rigoroso gli spazi vettoriali di rappresentazione e le distribuzioni di probabilità

necessarie per modellare i fenomeni di transizione cross-dominio nei sistemi NIDS. L'obiettivo è stabilire un modello astratto, indipendente dalle peculiarità di implementazione dei singoli dataset, in grado di garantire il rigore metodologico, la comparabilità delle metriche e l'assenza di contaminazione informativa.

Questa formalizzazione prosegue direttamente la disamina svolta nel Capitolo 2, traducendo in termini matematici e procedurali le criticità del confronto tra segmenti terrestri e spaziali e le esigenze di adattamento dei NIDS delineate nel contesto di riferimento.

A tal fine, la trattazione introduce preliminarmente la formalizzazione dei domini di origine e destinazione, delineando la semantica a doppia etichettatura (Y, A) , le regole di partizionamento preventivo anti-*data leakage* e il vincolo di proporzionamento parametrico ρ alla base delle cinque categorie astratte di benchmark, native e ibride, istanziabili dal framework. Successivamente, viene definito l'operatore di proiezione ψ_k , specifico per ciascun dominio $k \in \{S, T\}$, che mappa le caratteristiche grezze eterogenee nello spazio delle *feature* condiviso $\mathcal{X}$, esplicitando i criteri concettuali di filtraggio e armonizzazione degli attributi. La trattazione si conclude motivando, sulla base della proprietà di invarianza rispetto a trasformazioni monotoniche propria dei classificatori a partizionamento ricorsivo, l'omissione della fase esplicita di scalatura dalla definizione dello spazio delle *feature*.

Per facilitare la lettura della trattazione successiva, i principali simboli e costrutti matematici adottati sono sintetizzati nella Tabella 3.1.

Tabella 3.1: Sintesi della notazione matematica e dei simboli del framework.

Simbolo	Descrizione e Significato Concettuale
$\mathcal{D} = (\mathcal{X}, P(\mathbf{x}))$	Dominio di traffico (spazio delle <i>feature</i> e distribuzione marginale)
$\mathcal{X} \subseteq \mathbb{R}^d$	Spazio delle caratteristiche d -dimensionale omogeneizzato
$\mathbf{x} = (x_1, \dots, x_d)^\top$	Vettore delle <i>feature</i> del singolo flusso di rete
$\mathcal{Y} = \{0, 1\}$	Spazio discreto delle etichette primarie (0: Normal, 1: Malevolo)
$A \in \mathcal{A}$	Variabile descrittiva per la specifica famiglia di attacco (definita per $Y = 1$)
$\mathcal{A}_S, \mathcal{A}_T$	Insiemi delle tipologie di minaccia presenti nei domini sorgente e target
$\mathcal{D}_S, \mathcal{D}_T$	Dominio Terrestre Sorgente e Domini Target di Destinazione
$\mathcal{D}_k^{(train)}, \mathcal{D}_k^{(test)}$	Partizioni disgiunte di addestramento e test per il generico dominio k
$\psi_k(\cdot)$	Operatore di proiezione e combinazione dalle <i>feature</i> grezze allo spazio $\mathcal{X}$, specifico per il dominio $k \in \{S, T\}$
$N(\cdot)$	Operatore di cardinalità (numero di campioni estratti)

3.1.1 Formalizzazione dei Domini e Composizione Asimmetrica del Traffico

Il framework analitico richiede la definizione rigorosa degli spazi di rappresentazione vettoriale e delle distribuzioni di probabilità associate ai diversi contesti di rete considerati. Si definisce un dominio $\mathcal{D}$ come la coppia $(\mathcal{X}, P(\mathbf{x}))$, dove $\mathcal{X} \subseteq \mathbb{R}^d$ rappresenta lo spazio delle *feature* d -dimensionale condiviso e $P(\mathbf{x})$ la distribuzione di probabilità marginale definita sul vettore delle caratteristiche $\mathbf{x} = (x_1, x_2, \dots, x_d)^\top \in \mathcal{X}$.

Associato a ciascun dominio, la semantica del traffico è modellata tramite una doppia etichettatura. Lo spazio discreto primario $\mathcal{Y} = \{0, 1\}$ stabilisce la natura del flusso, dove $Y = 0$ indica il traffico nominale (*Normal*) e $Y = 1$ identifica genericamente un vettore malevolo. In aggiunta, si definisce un'ulteriore variabile descrittiva $A \in \mathcal{A}$ che specifica il nome della classe, garantendo una caratterizzazione puntuale sia per la componente benevola ($Y = 0$), per la quale A assume tipicamente un unico valore costante (*Normal*), sia per le diverse famiglie di attacco ($Y = 1$), per le quali A è genuinamente multi-valore.

Caratterizzazione Astratta dei Domini e Tassonomia delle Classi

Il framework analitico modella il fenomeno di transizione cross-dominio a partire da due macro-distribuzioni di partenza distinte:

1. **Dominio di Riferimento ($\mathcal{D}_S$):** rappresenta l'ambiente stazionario di origine, contenente sia istanze di traffico nominale ($Y = 0, A \in \mathcal{A}_S$) sia flussi malevoli ($Y = 1, A \in \mathcal{A}_S$). Per eliminare le componenti ad elevata sparsità ed esigua rappresentatività statistica, si applica una procedura di rimozione delle classi minoritarie (*minority removal*) sull'insieme $\mathcal{A}_S$, garantendo una caratterizzazione stabile del baseline di superficie.
2. **Domini di Destinazione ($\mathcal{D}_T$):** rappresentano i contesti soggetti a variazioni trasmissive o canali eterogenei, composti esclusivamente da vettori di traffico malevolo ($Y = 1, A \in \mathcal{A}_T$). Tali domini vengono strutturati applicando, laddove necessario, un raggruppamento delle classi minoritarie (*minority merge*) per accorpare famiglie d'attacco semanticamente affini e preservare la significatività campionaria.

Isolamento Preventivo delle Partizioni e Politiche Anti-Data Leakage

Per garantire il rigore metodologico ed evitare fenomeni di contaminazione informativa tra la fase di addestramento e quella di valutazione (*data leakage*)¹, la partizione di ciascun dominio primario $\mathcal{D}_k$ nei sotto-insiemi di addestramento ($\mathcal{D}_k^{(train)}$) e di verifica ($\mathcal{D}_k^{(test)}$) viene definita a monte di qualsiasi aggregazione o composizione dei benchmark derivati². In tutte le configurazioni sperimentali viene adottata una suddivisione stocastica stratificata con proporzione fissa:

$$\mathcal{D}_k = \mathcal{D}_k^{(train)} \cup \mathcal{D}_k^{(test)}, \quad \text{con } \mathcal{D}_k^{(train)} \cap \mathcal{D}_k^{(test)} = \emptyset \quad (3.1)$$

garantendo la totale indipendenza strutturale dei vettori impiegati per l'inferenza.

¹L'esecuzione dello split a monte di qualsiasi fusione previene che campioni identici della classe nominale, riutilizzati per benchmark diversi, compaiano simultaneamente negli insiemi di addestramento e di test.

²Dal punto di vista metodologico, tale partizionamento e la successiva ricomposizione dei vettori vengono eseguiti prima di qualsiasi composizione dei benchmark, evitando la creazione di dataset intermedi.

Sintesi dei Benchmark e Vincoli di Proporzionamento Riconfigurato

Sia per le configurazioni generate internamente al dominio sorgente sia per i benchmark ibridi ricomposti cross-dominio (necessari per l'assenza di traffico nominale etichettato nei contesti di destinazione), il framework applica una politica di proporzionamento controllato.

Per rispecchiare le condizioni realistiche di rete e prevenire uno sbilanciamento artificiale dei classificatori³, il framework impone un vincolo di proporzionamento parametrico ρ tra la componente benevola e la componente malevola complessiva:

$$\frac{N(Y = 0)}{\sum_{a \in \mathcal{A}_m} N(Y = 1, A = a)} = \rho \quad (3.2)$$

Tale vincolo definisce in modo esplicito l'asimmetria volumetrica tra il traffico nominale ($Y = 0$) e il traffico malevolo ($Y = 1$). Dove $N(\cdot)$ indica la cardinalità dei campioni estratti, $\mathcal{A}_m$ rappresenta l'insieme delle specifiche famiglie di attacco incluse nella configurazione e $\rho \in \mathbb{R}^+$ costituisce il fattore di asimmetria volumetrica prescritto dal protocollo metodologico.

A livello di estrazione, come regola generale si adotta il campionamento senza reinserimento (*without replacement*), precludendo la duplicazione sintetica delle istanze. Qualora, a seguito dell'applicazione del rapporto ρ , una delle due classi $Y \in \{0, 1\}$ non disponga di una quantità di dati sufficiente a raggiungere il volume teorico prefissato, il framework rifiuta l'uso del *replacing*. In tale evenienza, il dimensionamento complessivo del benchmark si ridimensiona ancorandosi alla classe minoritaria (ovvero quella con minore disponibilità di campioni): l'estrazione per l'altra classe viene riproporzionata di conseguenza per soddisfare il rapporto ρ , accettando la potenziale perdita o il mancato utilizzo di una quota di campioni della classe maggioritaria pur di preservare l'unicità e l'integrità statistica delle osservazioni.

Nei benchmark di tipo aggregato (sia sorgenti che ibridi), la composizione interna del sottoinsieme malevolo non adotta l'equiprobabilità artificiale tra le classi d'attacco. Si preserva invece la distribuzione naturale *a priori* $P(A = a \mid Y = 1)$ caratteristica dei domini di origine, evitando alterazioni sintetiche e mantenendo la fedeltà alle frequenze di occorrenza reali delle minacce.

³Nei sistemi NIDS reali, il traffico lecito costituisce la stragrande maggioranza del volume di rete. La parametrizzazione del rapporto emula tale asimmetria mantenendo una densità di attacchi sufficiente a garantire la stabilità statistica delle metriche.

L'intero processo di isolamento procedurale e ricomposizione dei benchmark è schematizzato nella Figura 3.1.

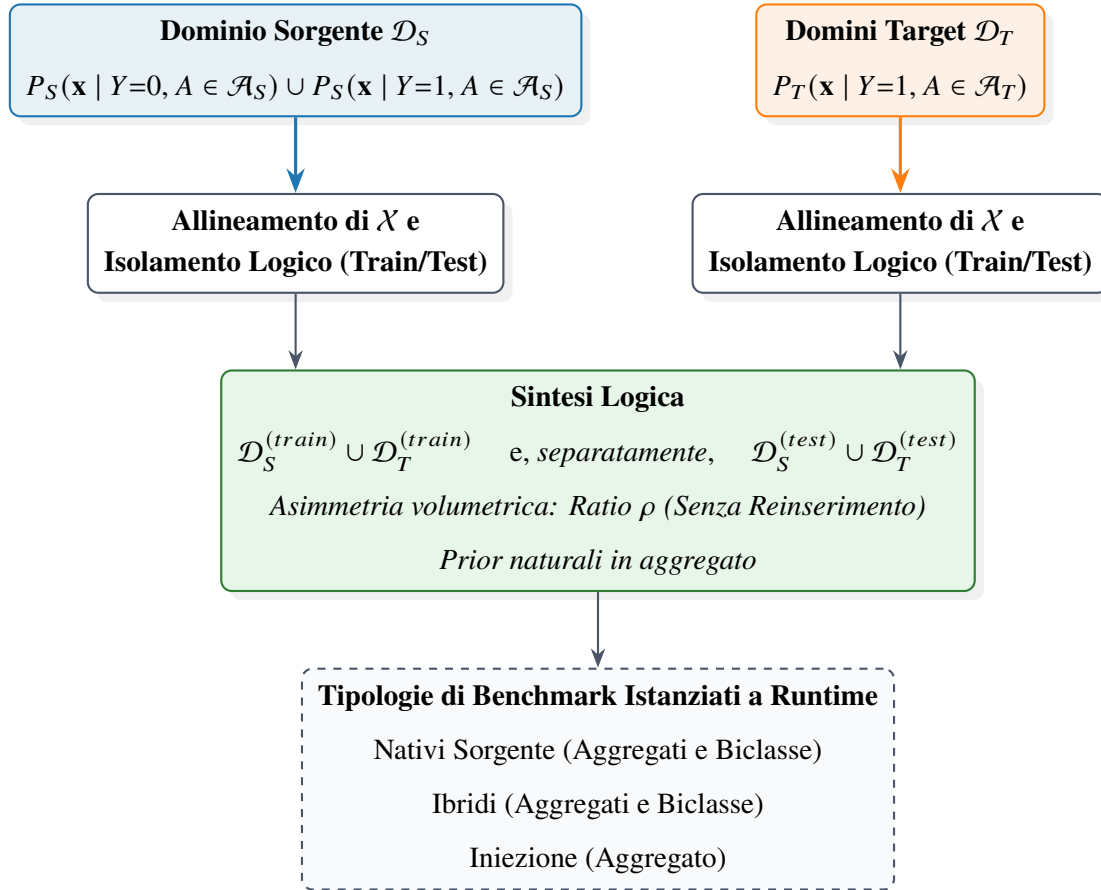


Figura 3.1: Pipeline logica astratta: allineamento dello spazio delle feature $\mathcal{X}$, isolamento preventivo Train/Test anti-*data leakage*, sintesi parametrica con ratio ρ senza reinserimento (con ancoraggio alla classe minoritaria) e istanziazione logica dei benchmark. I blocchi tratteggiati indicano le elaborazioni logiche, mentre i blocchi a linea continua rappresentano gli insiemi di dati e le categorie di output.

Tassonomia Astratta delle Configurazioni Sperimentali

L'applicazione sistematica delle regole metodologiche descritte porta alla generazione dinamica di cinque categorie funzionali di benchmark. Ognuna di queste configurazioni viene istanziata logicamente rispettando il preventivo isolamento logico delle partizioni di train e test e imponendo il vincolo di proporzionamento parametrico ρ tra traffico benevolo e malevolo:

1. **Set Nativo Sorgente Aggregato:** composto dalla baseline nominale $Y = 0$ di $\mathcal{D}_S$ unita alla totalità delle famiglie di attacco sorgenti $A \in \mathcal{A}_S$ (a valle della procedura di *minority removal*), preservando la distribuzione naturale a priori $P(A = a \mid Y = 1)$ caratteristica del dominio di origine.
2. **Set Nativi Sorgente Biclasse:** formati dall'unione della classe nominale $Y = 0$ con una singola famiglia di attacco $A = a \in \mathcal{A}_S$ isolata all'interno del dominio sorgente.
3. **Set Ibridi Aggregati:** composti dalla classe nominale $Y = 0$ del dominio sorgente unita alla totalità delle minacce dei domini di destinazione $A \in \mathcal{A}_T$ (strutturate tramite *minority merge*), preservando le frequenze *a priori* originali del dominio target.
4. **Set Ibridi Biclasse:** formati dalla combinazione della classe nominale $Y = 0$ sorgente con le singole famiglie d'attacco $A = a \in \mathcal{A}_T$ isolate all'interno dei domini target.
5. **Set di Iniezione di Riferimento:** progettato in conformità agli standard di letteratura [2], mantiene l'intera distribuzione nativa del dominio sorgente (sia il traffico nominale che l'aggregato malevolo di $\mathcal{D}_S$) integrandola con un'iniezione sparsa e controllata di vettori d'attacco provenienti dai domini target $\mathcal{D}_T$.

I dettagli analitici di composizione per ciascuna delle cinque categorie, opportunamente suddivisi per natura del traffico, sono riassunti nella Tabella 3.2.

Tabella 3.2: Tassonomia astratta delle cinque categorie di benchmark sperimentali istanziate dinamicamente.

Categoria Benchmark	Componente Benevola ($Y = 0$)	Componente Malevola ($Y = 1$)
Set Nativo Sorgente Aggregato	Baseline nativa $P_S(\mathbf{x} \mid Y = 0, A \in \mathcal{A}_S)$	Tutte le minacce sorgente $P_S(\mathbf{x} \mid Y = 1, A \in \mathcal{A}_S)$ con distribuzioni <i>a priori</i> naturali.
Set Nativo Sorgente Biclasse	Baseline nativa $P_S(\mathbf{x} \mid Y = 0, A \in \mathcal{A}_S)$	Singola minaccia sorgente $P_S(\mathbf{x} \mid Y = 1, A = a)$ isolata ($a \in \mathcal{A}_S$).
Set Ibrido Aggregato	Baseline nativa $P_S(\mathbf{x} \mid Y = 0, A \in \mathcal{A}_S)$	Tutte le minacce target $P_T(\mathbf{x} \mid Y = 1, A \in \mathcal{A}_T)$ con distribuzioni <i>a priori</i> naturali.
Set Ibrido Biclasse	Baseline nativa $P_S(\mathbf{x} \mid Y = 0, A \in \mathcal{A}_S)$	Singola minaccia target $P_T(\mathbf{x} \mid Y = 1, A = a)$ isolata ($a \in \mathcal{A}_T$).
Set di Iniezione di Riferimento	Baseline nativa $P_S(\mathbf{x} \mid Y = 0, A \in \mathcal{A}_S)$	Minacce sorgente in aggregato $P_S(\mathbf{x} \mid Y = 1, A \in \mathcal{A}_S)$ + iniezione altamente sparsa da $\mathcal{D}_T$ ($A \in \mathcal{A}_T$).

3.1.2 Allineamento dello Spazio delle Feature e Preprocessing Astratto

La disomogeneità strutturale tra le sonde di rilevamento e i formati di tracciamento adottati nei diversi ambienti di rete comporta un’asimmetria nativa negli spazi delle caratteristiche grezze. Per consentire la comparabilità cross-dominio, il framework definisce una procedura astratta di riallineamento spaziale e un impianto di trasformazione che mappa osservazioni eterogenee in uno spazio vettoriale comune, preservando la semantica trasmissiva e ottimizzando l’efficienza

concettuale. L'intero flusso di mappatura è sintetizzato in Figura 3.2.

Mappatura e Omogeneizzazione dello Spazio Vettoriale

Siano $\mathcal{X}_S^{(grezzo)} \subseteq \mathbb{R}^{d_S}$ e $\mathcal{X}_T^{(grezzo)} \subseteq \mathbb{R}^{d_T}$ gli spazi delle caratteristiche grezze associati rispettivamente al dominio sorgente e ai domini di destinazione. Per superare le differenze nomenclaturali, metriche e strutturali tra le d_S e d_T variabili native, si definisce, per ciascun dominio $k \in \{S, T\}$, un operatore di trasformazione e combinazione funzionale $\psi_k : \mathbb{R}^{d_k} \rightarrow \mathbb{R}^d$ tale per cui⁴:

$$\mathbf{x}_k = \psi_k(\mathbf{x}_k^{(grezzo)}) = \left(\psi_{k,1}(\mathbf{x}_k^{(grezzo)}), \psi_{k,2}(\mathbf{x}_k^{(grezzo)}), \dots, \psi_{k,d}(\mathbf{x}_k^{(grezzo)}) \right)^T \in \mathcal{X} \subseteq \mathbb{R}^d \quad (3.3)$$

Ciascuna componente $\psi_{k,j}$ ($j = 1, \dots, d$) rappresenta una combinazione algebrica o un'estrazione semantica costruita a partire dalle variabili primarie disponibili nel dominio k . Sebbene ψ_S e ψ_T siano definite separatamente per adattarsi alle rispettive variabili native grezze, entrambe sono vincolate a produrre uno spazio d'arrivo $\mathcal{X}$ condiviso e semanticamente equivalente, secondo i criteri di armonizzazione descritti di seguito. La formulazione consente di derivare un set di *feature* condiviso e omogeneo $\mathcal{X}$, indipendentemente dalle difformità applicative dei formati di origine.

Invarianza Concettuale e Filtraggio degli Attributi

La definizione dello spazio proiettato $\mathcal{X}$ risponde a rigorosi vincoli di filtraggio e armonizzazione basati su tre criteri fondamentali:

1. **Rimozione degli identificatori univoci e locali:** gli attributi legati all'identità specifica delle sessioni (es. indirizzi IP, marcatori temporali assoluti o porte di rete puntuali) vengono esclusi a priori per prevenire fenomeni di memorizzazione o correlazioni spurie (*shortcut learning*)⁵, garantendo che i modelli apprendano esclusivamente pattern trasmissivi e comportamentali generalizzabili.
2. **Vincoli di calcolabilità e misurabilità cross-dominio:** le variabili per le quali non sussiste la possibilità fisica o analitica di calcolo equivalente in entrambe le distribuzioni P_S e P_T

⁴La dimensione ridotta d costituisce una costante uniforme per l'intero framework.

⁵Nei contesti NIDS, lo *shortcut learning* si verifica quando il classificatore apprende associazioni mnemoniche tra identificatori locali rigidi (es. un IP specifico) e la classe malevola, perdendo qualsiasi capacità di generalizzare su topologie di rete non viste.

vengono omesse dal set condiviso. Ciò garantisce che ogni dimensione $x^{(j)} \in \mathcal{X}$ possieda un significato concettuale identico e direttamente confrontabile tra i domini.

3. **Armonizzazione delle unità di misura ed equivalenza dimensionale:** qualora una medesima metrica sia presente o ricavabile in entrambi i domini ma espressa mediante scala o unità di misura differenti, l'operatore ψ_k applica le necessarie trasformazioni dimensionali per ricondurla a una grandezza standard condivisa. L'uniformazione preventiva delle unità di misura garantisce che discrepanze puramente nominali nelle scale originarie non vengano erroneamente interpretate dai modelli come fenomeno di *domain shift*.

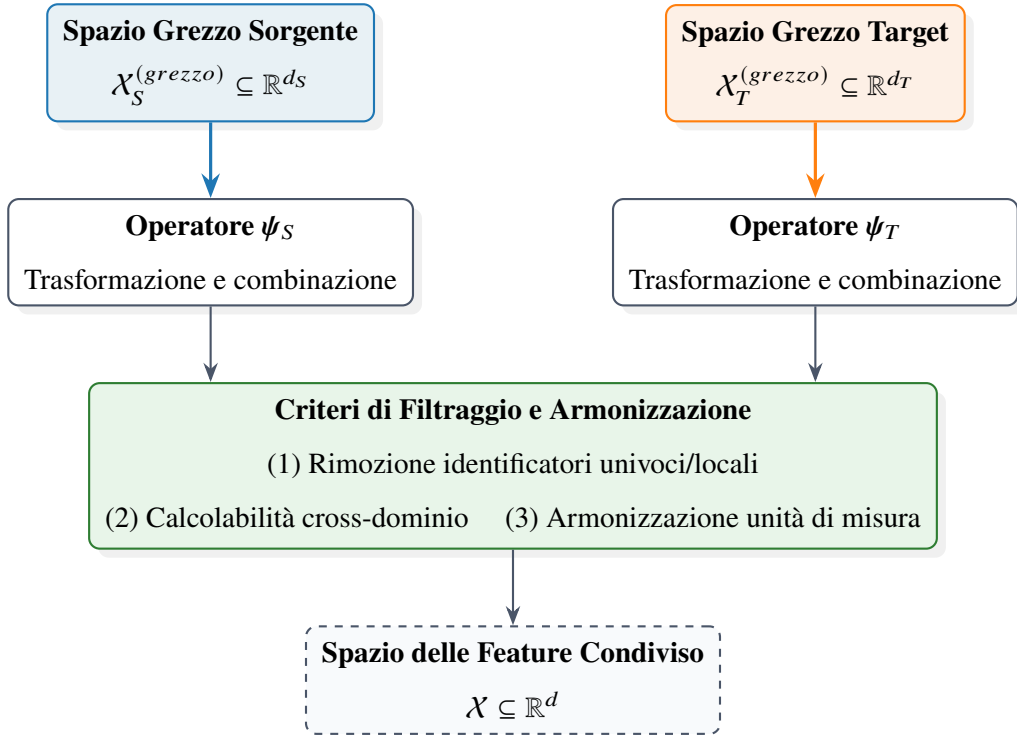


Figura 3.2: Flusso di mappatura e omogeneizzazione dello spazio delle *feature*: gli operatori ψ_S e ψ_T , definiti separatamente per ciascun dominio grezzo, convergono verso lo stesso spazio condiviso $\mathcal{X}$ attraverso i tre criteri comuni di filtraggio e armonizzazione descritti nel testo.

Invarianza Monotonica e Omissione della Scalatura Esplicita

La gestione della scala delle variabili è centrale. Sebbene le tecniche di riscaldamento lineare o di standardizzazione (Z-score, Min-Max scaling) siano ampiamente adottate in letteratura per

uniformare i domini di variabilità, il framework teorico sfrutta una nota proprietà algebrica delle famiglie di classificatori impiegate in questa tesi [4, 11].

Per la classe dei modelli basati sul partizionamento ricorsivo dello spazio dei dati⁶, la funzione di suddivisione $s(\mathbf{x})$ lungo ciascuna dimensione j poggia sull'ordinamento relativo dei valori rispetto a una soglia $\theta \in \mathbb{R}$:

$$s(\mathbf{x}) = \mathbb{I}(x^{(j)} \leq \theta) \quad (3.4)$$

Poiché l'indicatore di partizione $\mathbb{I}(\cdot)$ è rigorosamente invariante rispetto a qualsiasi trasformazione monotonica strettamente crescente $h : \mathbb{R} \rightarrow \mathbb{R}$, si verifica che:

$$\mathbb{I}(x^{(j)} \leq \theta) \iff \mathbb{I}(h(x^{(j)}) \leq h(\theta)) \quad (3.5)$$

Di conseguenza, la topologia dell'albero di decisione e la costruzione dei confini di separazione risultano algebricamente identiche sia operando sulle componenti nello spazio nativo $\mathbf{x}$, sia applicando una trasformazione di scala $h(\mathbf{x})$. Omettendo la fase di normalizzazione esplicita, il framework preserva la geometria naturale dei dati senza alterare la capacità discriminativa dei modelli, riducendo l'overhead computazionale nello stadio di parsificazione iniziale.

È opportuno precisare che tale omissione riguarda esclusivamente lo stadio di addestramento e inferenza dei classificatori. Qualora strumenti diagnostici non invarianti alla scala (es. *Principal Component Analysis* o stime di densità kernel, *KDE*) vengano impiegati a valle per l'ispezione visiva del *covariate shift* o per finalità di spiegabilità, essi richiedono una normalizzazione dedicata, applicata unicamente a scopo diagnostico-visivo e non reintrodotta nello stadio di classificazione.

3.2 Tassonomia delle Architetture di Classificazione e Valutazione Multi-Modello

La presente sezione formalizza la tassonomia delle architetture di classificazione impiegate all'interno del framework metodologico, delineando i criteri per la valutazione multi-modello e l'analisi

⁶Tale famiglia include architetture ad albero di decisione singolo e relative estensioni ad *ensemble*. La presente derivazione, e la conseguente omissione della scalatura esplicita, è valida esclusivamente per questa famiglia di modelli; qualora il framework venisse esteso a classificatori sensibili alla scala delle variabili (es. reti neurali, SVM a kernel, k -NN, regressione logistica regolarizzata), la normalizzazione andrebbe reintrodotta esplicitamente nello stadio di elaborazione iniziale.

prestazionale incrociata. L'intera architettura condivide una formulazione matematica unificata fondata sulla stima della probabilità a posteriori: subordinando l'assegnazione finale della classe a una soglia decisionale parametrica τ , viene garantita una base di confronto rigorosa, omogenea e strutturalmente indipendente dallo specifico algoritmo di apprendimento sottostante.

La trattazione procede definendo lo spazio dei modelli aggregati e le relative strategie di *transfer learning*, per poi introdurre la decomposizione in classificatori specialistici biclasse per l'isolamento analitico delle singole minacce e il relativo protocollo di valutazione incrociata (*cross-evaluation*). La sezione si conclude con la caratterizzazione formale delle famiglie di decisione alberiformi e dei paradigmi di *ensembling* (Bagging e Gradient Boosting), evidenziandone le proprietà algebriche di invarianza e robustezza rispetto alle distorsioni di canale.

Per agevolare la consultazione, i formalismi architetturali e i parametri funzionali introdotti in questa sezione sono riepilogati nella Tabella 3.3.

Tabella 3.3: Sintesi della notazione relativa alle architetture e ai parametri di classificazione.

Simbolo	Descrizione e Significato Concettuale
$f_{\theta}(\mathbf{x}) \approx P(Y = 1 \mid \mathbf{x})$	Funzione decisionale parametrica: stima della probabilità a posteriori che il campione appartenga alla classe malevola
$\hat{Y} \in \{0, 1\}$	Decisione binaria finale predetta dal classificatore
$\tau \in (0, 1)$	Soglia decisionale parametrica, tarabile lungo la curva ROC per il bilanciamento dei falsi positivi
$\mathcal{M}_{\text{base}}$	Modello baseline, addestrato sul <i>Set di Iniezione</i> $\mathcal{D}_{\text{inj}}^{(\text{train})}$ includendo conoscenza target sparsa
$\mathcal{M}_S$	Modello aggregato nativo sorgente, addestrato esclusivamente sulle distribuzioni stazionarie originarie $\mathcal{D}_S^{(\text{train})}$
$\mathcal{M}_H^{(T)}$	Modello aggregato ibrido (<i>Oracle</i>), addestrato unendo la componente lecita sorgente con la totalità degli attacchi target
$\mathcal{M}_{S,a}, \mathcal{M}_{H,T,a}$	Modelli biclasse (sorgente e ibridi), focalizzati analiticamente sulla rilevabilità della singola famiglia d'attacco $a \in \mathcal{A}$
$\mathcal{L}$	Funzione di perdita <i>Binary Cross-Entropy</i> (BCE) per l'ottimizzazione della stima parametrica
$I(R_m)$	Criterio di impurità (es. indice di Gini o entropia) per la scissione ottima dei nodi nello spazio delle caratteristiche

3.2.1 Modello di Riferimento (Baseline) e Modelli Aggregati

In questa sezione si formalizza lo spazio dei classificatori supervisionati operanti all'interno del framework analitico. Indipendentemente dalle specifiche architetture algoritmiche impiegate per la stima dei parametri, il problema di rilevamento delle intrusioni viene modellato in chiave rigorosamente binaria. Sebbene i sottoinsiemi malevoli racchiudano al loro interno un insieme eterogeneo di famiglie d'attacco $A \in \mathcal{A}$, la funzione di decisione appresa mappa lo spazio delle *feature* $\mathcal{X} \subseteq \mathbb{R}^d$ direttamente sulla natura del flusso $\mathcal{Y} = \{0, 1\}$, isolando le componenti lecite ($Y = 0$) da qualsiasi manifestazione anomala o minaccia ($Y = 1$).

Formulazione Matematica Astratta dell'Apprendimento Supervisionato

Sia $\mathcal{D}^{(\text{train})}$ una generica partizione di addestramento generata dalle procedure di sintesi descritte nella Sezione 3.1.1, definita come:

$$\mathcal{D}^{(\text{train})} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N, \quad \mathbf{x}_i \in \mathcal{X}, y_i \in \{0, 1\} \quad (3.6)$$

Un classificatore parametrico o non parametrico viene rappresentato astrattamente da una funzione $f_\theta : \mathcal{X} \rightarrow [0, 1]$, dove $f_\theta(\mathbf{x})$ stima la probabilità a posteriori $P(Y = 1 \mid \mathbf{x})$ dato il vettore delle caratteristiche $\mathbf{x}$.

Seguendo il principio di Minimizzazione del Rischio Empirico (ERM) proposto da Vapnik [35], l'addestramento si configura come la ricerca della configurazione parametrica θ^* che minimizza la perdita media valutata sul dataset di addestramento. Adottando la funzione di perdita canonica di *Binary Cross-Entropy* (BCE), definita per il singolo campione come:

$$\mathcal{L}(f_\theta(\mathbf{x}_i), y_i) = -[y_i \log f_\theta(\mathbf{x}_i) + (1 - y_i) \log (1 - f_\theta(\mathbf{x}_i))] \quad (3.7)$$

il problema di ottimizzazione assume la seguente formulazione analitica:

$$\theta^* = \arg \min_{\theta} \mathcal{R}_{\text{emp}}(\theta; \mathcal{D}^{(\text{train})}) = \arg \min_{\theta} \frac{1}{|\mathcal{D}^{(\text{train})}|} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}^{(\text{train})}} \mathcal{L}(f_\theta(\mathbf{x}_i), y_i) \quad (3.8)$$

Per la classe di modelli basati su alberi di decisione adottata nel framework, tale formalizzazione esprime la minimizzazione diretta dell'errore nel caso di ensemble additivi basati su boosting (es. *Gradient Boosted Decision Trees*), in cui l'ottimizzazione avviene tramite discesa funzionale del gradiente sulla loss BCE. Nel caso di foreste casuali (es. *Random Forest*), l'addestramento locale minimizza i criteri di impurità nei nodi e $f_\theta(\mathbf{x})$ rappresenta la frequenza relativa dei campioni malevoli nelle foglie, rendendo la BCE il criterio di misura globale dell'errore di calibrazione probabilistica.

La decisione binaria finale $\hat{Y} \in \{0, 1\}$ per un generico vettore di test viene determinata applicando la funzione indicatrice $\mathbb{I}(\cdot)$ rispetto a una soglia canonica di decisione τ^7 :

$$\hat{Y} = \mathbb{I}(f_{\theta^*}(\mathbf{x}) \geq \tau) \quad (3.9)$$

⁷Sebbene $\tau = 0.5$ sia neutro sotto costi simmetrici, nei contesti NIDS reali la soglia viene adattata lungo la curva ROC in funzione del trade-off tra sensibilità e falsi positivi. L'Area Under the ROC Curve (AUC) quantifica la capacità discriminativa complessiva indipendentemente dalla soglia.

Il Modello Baseline Proposto ($\mathcal{M}_{\text{base}}$)

Contrariamente agli approcci convenzionali della letteratura che adottano come baseline modelli addestrati su sole distribuzioni stazionarie native, il framework metodologico propone come unico modello baseline di riferimento ($\mathcal{M}_{\text{base}}$) un classificatore addestrato sulla partizione di train del *Set di Iniezione di Riferimento* ($\mathcal{D}_{\text{inj}}^{(\text{train})}$).

Questo modello definisce un'ancora di prestazione per contesti affetti da transizione trasmissiva (es. il passaggio da reti terrestri a canali satellitari). In tale ruolo, $\mathcal{M}_{\text{base}}$ fornisce un riferimento stabilizzante per la valutazione dei modelli e consente di confrontare le prestazioni lungo una traiettoria di cambiamento di canale reale. $\mathcal{M}_{\text{base}}$ è un sistema NIDS ad elevata capacità adattiva, in grado di generalizzare non solo su minacce non viste della medesima distribuzione, ma anche su distribuzioni eterogenee grazie alla conoscenza preventiva sparsa. Il vettore dei parametri ottimali θ_{base}^* viene ottenuto mediante:

$$\theta_{\text{base}}^* = \arg \min_{\theta} \mathcal{R}_{\text{emp}} \left(\theta; \mathcal{D}_{\text{inj}}^{(\text{train})} \right) \quad (3.10)$$

dove la composizione del dataset di addestramento incorpora la baseline unita all'iniezione target:

$$\mathcal{D}_{\text{inj}}^{(\text{train})} = \mathcal{D}_S^{(\text{train})} \cup \Delta \mathcal{D}_T^{(\text{train})} \quad (3.11)$$

con $\Delta \mathcal{D}_T^{(\text{train})}$ rappresentante il campionamento ridotto ed altamente sparso delle classi d'attacco ($Y = 1, A \in \mathcal{A}_T$)⁸.

In conformità al rigoroso protocollo di isolamento delle partizioni, il modello $\mathcal{M}_{\text{base}}$ viene inizialmente addestrato su $\mathcal{D}_{\text{inj}}^{(\text{train})}$ e validato sulla rispettiva partizione di test isolata $\mathcal{D}_{\text{inj}}^{(\text{test})}$, garantendo la riproducibilità e l'assenza di *data leakage*.

Classificatori Aggregati e Mappatura delle Distribuzioni

Per isolare in modo analitico l'impatto del *domain shift* e valutare le capacità di trasferimento della conoscenza, il framework affianca alla baseline proposta due categorie di modelli aggregati multi-classe (ma a decisione binaria):

⁸Matematicamente, la condizione di sparsità è definita dal vincolo cardinale $|\Delta \mathcal{D}_T^{(\text{train})}| \ll |\mathcal{D}_S^{(\text{train})}|$, emulando uno scenario di *few-shot learning* in cui la visibilità sul canale di destinazione è limitata a pochissimi campioni rappresentativi.

1. **Modello Aggregato Nativo Sorgente ($\mathcal{M}_S$):** addestrato esclusivamente sulla partizione di addestramento del dominio sorgente nativo ($\mathcal{D}_S^{(\text{train})}$). Rappresenta il sistema NIDS tradizionale addestrato su sole distribuzioni stazionarie:

$$\theta_S^* = \arg \min_{\theta} \mathcal{R}_{\text{emp}} \left(\theta; \mathcal{D}_S^{(\text{train})} \right) \quad (3.12)$$

La valutazione di $\mathcal{M}_S$ sui benchmark target consente di misurare la pura degradazione prestazionale causata dalle distorsioni di canale senza alcuna forma di adattamento.

2. **Modelli Aggregati Ibridi ($\mathcal{M}_H^{(T)}$):** per ciascun contesto di destinazione T , viene istanziata un'opportuna variante di modello ibrido $\mathcal{M}_H^{(T)}$ addestrata sulla relativa partizione di addestramento ricomposta ($\mathcal{D}_{H,T}^{(\text{train})}$):

$$\mathcal{D}_{H,T}^{(\text{train})} = \mathcal{D}_S^{(\text{train})} \Big|_{Y=0} \cup \mathcal{D}_T^{(\text{train})} \quad (3.13)$$

in cui la classe nominale $Y = 0$ di origine è unita alla totalità dei vettori d'attacco $Y = 1$ del contesto di destinazione ($\mathcal{D}_T^{(\text{train})}$). Il rispettivo vettore di parametri ottimali risponde alla formulazione:

$$\theta_{H,T}^* = \arg \min_{\theta} \mathcal{R}_{\text{emp}} \left(\theta; \mathcal{D}_{H,T}^{(\text{train})} \right) \quad (3.14)$$

Tali modelli formalizzano il limite teorico di rilevamento (*upper bound*) raggiungibile quando il classificatore acquisisce piena visibilità della componente malevola nel dominio target⁹.

⁹Nella letteratura del trasferimento di dominio, $\mathcal{M}_H^{(T)}$ risponde al paradigma dell'*Oracle supervised*: fornisce la massima accuratezza raggiungibile se si disponesse del dataset target integralmente etichettato.

Tabella 3.4: Sintesi comparativa dei modelli di riferimento e aggregati: composizione del training set e finalità concettuale.

Modello	Composizione del <i>Training Set</i>	Finalità Concettuale
$\mathcal{M}_{\text{base}}$	$\mathcal{D}_{\text{inj}}^{(\text{train})} = \mathcal{D}_S^{(\text{train})} \cup \Delta \mathcal{D}_T^{(\text{train})}$	Baseline proposta: ancora di prestazione con conoscenza preventiva sparsa (<i>few-shot</i>) sul canale target.
$\mathcal{M}_S$	$\mathcal{D}_S^{(\text{train})}$	NIDS tradizionale addestrato su sole distribuzioni stazionarie; misura la degradazione da <i>domain shift</i> senza adattamento.
$\mathcal{M}_H^{(T)}$	$\mathcal{D}_{H,T}^{(\text{train})} = \mathcal{D}_S^{(\text{train})} _{Y=0} \cup \mathcal{D}_T^{(\text{train})}$	Limite teorico (<i>upper bound</i> , paradigma <i>Oracle supervised</i>) con piena visibilità della componente malevola target.

Conservazione dei Modelli, Inferenza e Disaggregazione delle Prestazioni

Al completamento della fase di ottimizzazione parametrica e dopo la verifica della convergenza sui rispettivi set di test locali, tutti i modelli generati ($\mathcal{M}_{\text{base}}$, $\mathcal{M}_S$ e le istanze $\mathcal{M}_H^{(T)}$) vengono mantenuti fissi nella propria struttura.

Il protocollo di valutazione per tali classificatori, schematizzato in Figura 3.3 secondo la notazione standard a diagramma di flusso (*flowchart*)¹⁰, segue un doppio binario analitico:

1. **Valutazione di Generalizzazione Globale:** ciascun modello immutabile viene proiettato sulle partizioni di test eterogenee $\mathcal{D}^{(\text{test})}$, consentendo di quantificare la tolleranza al cambio di canale e la capacità di generalizzazione rispetto a minacce target mai osservate in addestramento;

¹⁰A differenza degli schemi architetturali e compositivi delle Figure 3.1 e 3.2, che rappresentano relazioni statiche tra insiemi di dati, la Figura 3.3 e la successiva Figura 3.4 descrivono un protocollo procedurale con un autentico punto di biforcazione (il confronto $f_{\theta^*}(\mathbf{x}) \geq \tau$). Per tale ragione si adotta qui la notazione a diagramma di flusso convenzionale (nodi ellittici per gli insiemi di dati, rombi per i punti di decisione, rettangoli per i processi), coerentemente con la prassi diffusa in letteratura per la rappresentazione di protocolli algoritmici a bivi condizionali.

2. **Disaggregazione delle Prestazioni per Classe:** i modelli aggregati vengono contestualmente valutati sulle singole sottopartizioni per famiglia d’attacco $\mathcal{D}_a^{(\text{test})} \subseteq \mathcal{D}^{(\text{test})}$, ristrette ai soli vettori con $A = a$ per una data classe di minaccia $a \in \mathcal{A}$. Tale analisi disaggregata evita l’effetto di mascheramento delle prestazioni (*class shadowing*), verificando se un’elevata accuratezza globale celi un decadimento selettivo su specifiche classi di minaccia.

È fondamentale sottolineare che l’insieme di test $\mathcal{D}^{(\text{test})}$ non rappresenta un bacino dati specifico e isolato, locale alla sola famiglia dei modelli aggregati. Al contrario, esso coincide formalmente con l’insieme universale condiviso dei benchmark $\mathcal{B}$, che verrà rigorosamente definito nella Sezione 3.3.1. Di conseguenza, l’intero pool di modelli (siano essi aggregati o biclasse) viene proiettato sistematicamente sull’intera totalità di $\mathcal{B}$, garantendo un incrocio analitico esaustivo tra tutte le architetture e tutte le partizioni sperimentali.

Si precisa che la formalizzazione dei modelli biclasse specialistici e il relativo protocollo di *cross-evaluation* speculare sono trattati separatamente nella Sezione 3.2.2.

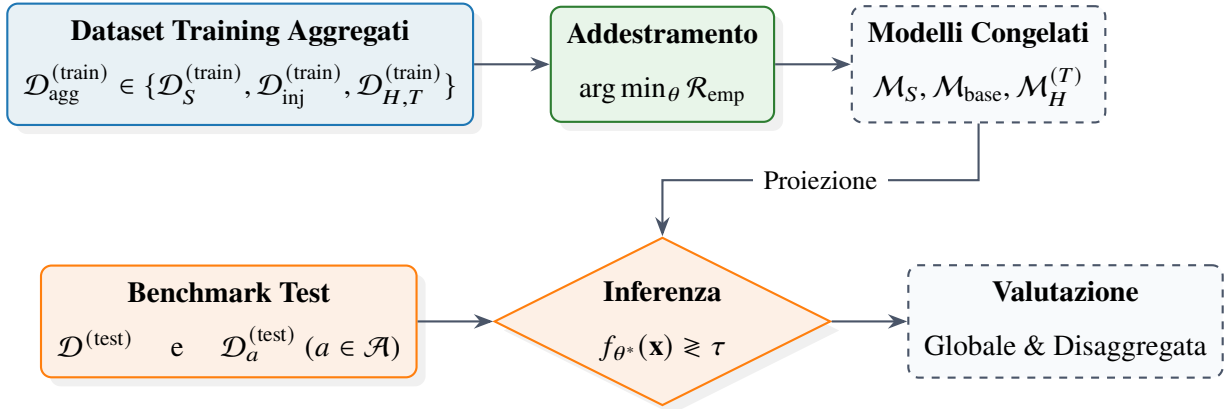


Figura 3.3: Flusso concettuale del protocollo di addestramento, inferenza e disaggregazione delle prestazioni per i modelli aggregati ($\mathcal{M}_{\text{base}}, \mathcal{M}_S, \mathcal{M}_H^{(T)}$). I modelli, congelati post-ottimizzazione, vengono valutati sia sull’intero benchmark multi-classe per misurare la generalizzazione di dominio, sia sulle singole partizioni per famiglia d’attacco $\mathcal{D}_a^{(\text{test})}$ per l’analisi di disaggregazione puntuale.

3.2.2 Decomposizione Specialistica Biclasse

In integrazione ai modelli aggregati definiti nella Sezione 3.2.1, il framework metodologico prevede una formalizzazione specialistica basata sulla decomposizione del problema di rilevamento in

classificatori biclasse focalizzati su singole famiglie d'attacco $a \in \mathcal{A}$. L'obiettivo primario di tale decomposizione non consiste nella riconfigurazione dell'architettura di inferenza, bensì nell'isolamento analitico delle singole tipologie di minaccia per valutarne la rilevabilità puntuale rispetto al profilo di traffico nominale e confrontarne direttamente le prestazioni con le rispettive varianti multi-classe¹¹.

Sintesi e Bilanciamento delle Partizioni Biclasse

Ciascun dataset biclasse viene generato rispettando rigorosamente il medesimo protocollo di partizionamento, il rapporto di suddivisione train/test e il bilanciamento tra componente lecita e malevola formalizzati nella Sezione 3.1.1. Per ogni specifica famiglia d'attacco a , sia la partizione di addestramento che quella di test comprendono:

1. una frazione di traffico lecito ($Y = 0$), costantemente estratta dalla distribuzione sorgente nativa $\mathcal{D}_S$;
2. una frazione di traffico anomalo ($Y = 1$), ristretta esclusivamente ai vettori associati alla specifica classe di minaccia a .

Tassonomia dei Modelli Biclasse

A differenza della caratterizzazione adottata per i classificatori aggregati, la famiglia dei modelli biclasse esclude la formulazione di architetture basate su iniezione sparsa (ovvero non è definita alcuna variante biclasse analoga a $\mathcal{M}_{\text{base}}$)¹². Lo spazio dei modelli biclasse si articola pertanto nelle seguenti due categorie:

1. **Modelli Biclasse Sorgente ($\mathcal{M}_{S,a}$):** specializzati sulle singole famiglie d'attacco $a \in \mathcal{A}_S$ appartenenti al dominio sorgente nativo. La relativa partizione di addestramento $\mathcal{D}_{S,a}^{(\text{train})}$ è

¹¹La decomposizione introduce un costo computazionale di addestramento proporzionale alla cardinalità del set delle minacce $O(|\mathcal{A}|)$. Tale overhead rappresenta un compromesso metodologico consapevole, poiché l'obiettivo primario della formulazione risiede nell'isolamento analitico della rilevabilità e non nell'ottimizzazione dell'efficienza.

¹²L'assenza di modelli biclasse con iniezione deriva dalla volontà di valutare la risposta specialistica o nella condizione di assoluta assenza di adattamento (sorgente pura) o nella condizione di visibilità integrale della minaccia nel dominio target (ibrido/oracle).

definita come:

$$\mathcal{D}_{S,a}^{(\text{train})} = \mathcal{D}_S^{(\text{train})} \Big|_{Y=0} \cup \mathcal{D}_S^{(\text{train})} \Big|_{Y=1, A=a} \quad (3.15)$$

Il vettore ottimale dei parametri $\theta_{S,a}^*$ viene determinato mediante la minimizzazione del rischio empirico ristretto al dominio dell'attacco a :

$$\theta_{S,a}^* = \arg \min_{\theta} \mathcal{R}_{\text{emp}} \left(\theta; \mathcal{D}_{S,a}^{(\text{train})} \right) \quad (3.16)$$

2. **Modelli Biclasse Ibridi/Target ($\mathcal{M}_{H,T,a}$):** istanziati per ciascuna sotto-distribuzione target T e per ciascuna specifica famiglia d'attacco $a \in \mathcal{A}_T$ presente nel dominio di destinazione. La composizione del dataset ricomposto $\mathcal{D}_{H,T,a}^{(\text{train})}$ adotta la componente lecita sorgente unita alla sola classe malevola a del contesto target:

$$\mathcal{D}_{H,T,a}^{(\text{train})} = \mathcal{D}_S^{(\text{train})} \Big|_{Y=0} \cup \mathcal{D}_T^{(\text{train})} \Big|_{Y=1, A=a} \quad (3.17)$$

con la corrispondente stima parametrica fornita da:

$$\theta_{H,T,a}^* = \arg \min_{\theta} \mathcal{R}_{\text{emp}} \left(\theta; \mathcal{D}_{H,T,a}^{(\text{train})} \right) \quad (3.18)$$

Le caratteristiche essenziali e le finalità concettuali delle due categorie di modelli biclasse sono sintetizzate nella Tabella 3.5.

Tabella 3.5: Sintesi dei modelli biclasse specializzati: composizione del *training set* e finalità concettuale.

Modello	Composizione del <i>Training Set</i>	Finalità Concettuale
$\mathcal{M}_{S,a}$	$\mathcal{D}_{S,a}^{(\text{train})} = \mathcal{D}_S^{(\text{train})} \Big _{Y=0} \cup \mathcal{D}_S^{(\text{train})} \Big _{Y=1, A=a}$	Rilevatore specialistico addestrato su una singola minaccia sorgente $a \in \mathcal{A}_S$; valuta la selettività senza distorsioni di canale.
$\mathcal{M}_{H,T,a}$	$\mathcal{D}_{H,T,a}^{(\text{train})} = \mathcal{D}_S^{(\text{train})} \Big _{Y=0} \cup \mathcal{D}_T^{(\text{train})} \Big _{Y=1, A=a}$	Limite teorico (<i>Oracle</i> specialistico) per la singola minaccia $a \in \mathcal{A}_T$ nel dominio target T .

Invarianza dell’Inferenza, Valutazione Incrociata e Persistenza

Il meccanismo decisionale dei classificatori biclasse rimane del tutto identico a quello formalizzato nell’Equazione (3.9): la funzione appresa $f_{\theta^*,a}(\mathbf{x})$ stima la probabilità a posteriori $P(Y = 1 \mid \mathbf{x})$, e la decisione finale $\hat{Y} \in \{0, 1\}$ viene determinata dal confronto con la soglia canonica τ .

Poiché l’output di classificazione è limitato al dominio binario $\mathcal{Y} = \{0, 1\}$ (presenza o assenza di intrusione), ciascun modello biclasse è strutturalmente autocontenuto e indipendente. Non viene applicata alcuna regola di aggregazione d’insieme (es. sistemi di voto o regole di massimo); i modelli rimangono invariati nello stato post-addestramento, rispecchiando la procedura di conservazione definita per i modelli aggregati.

In fase di valutazione sperimentale, il protocollo per i classificatori biclasse, schematizzato in Figura 3.4 secondo la medesima notazione a diagramma di flusso introdotta in Figura 3.3, segue due direttive analitiche complementari:

1. **Valutazione Nominale Specialistica (*In-Domain*):** ciascun modello $\mathcal{M}_{S,a}$ (o $\mathcal{M}_{H,T,a}$) viene inizialmente testato sulla partizione dedicata alla singola minaccia $\mathcal{D}_a^{(\text{test})}$, stabilendo la prestazione ideale del classificatore nel proprio dominio di addestramento primario;
2. **Valutazione Incrociata di Classe (*Cross-Evaluation*):** il modello specialistico viene successivamente proiettato sull’intera gamma di benchmark multi-classe eterogenei $\mathcal{D}^{(\text{test})}$. Tale analisi permette di verificare se la firma appresa per la minaccia a generi falsi allarmi su attacchi $b \neq a$, quantificando la selettività di classe e l’eventuale polarizzazione dello spazio decisionale.

Come anticipato per la famiglia dei modelli aggregati, l’insieme eterogeneo $\mathcal{D}^{(\text{test})}$ impiegato nella *cross-evaluation* coincide esattamente con l’insieme universale dei benchmark $\mathcal{B}$ (Sezione 3.3.1). In quella sede verrà introdotta la matrice strutturata $H_k \in \mathbb{R}^{|\mathcal{M}| \times |\mathcal{B}|}$, in cui verranno incrociati tutti i modelli formulati $\{\mathcal{M}_{\text{base}}, \mathcal{M}_S, \mathcal{M}_H^{(T)}, \mathcal{M}_{S,a}, \mathcal{M}_{H,T,a}\}$ con l’intera estensione dei domini di test generati, fornendo così la rappresentazione visiva e matematica unificata che concretizza questo incrocio prestazionale.

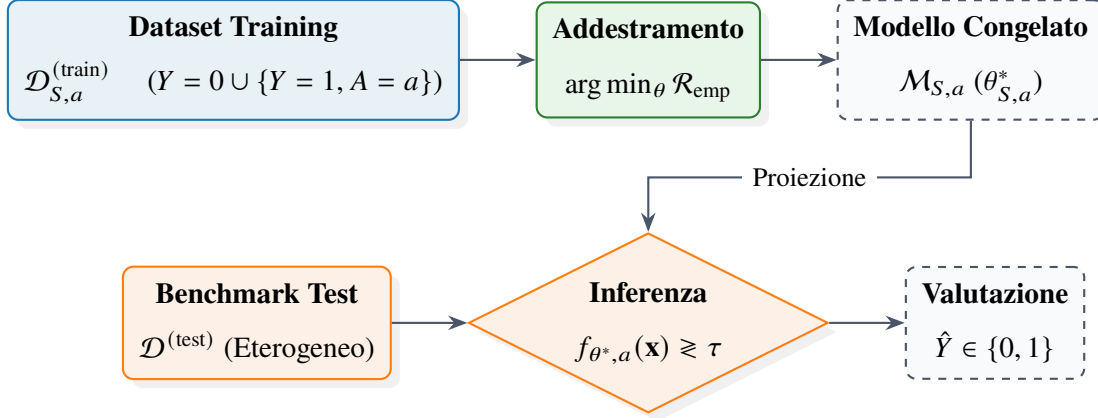


Figura 3.4: Flusso concettuale del protocollo di valutazione incrociata (*cross-evaluation*). Il modello specialistico $\mathcal{M}_{S,a}$, addestrato esclusivamente sulla singola minaccia a , rimane invariato e viene proiettato in fase di inferenza sull'intero benchmark eterogeneo $\mathcal{D}^{(test)}$.

3.2.3 Spazio delle Famiglie di Decisione Alberiformi ed Ensembling

La modellazione della superficie di decisione nei sistemi di *Network Intrusion Detection* (NIDS) operanti su canali ibridi terrestri-spaziali richiede architetture di apprendimento capaci di gestire elevata non-linearità, eterogeneità delle metriche e degradazioni del segnale trasmissivo. Questa sezione formalizza lo spazio delle famiglie di decisione basate sul partizionamento dello spazio delle caratteristiche, analizzandone le proprietà di invarianza e l'estensione a schemi di stima aggregata (*ensembling*).

Partizionamento Ricorsivo dello Spazio delle Caratteristiche

Sia $\mathcal{X} \subseteq \mathbb{R}^d$ lo spazio delle caratteristiche d -dimensionale rappresentativo delle metriche di traffico di rete e telemetria, e sia $\mathcal{Y} = \{0, 1\}$ lo spazio delle etichette corrispondente a traffico lecito o malevolo. Un albero di decisione [4] definisce una mappa $f : \mathcal{X} \rightarrow [0, 1]$ mediante un partizionamento ricorsivo dello spazio $\mathcal{X}$ in M regioni disgiunte e iper-rettangolari $\{R_m\}_{m=1}^M$, tali per cui:

$$\bigcup_{m=1}^M R_m = \mathcal{X} \quad \text{e} \quad R_m \cap R_k = \emptyset, \quad \forall m \neq k \quad (3.19)$$

La funzione di decisione associata all'albero assume la forma analitica espressa da:

$$f(\mathbf{x}) = \sum_{m=1}^M c_m \mathbb{I}(\mathbf{x} \in R_m) \quad (3.20)$$

dove $\mathbb{I}(\cdot)$ denota la funzione indicatrice e $c_m \in [0, 1]$ rappresenta la stima della probabilità a posteriori della classe malevola all'interno della regione R_m , coerentemente con la formulazione $f_\theta : \mathcal{X} \rightarrow [0, 1]$ introdotta nella Sezione 3.2.1. Sulla base del dataset di addestramento $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, tale stima è definita come la frequenza relativa empirica della classe positiva nella foglia:

$$c_m = \frac{1}{|R_m|} \sum_{\mathbf{x}_i \in R_m} \mathbb{I}(y_i = 1) \quad (3.21)$$

La decisione binaria finale $\hat{Y}$ viene poi ottenuta applicando la soglia canonica τ già formalizzata nell'Equazione (3.9), in coerenza con l'intero framework di ottimizzazione ERM/BCE.

La selezione delle soglie ottimali di split θ_j per la j -esima caratteristica x_j avviene ricorsivamente minimizzando un criterio di impurità $I(R_m)$, come l'indice di Gini o l'entropia cross-entropica [4]. Formalmente, in un task di classificazione binaria, tali criteri sono espressi in funzione della probabilità stimata c_m come:

$$I_{\text{Gini}}(R_m) = 2c_m(1 - c_m), \quad I_{\text{entropy}}(R_m) = -c_m \log_2(c_m) - (1 - c_m) \log_2(1 - c_m) \quad (3.22)$$

Proprietà di Invarianza e Tolleranza alle Distorsioni di Canale

Gli alberi di decisione esibiscono proprietà algebriche e geometriche fondamentali che li rendono intrinsecamente robusti alle perturbazioni introdotte dai canali trasmissivi radiofrequenza (RF) e dalle perdite di pacchetto.

Invarianza alle Trasformazioni Monotone Come introdotto nella Sezione 3.1.2 a giustificazione dell'omissione della fase di scalatura esplicita, le metriche estratte dai canali spaziali possono subire variazioni di scala, attenuazioni o calibrazioni non-lineari. La proprietà viene qui derivata in forma rigorosa e generalizzata a trasformazioni specifiche per singola caratteristica. Sia $h_j : \mathbb{R} \rightarrow \mathbb{R}$ una trasformazione strettamente monotonicamente crescente applicata alla caratteristica x_j . Poiché la decisione di uno split ad un generico nodo dell'albero è governata dalla funzione di soglia $\mathbb{I}(x_j > \theta_j)$, si osserva che:

$$x_j > \theta_j \iff h_j(x_j) > h_j(\theta_j) \quad (3.23)$$

Definendo la nuova soglia $\theta'_j = h_j(\theta_j)$, l'ordinamento topologico dei punti all'interno dello spazio $\mathcal{X}$ rimane inalterato, garantendo l'invarianza della partizione R_m e della decisione $f(\mathbf{x})$:

$$f(\mathbf{x}) = f(h_1(x_1), h_2(x_2), \dots, h_d(x_d)) \quad (3.24)$$

Tale proprietà elimina la necessità di normalizzazioni rigide (es. z-score o Min-Max) che risulterebbero instabili sotto condizioni di variabilità dinamica del canale.

Robustezza a Rumore Additivo e Distorsione Si consideri una misura corrotta da distorsione o rumore di canale $\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\eta}$, dove $\boldsymbol{\eta}$ rappresenta una componente stocastica con $\mathbb{E}[\boldsymbol{\eta}] = \mathbf{0}$. Poiché le regioni R_m sono delimitate da iperpiani di decisione paralleli agli assi coordinati, la classificazione del vettore perturbato $\tilde{\mathbf{x}}$ risulta invariante rispetto al vettore nominale $\mathbf{x}$ per qualsiasi perturbazione limitata che non attraversi il confine dell'iperpiano:

$$f(\tilde{\mathbf{x}}) = f(\mathbf{x}), \quad \forall \boldsymbol{\eta} \text{ t.c. } \text{sgn}(x_j + \eta_j - \theta_j) = \text{sgn}(x_j - \theta_j) \quad (3.25)$$

Questa natura a scalino (*step function*) riduce la sensibilità al rumore gaussiano e alle fluttuazioni di piccola entità rispetto ai modelli a separazione lineare o a base di distanza euclidea.

Gestione Nativa dei Dati Mancanti (*Packet Loss*) In presenza di perdite di pacchetti o corruzione dei dati di header, alcune caratteristiche x_j possono risultare non disponibili ($x_j = \text{NaN}$). Le strutture alberiformi gestiscono tale incompletezza mediante due meccanismi distinti proposti in letteratura, entrambi privi della necessità di imputazione esplicita del dato: gli *split surrogati*, introdotti nella formulazione originaria di CART [4], che individuano uno split alternativo su una caratteristica correlata a x_j qualora quest'ultima risulti mancante; e l'instradamento a *direzione predefinita* (*default direction*), proposto nel contesto degli algoritmi di boosting scalabili [5], in cui la direzione ottimale $\delta_j^* \in \{\text{sinistra}, \text{destra}\}$ per ciascun nodo di split su x_j viene appresa direttamente durante l'addestramento, minimizzando la perdita empirica sui pattern di sparsità osservati:

$$\text{Posizione}(\mathbf{x}) = \begin{cases} \text{Sinistra} & \text{se } x_j \leq \theta_j \\ \text{Destra} & \text{se } x_j > \theta_j \\ \delta_j^* & \text{se } x_j = \text{NaN} \end{cases} \quad (3.26)$$

Formulazione Generale degli Ensemble per la Riduzione dell'Errore

Sebbene il singolo albero di decisione presenti un basso errore di bias, esso soffre di elevata varianza. Per mitigare tale limite, si ricorre a tecniche di *ensembling* che combinano B stimatori base $\{f_b(\mathbf{x})\}_{b=1}^B$ per formare un predittore aggregato $F(\mathbf{x})$:

$$F(\mathbf{x}) = \sum_{b=1}^B \beta_b f_b(\mathbf{x}) \quad (3.27)$$

Questa astrazione unifica i due principali approcci¹³: l'addestramento parallelo per la riduzione della varianza e l'addestramento sequenziale per la riduzione del bias, le cui differenze chiave sono schematizzate in Tabella 3.6 e in Figura 3.5.

Riduzione della Varianza via Bagging (*Bootstrap Aggregating*) Nello schema Bagging, di cui le architetture *Random Forest* [3] costituiscono l'estensione fondamentale, ciascun stimatore $f_b(\mathbf{x})$ viene addestrato in modo indipendente su un campionamento bootstrap del dataset $\mathcal{D}_b^*$, introducendo un'ulteriore de-correlazione mediante la selezione casuale di un sottoinsieme di $k < d$ caratteristiche a ciascun nodo.

Sia σ^2 la varianza del singolo albero e ρ il coefficiente di correlazione campionaria medio tra le predizioni di due alberi distinti. La varianza dello stimatore mediato $\bar{f}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B f_b(\mathbf{x})$ si esprime analiticamente come [11]:

$$\text{Var}(\bar{f}(\mathbf{x})) = \rho \sigma^2 + \frac{1 - \rho}{B} \sigma^2 \quad (3.28)$$

Al crescere del numero di alberi $B \rightarrow \infty$, il secondo termine tende a zero, riducendo la varianza complessiva del modello al limite teorico $\rho \sigma^2$. La de-correlazione forzata delle caratteristiche riduce il fattore ρ , garantendo un'elevata capacità di generalizzazione anche su traffico malevolo mai osservato.

Riduzione del Bias via Gradient Boosting A differenza dell'approccio parallelo del Bagging, gli algoritmi di *Gradient Boosting* [10] (incluso il paradigma con discretizzazione a istogrammi

¹³Come formalizzato nei paragrafi seguenti, nel caso del Bagging si assegna un peso uniforme $\beta_b = 1/B$, mentre la ricorsione additiva del Boosting, se srotolata su B passi, si riconduce a questa formulazione ponendo $\beta_b = \nu$ (a meno di un predittore costante iniziale F_0).

Histogram-based Gradient Boosting [14]) costruiscono l'ensemble in modo sequenziale ed additivo. Dato un modello al passo $b - 1$ pari a $F_{b-1}(\mathbf{x})$, lo stimatore successivo $f_b(\mathbf{x})$ viene addestrato per approssimare i pseudo-residui $r_{i,b}$, definiti come il gradiente negativo della funzione di perdita differenziabile $L(y_i, F(\mathbf{x}_i))$ valutato sulla predizione corrente:

$$r_{i,b} = - \left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F(\mathbf{x}_i)=F_{b-1}(\mathbf{x}_i)} \quad (3.29)$$

L'aggiornamento del modello aggregato avviene secondo la regola:

$$F_b(\mathbf{x}) = F_{b-1}(\mathbf{x}) + \nu \cdot f_b(\mathbf{x}) \quad (3.30)$$

dove $\nu \in (0, 1]$ rappresenta il tasso di apprendimento (*learning rate*). Tale approccio garantisce una minimizzazione progressiva del bias dell'errore di classificazione, consentendo di catturare sottili pattern di attacco ad alta dimensionalità tipici delle minacce cibernetiche complesse.

Tabella 3.6: Confronto delle proprietà architetture e degli obiettivi di ottimizzazione tra i paradigmi di *ensembling* Bagging e Gradient Boosting.

Caratteristica	Bagging	Gradient Boosting
Costruzione	Parallela, stimatori indipendenti	Sequenziale, additiva
Obiettivo primario	Riduzione della varianza	Riduzione del bias
Combinazione	Media: $\bar{f} = \frac{1}{B} \sum f_b$	Somma pesata: $F_b = F_{b-1} + \nu f_b$
Target addestramento per f_b	Etichette originali su campionamenti bootstrap $\mathcal{D}_b^*$	Pseudo-residui $r_{i,b}$ valutati al passo $b - 1$

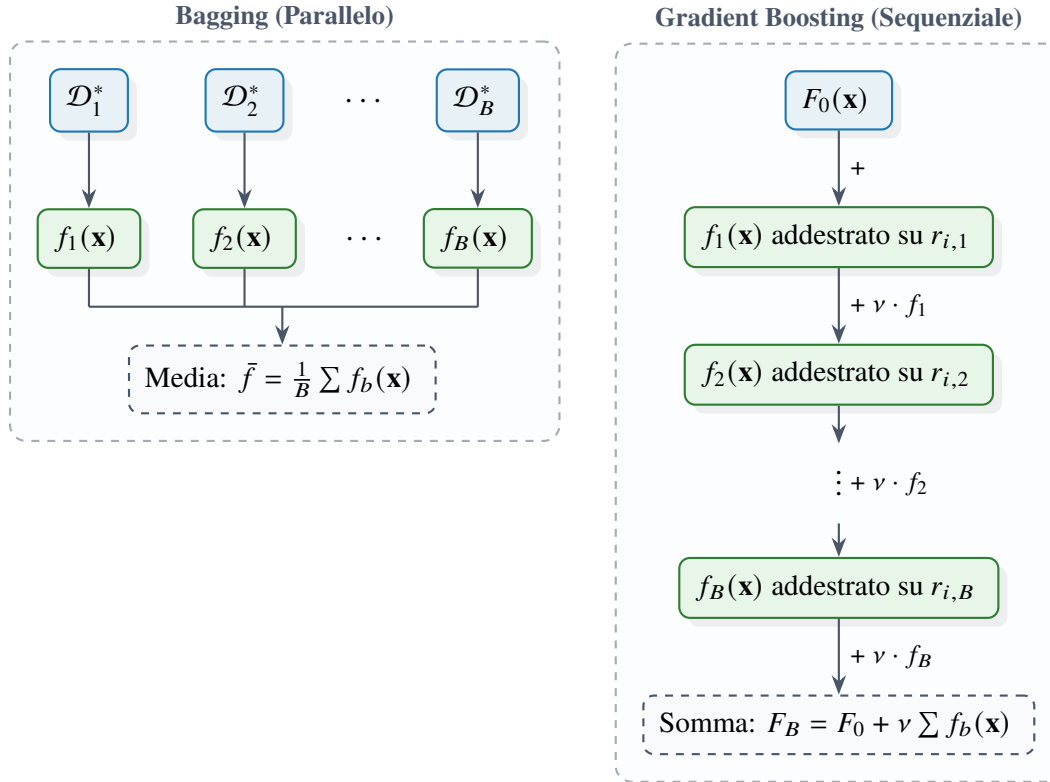


Figura 3.5: Confronto logico-procedurale tra gli schemi di aggregazione. A sinistra, il paradigma *Bagging* esegue un addestramento parallelo su campionamenti bootstrap indipendenti. A destra, il paradigma *Gradient Boosting* adotta un approccio sequenziale in cui ciascun albero mira a correggere i residui iterativi generati dall'insieme precedente.

3.3 Framework di Adattamento, Granularità e Stabilità Statistica

La valutazione dell'efficacia delle architetture e delle strategie di *transfer learning* precedentemente formalizzate richiede un apparato metodologico capace di isolare i reali guadagni prestazionali dal rumore stocastico intrinseco ai processi di apprendimento. Questa sezione definisce il framework di valutazione e di analisi teorica adottato per garantire la solidità e la riproducibilità dei risultati.

Nella prima parte della trattazione viene affrontato il problema dell'incertezza algoritmica legata ai modelli ad albero, introducendo un rigoroso protocollo di valutazione *multi-seed*. Vengono definite le metriche prestazionali di aggregazione e viene formalizzato l'impiego del test t di

Welch per stabilire oggettivamente la significatività statistica degli scostamenti osservati rispetto al modello di riferimento (*baseline*). Successivamente, l'analisi si sposta sulle proprietà strutturali e geometriche dello spazio decisionale. Viene introdotto l'indice di granularità per valutare l'impatto della formulazione del problema — differenziando tra partizioni a grana grossa e a grana fine — sulla morfologia dei confini decisionali, discutendo il compromesso teorico tra rigidità specialistica e tolleranza al *domain shift*.

Per agevolare la consultazione delle metriche e dei parametri statistici discussi, la notazione principale introdotta in questa sezione è riepilogata nella Tabella 3.7.

Tabella 3.7: Sintesi della notazione relativa al framework di valutazione e alla stabilità statistica.

Simbolo	Descrizione e Significato Concettuale
s_0	Seed deterministico utilizzato per il partizionamento dei dati e la generazione invariante dei benchmark
$S = \{s_1, \dots, s_K\}$	Insieme di K seed pseudo-casuali per l'inizializzazione stocastica e l'addestramento dei classificatori
$H_k(M, D)$	Matrice delle prestazioni (es. TPR, TNR) ottenute dal modello M sul benchmark D con il seed s_k
$\hat{\mu}(M, D), \hat{\sigma}^2(M, D)$	Stima puntuale del valore atteso e della varianza campionaria delle metriche prestazionali marginalizzate sui K seed
t, ν_{eff}	Statistica del test e gradi di libertà effettivi nel test t di Welch per la significatività rispetto al baseline
$\Gamma(M) = \mathcal{A}_m(M) $	Indice di granularità: cardinalità delle famiglie d'attacco osservate in addestramento dal modello M
$R_1(M)$	Regione decisionale positiva nello spazio $\mathcal{X}$, corrispondente alla previsione dell'etichetta malevola ($Y = 1$)
$\text{supp } P(\mathbf{x} \mid Y = 1, A = a)$	Supporto spaziale effettivo della distribuzione condizionata associata a una singola famiglia di attacco a

3.3.1 Incertezza Stocastica e Valutazione Multi-Seed

Le famiglie di classificatori adottate nel framework (Sezione 3.2.3) presentano componenti algoritmiche intrinsecamente stocastiche: il campionamento bootstrap e la selezione casuale del sottoinsieme di caratteristiche per nodo negli ensemble Bagging, così come le procedure di inizializzazione e di sotto-campionamento stocastico tipiche degli algoritmi di Gradient Boosting. Di conseguenza, la prestazione puntuale di un singolo modello addestrato con un'unica inizializzazione non costituisce, di per sé, una stima affidabile della sua reale capacità discriminativa: un risultato favorevole isolato potrebbe essere l'esito di una particolare configurazione stocastica favorevole, piuttosto che un'evidenza sistematica della superiorità del modello. Il framework metodologico introduce pertanto un protocollo di valutazione *multi-seed*, volto a stimare non solo il valore atteso della prestazione, ma anche la sua dispersione, e a verificarne la significatività statistica rispetto al modello di riferimento.

Separazione tra Seed di Composizione del Dataset e Seed di Addestramento

Per isolare la stocasticità algoritmica dei classificatori da qualsivoglia variabilità indotta dalla composizione dei dati, il framework distingue rigorosamente due categorie di seed pseudo-casuali, aventi ruolo e ambito di applicazione disgiunti:

- un seed deterministico s_0 , fissato una tantum e impiegato esclusivamente nelle procedure di split stratificato e di sintesi dei benchmark descritte nella Sezione 3.1.1. Tale seed determina univocamente le partizioni $\mathcal{D}_k^{(\text{train})}$, $\mathcal{D}_k^{(\text{test})}$ e la composizione di ciascun benchmark derivato, mantenendole identiche e invarianti per l'intero protocollo sperimentale;
- un insieme di K seed indipendenti $\mathcal{S} = \{s_1, s_2, \dots, s_K\}$, impiegati esclusivamente nelle fasi di addestramento dei classificatori (inizializzazione degli stimatori, campionamento bootstrap, selezione casuale delle caratteristiche), con K inteso come parametro simbolico del framework teorico.

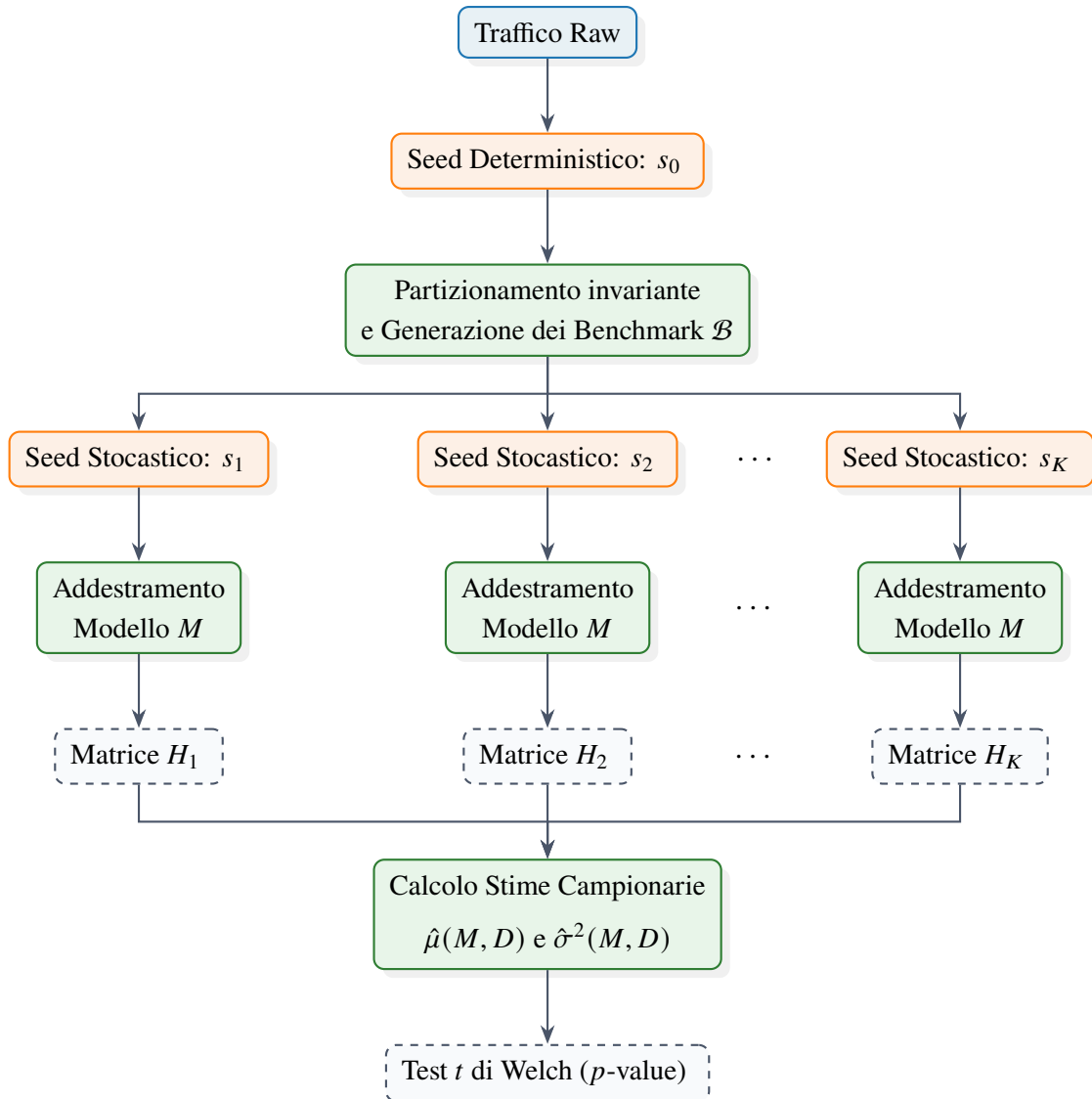


Figura 3.6: Diagramma di flusso del protocollo di isolamento e valutazione stocastica. La stabilità della partizione dei dati è forzata top-down dall'unico seed deterministico s_0 , delegando la totalità della varianza osservata tra le matrici H_k all'inizializzazione stocastica dettata dall'insieme dei seed S interni agli stimatori.

La netta separazione tra s_0 e S garantisce che tutti i modelli, per ciascuna replica k , operino esattamente sullo stesso insieme di dati di addestramento e di test: nessun seed di modello risulta favorito da una composizione del dataset a lui più congeniale, e l'intera variabilità osservata tra le K repliche è pertanto attribuibile univocamente alla stocasticità algoritmica del classificatore, e non a un artefatto della partizione dei dati.

Metriche di Valutazione e Aggregazione Multi-Seed

Per un generico modello M (istanza di una qualsiasi delle famiglie $\mathcal{M}_{\text{base}}, \mathcal{M}_S, \mathcal{M}_H^{(T)}, \mathcal{M}_{S,a}, \mathcal{M}_{H,T,a}$) addestrato con il seed $s_k \in \mathcal{S}$ e valutato su un benchmark di test D , siano TP_k, TN_k, FP_k, FN_k le cardinalità della matrice di confusione ottenute dal confronto tra $\hat{Y}$ e l'etichetta reale Y ¹⁴. Si definiscono la *True Positive Rate* (TPR, sensibilità) e la *True Negative Rate* (TNR, specificità) come:

$$\text{TPR}_k(M, D) = \frac{TP_k}{TP_k + FN_k}, \quad \text{TNR}_k(M, D) = \frac{TN_k}{TN_k + FP_k} \quad (3.31)$$

Per ciascun seed s_k , i valori $\text{TPR}_k(M, D)$ e $\text{TNR}_k(M, D)$ vengono organizzati in una matrice $H_k \in \mathbb{R}^{|\mathcal{M}| \times |\mathcal{B}|}$, con righe indicizzate dai modelli $M \in \mathcal{M}$ e colonne dai benchmark di test $D \in \mathcal{B}$ coinvolti nel protocollo di valutazione incrociata (Sezioni 3.2.1 e 3.2.2), corredata da una colonna aggiuntiva di media di riga:

$$\bar{H}_k(M) = \frac{1}{|\mathcal{B}|} \sum_{D \in \mathcal{B}} H_k(M, D) \quad (3.32)$$

L'impianto metodologico fin qui descritto culmina nella definizione della matrice H_k , che concretizza l'operazione di proiezione incrociata di *tutte* le architetture formulate (siano esse aggregati multiclasse o specialisti biclasse) sull'intera estensione dei domini di test disponibili. Come illustrato nella Tabella 3.8, lo spazio delle valutazioni si articola in una griglia strutturata che elimina ogni barriera valutativa tra le categorie di classificazione e quelle di benchmark, rendendo esplicita l'universalità dell'insieme di test $\mathcal{B}$.

¹⁴Rispettivamente: veri positivi, veri negativi, falsi positivi e falsi negativi, secondo la convenzione per cui $Y = 1$ denota traffico malevolo.

Tabella 3.8: Rappresentazione concettuale della matrice di valutazione incrociata $H_k \in \mathbb{R}^{|\mathcal{M}| \times |\mathcal{B}|}$. Ogni riga corrisponde a una famiglia di modelli $\mathcal{M}$ (aggregati e biclasse) e ogni colonna a una macro-categoria dell'insieme universale dei benchmark $\mathcal{B}$. Le spunte ($\checkmark$) evidenziano l'esecuzione sistematica della proiezione di ciascun modello su tutte le partizioni di test, indipendentemente dalla loro natura nativa, coprendo così sia gli scenari in-domain che di cross-evaluation.

$\mathbf{H}_k \in \mathbb{R}^{ \mathcal{M} \times \mathcal{B} }$	$\mathcal{D}^{(\text{test})}$	$\mathcal{D}_a^{(\text{test})}$	$\mathcal{D}_{b \neq a}^{(\text{test})}$
	(Aggregati)	(Biclasse In-Domain)	(Biclasse Cross)
$\mathcal{M}_{\text{base}}$	$\checkmark$	$\checkmark$	$\checkmark$
$\mathcal{M}_S$	$\checkmark$	$\checkmark$	$\checkmark$
$\mathcal{M}_H^{(T)}$	$\checkmark$	$\checkmark$	$\checkmark$
$\mathcal{M}_{S,a}$	$\checkmark$	$\checkmark$	$\checkmark$
$\mathcal{M}_{H,T,a}$	$\checkmark$	$\checkmark$	$\checkmark$

La stima puntuale del valore atteso e della dispersione prestazionale per la coppia (M, D) , marginalizzata sulle K repliche stocastiche, è data dalla media e dalla varianza campionarie:

$$\hat{\mu}(M, D) = \frac{1}{K} \sum_{k=1}^K H_k(M, D), \quad \hat{\sigma}^2(M, D) = \frac{1}{K-1} \sum_{k=1}^K (H_k(M, D) - \hat{\mu}(M, D))^2 \quad (3.33)$$

La matrice aggregata $\hat{H} = [\hat{\mu}(M, D)]$, corredata dai corrispondenti valori di $\hat{\sigma}^2(M, D)$, costituisce la sintesi prestazionale multi-seed per ciascuna tipologia di modello, riducendo l'influenza di configurazioni stocastiche anomale sulla valutazione complessiva del framework.

A partire dalle medesime cardinalità TP_k, TN_k, FP_k, FN_k , si definiscono inoltre le metriche di *Precision* e di *F1-Score*, quest'ultima impiegata specificamente nel protocollo di verifica della significatività statistica formalizzato nella Sezione 3.3.1:

$$\text{Precision}_k(M, D) = \frac{TP_k}{TP_k + FP_k}, \quad \text{F1}_k(M, D) = \frac{2 \cdot \text{Precision}_k(M, D) \cdot \text{TPR}_k(M, D)}{\text{Precision}_k(M, D) + \text{TPR}_k(M, D)} \quad (3.34)$$

dove l'F1-Score, in quanto media armonica di Precision e Recall ($\equiv$ TPR), costituisce una metrica sintetica particolarmente indicata a penalizzare squilibri tra falsi positivi e falsi negativi, coerentemente con l'asimmetria di classe intrinseca ai benchmark del framework (Sezione 3.1.1).

Verifica di Significatività Statistica mediante Test di Welch

La sola stima di $\hat{\mu}$ e $\hat{\sigma}^2$ non è sufficiente a stabilire se lo scostamento prestazionale tra un modello M e il modello baseline $\mathcal{M}_{\text{base}}$ sia statisticamente rilevante o riconducibile alla naturale variabilità campionaria. Il framework adotta a tal fine il test t di Welch [37], nella sua formulazione per campioni indipendenti a varianza non necessariamente omogenea.

La letteratura sul confronto statistico di classificatori [7] sconsiglia, in generale, l'impiego di test parametrici quando il confronto avviene su un numero ridotto di dataset eterogenei, per i quali ciascun dataset fornisce un singolo valore appaiato e l'assunzione di normalità risulta difficilmente verificabile; in tali condizioni sono raccomandate alternative non parametriche (test dei ranghi con segno di Wilcoxon, test di Friedman). Lo scenario qui considerato è tuttavia strutturalmente distinto: ciascuna osservazione $\overline{\text{F1}}_k(M)$ non è un valore puntuale su un singolo dataset, bensì la media di riga dell'F1-Score, calcolata su $|\mathcal{B}|$ benchmark di test per un dato seed s_k . In virtù del Teorema del Limite Centrale, tale quantità aggregata tende a una distribuzione approssimativamente normale al crescere di $|\mathcal{B}|$, indipendentemente dalla distribuzione delle singole osservazioni F1 per dataset, fornendo una base più solida per l'impiego di un test parametrico rispetto al caso di confronto diretto su valori singoli non aggregati. La formulazione di Welch, in particolare, è preferita alla variante omoschedastica dello Student's t -test poiché non assume l'uguaglianza delle varianze tra le distribuzioni per-seed del modello M e del baseline, condizione non garantita a priori tra famiglie di modelli strutturalmente eterogenee.

Per ciascun modello M (con esclusione del baseline stesso), sia $\overline{\text{F1}}_k(M) = \frac{1}{|\mathcal{B}|} \sum_{D \in \mathcal{B}} \text{F1}_k(M, D)$ la media di riga dell'F1-Score per il seed s_k , definita in analogia all'Equazione (3.32) a partire dalla metrica dell'Equazione (3.34). Siano $\{\overline{\text{F1}}_k(M)\}_{k=1}^K$ e $\{\overline{\text{F1}}_k(\mathcal{M}_{\text{base}})\}_{k=1}^K$ le rispettive sequenze di medie per seed per il modello M e per il baseline. Definita l'ipotesi nulla $H_0 : \mu_M = \mu_{\text{base}}$, la statistica del test è espressa come:

$$t = \frac{\bar{x}_M - \bar{x}_{\text{base}}}{\sqrt{\frac{s_M^2}{K} + \frac{s_{\text{base}}^2}{K}}} \quad (3.35)$$

dove $\bar{x}_M$, $\bar{x}_{\text{base}}$ e s_M^2 , s_{base}^2 denotano rispettivamente la media e la varianza campionaria dell'F1-Score calcolate sulle K osservazioni per-seed di M e di $\mathcal{M}_{\text{base}}$. I gradi di libertà effettivi ν_{eff} sono stimati

mediante l'approssimazione di Welch–Satterthwaite:

$$\nu_{\text{eff}} = \frac{\left(\frac{s_M^2}{K} + \frac{s_{\text{base}}^2}{K} \right)^2}{\frac{(s_M^2/K)^2}{K-1} + \frac{(s_{\text{base}}^2/K)^2}{K-1}} \quad (3.36)$$

Il valore p risultante viene confrontato con una soglia di significatività α : qualora $p < \alpha$, l'ipotesi nulla viene rigettata, e lo scostamento prestazionale tra il modello M e il baseline $\mathcal{M}_{\text{base}}$ è considerato statisticamente significativo e non attribuibile alla sola variabilità stocastica intrinseca degli stimatori. Il confronto viene condotto in modo puntuale per ciascun modello rispetto al baseline, coerentemente con il ruolo di $\mathcal{M}_{\text{base}}$ quale ancora di prestazione di riferimento stabilita nella Sezione 3.2.1.

3.3.2 Granularità della Risposta: Formulazione Binaria vs. Multiclasse

Sebbene lo spazio di uscita del framework rimanga rigorosamente binario ($\mathcal{Y} = \{0, 1\}$, cfr. Sezione 3.1.1)¹⁵, le due famiglie di classificatori sin qui formalizzate differiscono strutturalmente per il numero di famiglie d'attacco rappresentate simultaneamente all'interno della medesima frontiera di decisione. Si definisce pertanto un indice di *granularità di rappresentazione* $\Gamma(M)$, pari alla cardinalità dell'insieme delle famiglie d'attacco incluse nella partizione di addestramento di un modello M :

$$\Gamma(M) = |\mathcal{A}_m(M)|, \quad \mathcal{A}_m(M) \subseteq \mathcal{A} \quad (3.37)$$

dove $\mathcal{A}_m(M)$ denota l'insieme delle famiglie d'attacco osservate in addestramento dal modello M . Per i modelli aggregati (Sezione 3.2.1) si ha $\Gamma(\mathcal{M}_S) = |\mathcal{A}_S|$ e $\Gamma(\mathcal{M}_H^{(T)}) = |\mathcal{A}_T|$, ossia l'intera tassonomia disponibile nel rispettivo dominio; per i modelli biclasse (Sezione 3.2.2) si ha invece $\Gamma(\mathcal{M}_{S,a}) = \Gamma(\mathcal{M}_{H,T,a}) = 1$ per costruzione. In questo senso, i modelli aggregati costituiscono l'estremo a granularità grossolana (“*multiclasse*”, in quanto la loro frontiera deve generalizzare simultaneamente su una molteplicità eterogenea di segnature d'attacco), mentre i modelli biclasse

¹⁵Nell'ambito della presente trattazione, l'uso dei termini “multiclasse” e “biclasse” non si riferisce alla cardinalità dello spazio di codifica delle uscite $\mathcal{Y}$, che rimane sempre scalare e binario, bensì alla granularità semantica delle componenti d'attacco aggregate all'interno della medesima etichetta positiva $Y = 1$.

costituiscono l'estremo a granularità fine (“*biclasse*” propriamente detto, con una frontiera dedicata alla singola famiglia).

Regione Decisionale e Copertura del Supporto Malevolo

Sia $R_1(M) = \{\mathbf{x} \in \mathcal{X} : f_{\theta^*}(\mathbf{x}) \geq \tau\}$ la regione decisionale positiva associata al modello M , secondo la regola già formalizzata nell'Equazione (3.9). Per un modello aggregato, tale regione deve necessariamente coprire l'unione dei supporti delle distribuzioni condizionate di ciascuna famiglia d'attacco osservata:

$$R_1(\mathcal{M}_S) \supseteq \bigcup_{a \in \mathcal{A}_S} \text{supp } P_S(\mathbf{x} \mid Y = 1, A = a) \quad (3.38)$$

mentre per un modello biclasse la regione positiva è calibrata in modo mirato sul supporto di un'unica sotto-distribuzione¹⁶:

$$R_1(\mathcal{M}_{S,a}) \supseteq \text{supp } P_S(\mathbf{x} \mid Y = 1, A = a) \quad (3.39)$$

Tale distinzione strutturale ha una diretta implicazione geometrica, coerente con il partizionamento ricorsivo $\{R_m\}_{m=1}^M$ formalizzato nella Sezione 3.2.3: al crescere di $\Gamma(M)$, la regione $R_1(M)$ deve estendersi su un volume dello spazio $\mathcal{X}$ tendenzialmente più ampio ed eterogeneo, risultante dalla composizione delle regioni foglia associate a più sotto-distribuzioni distinte; al diminuire di $\Gamma(M)$ (limite $\Gamma = 1$), $R_1(M)$ si concentra invece attorno a un unico manifold di supporto, con margine decisionale calibrato sulla sola distribuzione della famiglia a .

Compromesso tra Rigidità Specialistica e Copertura Generalista

La calibrazione di $R_1(M)$ su un manifold ristretto ($\Gamma(M) = 1$) produce un confine decisionale a elevata specificità: il classificatore biclasse è, per costruzione, ottimizzato per discriminare la sola famiglia a dal traffico nominale, senza compromessi indotti dalla necessità di generalizzare su signature eterogenee. Tale specificità comporta tuttavia una maggiore *rigidità* del confine sotto *domain shift*: qualora il vettore osservato nel dominio di destinazione risulti sistematicamente

¹⁶In un contesto d'apprendimento induttivo reale, la regione decisionale $R_1(\mathcal{M}_{S,a})$ racchiude il supporto effettivo $\text{supp } P_S(\mathbf{x} \mid Y = 1, A = a)$ estendendosi oltre di esso per un margine di generalizzazione $\epsilon > 0$, indotto dalla struttura dell'iperpiano o dagli split degli alberi decisionali.

perturbato secondo il modello $\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\eta}$ già discusso in Sezione 3.2.3, uno spostamento del manifold malevolo oltre il margine appreso in fase di addestramento non è compensato da alcuna informazione di supporto proveniente da altre famiglie d’attacco, risultando in un decadimento prestazionale puntuale e localizzato.

Al contrario, un modello aggregato, essendo vincolato a costruire una regione $R_1(M)$ che copra congiuntamente più sotto-distribuzioni (Equazione (3.38)), sviluppa margini decisionali strutturalmente più ampi e meno specifici per la singola famiglia d’attacco. Tale maggiore ampiezza può conferire una tolleranza intrinseca a perturbazioni locali di una singola sotto-distribuzione, a scapito però della purezza discriminativa per classe¹⁷.

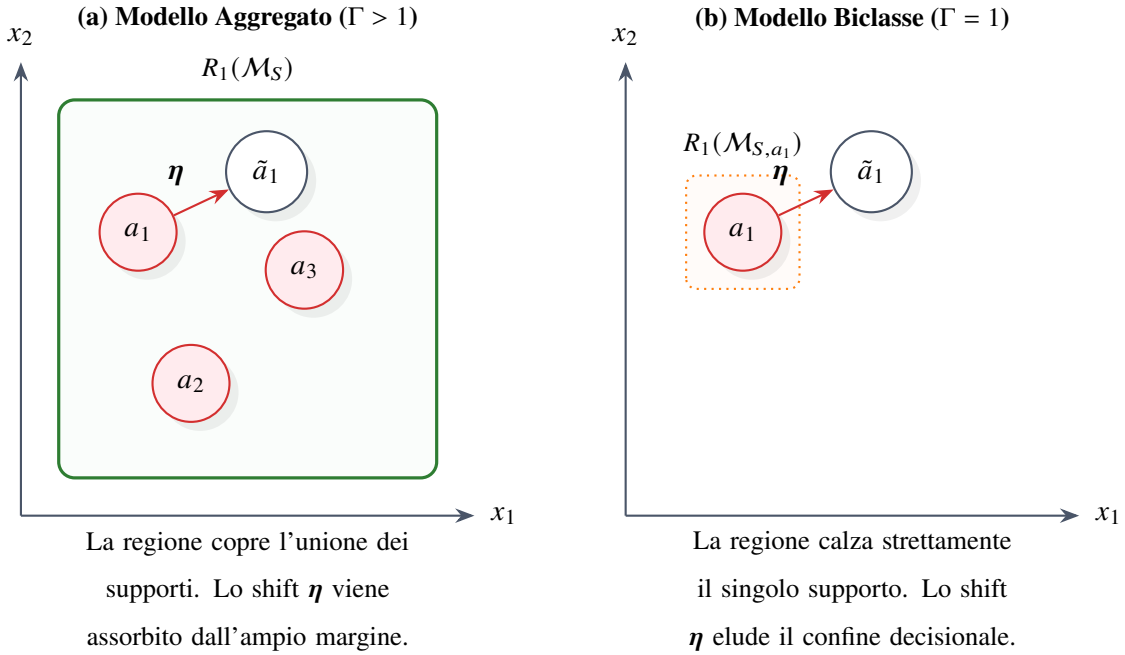


Figura 3.7: Rappresentazione bidimensionale nello spazio delle caratteristiche $\mathcal{X}$ dell’impatto del *domain shift* (vettore di traslazione $\boldsymbol{\eta}$) a seconda della granularità del modello. A sinistra, la maggiore ampiezza spaziale del modello aggregato garantisce tolleranza generalista; a destra, l’ottimizzazione specialistica sul singolo supporto malevolo determina rigidità sotto perturbazione.

¹⁷Questo compromesso è la controparte teorica dei fenomeni empirici già anticipati nel framework: il *class shadowing* (Sezione 3.2.1), per cui un’elevata accuratezza aggregata può mascherare un decadimento su famiglie specifiche, e la polarizzazione dello spazio decisionale osservabile in fase di *cross-evaluation* per i modelli biclasse (Sezione 3.2.2).

Il Modello Baseline come Punto di Riferimento Intermedio

Il modello $\mathcal{M}_{\text{base}}$ (Sezione 3.2.1) occupa, rispetto all'indice Γ , una posizione concettualmente intermedia tra i due estremi. La sua regione decisionale eredita la copertura ad ampio spettro propria dei modelli aggregati, essendo $\mathcal{A}_S \subseteq \mathcal{A}_m(\mathcal{M}_{\text{base}})$, ma viene incrementalmente perturbata da una conoscenza mirata e altamente sparsa del dominio target ($\Delta\mathcal{D}_T^{(\text{train})}$, Equazione (3.11)), la cui esiguità cardinale ($|\Delta\mathcal{D}_T^{(\text{train})}| \ll |\mathcal{D}_S^{(\text{train})}|$) ne impedisce una piena riqualificazione come copertura multiclasse del dominio di destinazione. $\mathcal{M}_{\text{base}}$ costituisce pertanto l'ancora di riferimento rispetto alla quale entrambi gli estremi della granularità — l'ampia copertura generalista dei modelli aggregati $\mathcal{M}_S, \mathcal{M}_H^{(T)}$ e l'elevata specificità dei modelli biclasse $\mathcal{M}_{S,a}, \mathcal{M}_{H,T,a}$ — vengono confrontati, secondo il protocollo di verifica statistica formalizzato in Sezione 3.3.1, al fine di determinare quale livello di granularità offra il compromesso più favorevole tra specificità discriminativa e robustezza al *domain shift*.

3.4 Framework Analitico, Spiegabilità e Significatività Statistica

La presente sezione definisce il framework analitico, diagnostico e statistico adottato per caratterizzare lo scostamento distributivo (*domain shift*), decodificare le logiche decisionali dei modelli e valutare la significatività delle variazioni prestazionali tra domini eterogenei. L'impianto metodologico integra tecniche di riduzione dimensionale, stima non parametrica di densità e attribuzione dell'importanza delle caratteristiche fondata sulla teoria dei giochi cooperativi, affiancandole a un formalismo rigoroso per la gestione delle soglie e la verifica di ipotesi statistiche.

Per agevolare la consultazione della trattazione, i simboli matematici, i costrutti diagnostici e le metriche prestazionali introdotti in questa sezione sono riepilogati nella Tabella 3.9.

Tabella 3.9: Sintesi della notazione relativa al framework analitico, all’interpretabilità e alla valutazione statistica.

Simbolo	Descrizione e Significato Concettuale
$\mathbf{Z}_D$	Matrice dei dati del benchmark D standardizzata mediante la media μ_S e la deviazione standard σ_S del solo dominio sorgente
$\Sigma_D, \mathbf{W}_D$	Matrice di covarianza campionaria e matrice di proiezione ortogonale sui primi due assi della PCA
$\hat{f}_y(z), h_{\text{KDE}}$	Stima di densità univariata KDE per la classe y e relativo parametro di liscio (<i>bandwidth</i>) con kernel gaussiano $\kappa(\cdot)$
$\phi_j(\mathbf{x})$	Valore SHAP associato alla j -esima caratteristica del vettore $\mathbf{x}$, rappresentante il suo contributo marginale esatto
$I_j^{(D)}, C_j$	Importanza media assoluta SHAP della j -esima caratteristica sul benchmark D e frequenza globale di inclusione nella <i>top-3</i>
$TP(\tau), FP(\tau), TN(\tau), FN(\tau)$	Cardinalità della matrice di confusione determinate in funzione della soglia decisionale parametrica τ
AUC_{ROC}, AUC_{PR}	Area sottesa alle curve <i>Receiver Operating Characteristic</i> e <i>Precision-Recall</i> per la caratterizzazione al variare di $\tau \in [0, 1]$
ΔF_1	Variazione prestazionale in termini di F_1 -score per la quantificazione del degrado da <i>covariate shift</i> tra sorgente $\mathcal{D}_S$ e target $\mathcal{D}_T$
t, ν, H_0	Statistica del test t di Welch, gradi di libertà effettivi di Welch–Satterthwaite e ipotesi nulla di equivalenza delle medie prestazionali

3.4.1 Caratterizzazione Densometrica e Proiezioni Latenti

L'analisi dello scostamento distributivo (*domain shift*) e la valutazione della separabilità geometrica tra traffico nominale e traffico malevolo richiedono l'impiego di tecniche di riduzione dimensionale e stima di densità. Come anticipato nella Sezione 3.1.2, mentre l'architettura di classificazione basata su modelli ad albero gode dell'invarianza alle trasformazioni monotoniche e prescinde dalla scala dei dati, gli strumenti diagnostici e visivi (PCA e KDE) sono sensibili alla grandezza numerica delle variabili.

Prima dell'estrazione delle proiezioni e della stima densometrica, viene applicata una normalizzazione dedicata a ciascun benchmark $D \in \mathcal{B}$, secondo l'insieme dei benchmark di test già introdotto nella Sezione 3.3.1. Tale trasformazione è ancorata unicamente alle statistiche del dominio sorgente $\mathcal{D}_S$ e rimane rigorosamente disaccoppiata dallo stadio di addestramento dei modelli.

Standardizzazione Diagnostica Dipendente dal Sorgente

Per evitare fenomeni di contaminazione informativa (*data leakage*), derivati dall'estrazione di statistiche da domini target incogniti, e per preservare la reale ampiezza del *domain shift*, i dati di ciascun benchmark $D \in \mathcal{B}$ vengono standardizzati utilizzando esclusivamente la media μ_S e la deviazione standard σ_S calcolate sul benchmark sorgente di riferimento S .

Sia $\mathbf{X}_D \in \mathbb{R}^{N_D \times d}$ la matrice dei dati grezzi del generico benchmark D . Il generico elemento della matrice trasformata $\mathbf{Z}_D \in \mathbb{R}^{N_D \times d}$ è definito come:

$$\mathbf{Z}_{D,ij} = \frac{\mathbf{X}_{D,ij} - \mu_{S,j}}{\sigma_{S,j} + \epsilon} \quad (3.40)$$

dove $\mu_{S,j}$ e $\sigma_{S,j}$ rappresentano la media e la deviazione standard campionarie della j -esima caratteristica stimata sul set sorgente S , e $\epsilon > 0$ è una costante infinitesima introdotta a scopo di stabilità numerica, per prevenire divisioni per valori prossimi a zero in presenza di caratteristiche a varianza sorgente quasi nulla (es. indicatori binari fortemente sbilanciati).

L'ancoraggio a (μ_S, σ_S) , anziché a statistiche ricalcolate localmente su ciascun benchmark D , garantisce che eventuali traslazioni di media o contrazioni di varianza presenti nel benchmark D non vengano annullate artificialmente: una standardizzazione locale ricentrerebbe infatti ogni benchmark sulla propria origine, occultando esattamente lo scostamento distributivo che l'analisi

si propone di rendere visibile. Questo ancoraggio è una scelta di riferimento per la sola finalità diagnostico-visiva e non implica alcuna dipendenza dalla scala da parte del classificatore: come dimostrato nella Sezione 3.1.2, la decisione dei modelli ad albero è invariante rispetto a qualsiasi trasformazione monotonica crescente, ancorata o meno al dominio sorgente. L'ancoraggio a (μ_S, σ_S) risponde dunque esclusivamente all'esigenza di disporre di un sistema di riferimento fisso e interpretabile per l'ispezione visiva, non a un requisito del modello di classificazione.

Proiezioni Ortogonali e Relatività dello Spazio Latente (PCA)

Per ispezionare visivamente le relazioni topologiche del traffico di rete, lo spazio delle caratteristiche viene mappato in un sottospazio latente a due dimensioni tramite Analisi delle Componenti Principali (PCA) [26, 13].

Necessità tecnica dello scaling per la PCA La PCA si basa sulla decomposizione spettrale della matrice di covarianza Σ_D , massimizzando la varianza lungo gli assi ortogonali. In assenza della trasformazione espressa nell'Equazione (3.40), l'impiego diretto di caratteristiche in scala grezza eterogenea (es. volumi in byte con varianze dell'ordine di 10^6 affiancati a *ratio* adimensionali compresi in $[0, 1]$) causerebbe una dominanza artificiale della matrice di covarianza da parte delle variabili a magnitudo numerica maggiore. La prima componente principale si schiaccerebbe unicamente sulle variabili a scala più ampia, indipendentemente dal loro effettivo valore informativo o discriminante.

Sulla matrice standardizzata $\mathbf{Z}_D$, la matrice di covarianza campionaria Σ_D è formulata come:

$$\Sigma_D = \frac{1}{N_D - 1} \mathbf{Z}_D^\top \mathbf{Z}_D \quad (3.41)$$

La matrice di proiezione $\mathbf{W}_D \in \mathbb{R}^{d \times 2}$ è composta dagli autovettori ortonormali associati ai due principali autovalori di Σ_D .

La standardizzazione preventiva isola una proprietà rilevante del framework: sebbene i campioni della classe lecita ($Y = 0$) derivino dalla medesima distribuzione stazionaria di base in tutti i benchmark, la loro proiezione nel piano latente muta geometricamente al variare del benchmark D . Poiché Σ_D risente della varianza congiunta dell'intero set di dati, l'accoppiamento con vettori malevoli ($Y = 1$) eterogenei riorienta gli assi di massima varianza. L'eliminazione degli artefatti di

scala mediante (μ_S, σ_S) assicura che tali rotazioni non siano attribuibili a discrepanze puramente dimensionali, ma riflettano la specifica struttura dell'attacco contestuale e il relativo *domain shift*. Va precisato che, trattandosi di una tecnica non supervisionata, la PCA massimizza la varianza totale dei dati senza impiego esplicito dell'etichetta Y : l'osservazione secondo cui l'asse di massima varianza tende a riflettere la separazione tra le classi è pertanto un comportamento empiricamente frequente, ma non una proprietà garantita in generale, a differenza di tecniche supervisionate come l'Analisi Discriminante Lineare (LDA).

Stima di Densità (KDE) su Sub-spazio Selettivo

Per quantificare la sovrapposizione distributiva e l'incertezza decisionale tra le classi, l'analisi si avvale della *Kernel Density Estimation* (KDE) univariata, i cui fondamenti teorici risalgono ai lavori di Rosenblatt [27] e Parzen [25]. Per garantire l'interpretabilità geometrica e superare la sparsità dei dati in alta dimensione, la KDE viene confinata alle tre caratteristiche di maggiore rilevanza empirica individuate tramite l'aggregazione dei valori di Shapley (SHAP, Sezione 3.4.2).

Motivazione metodologica dello scaling per la KDE A differenza della PCA, dal punto di vista strettamente matematico la forma della stima di densità univariata è equivariante alle trasformazioni lineari (la geometria della curva non si distorce, ma trasla e si riscalda coerentemente). Tuttavia, l'applicazione della standardizzazione Z-score dipendente dal sorgente risulta indispensabile per tre ragioni pratiche e metodologiche:

1. **Preservazione visiva del Domain Shift:** l'utilizzo delle metriche del sorgente $(\mu_{S,j}, \sigma_{S,j})$ fa sì che se una caratteristica nel benchmark D subisce una traslazione sistematica della media, la curva di densità KDE risulti fisicamente traslata rispetto all'origine $z = 0$, rendendo immediata l'ispezione visiva del *domain shift*;
2. **Comparabilità grafica cross-feature:** le tre *feature* selezionate mediante SHAP, originariamente espresse in unità di misura disomogenee (es. microsecondi vs. byte/s), vengono mappate sullo stesso asse adimensionale espresso in deviazioni standard dal sorgente, consentendone l'affiancamento visivo su scale omogenee;

3. **Calibrazione stabile della Bandwidth (h_{KDE}):** la condivisione dello spazio adimensionale ($z \sim \mathcal{N}(0, 1)$ sul dominio sorgente) stabilizza l'ottimizzazione della larghezza di banda del kernel h_{KDE} , prevenendo fenomeni di *undersmoothing* (una banda eccessivamente stretta, che produce una curva frastagliata e sovra-adattata al singolo campione osservato) o di *oversmoothing* selettivo (una banda eccessivamente ampia, che appiattisce dettagli distributivi rilevanti) tra variabili a diversa varianza grezza.

Sia $z \in \mathbb{R}$ la variabile standardizzata derivata tramite l'Equazione (3.40) per una singola caratteristica del sottoinsieme SHAP. Coerentemente con la natura dicotomica delle decisioni dei sistemi NIDS, i dati vengono raggruppati secondo l'etichetta binaria $Y \in \mathcal{Y} = \{0, 1\}$ (traffico lecito vs. malevolo). La stima marginale di densità continua $\hat{f}_y(z)$ per la classe $Y = y$ calcolata su N_y campioni è definita da:

$$\hat{f}_y(z) = \frac{1}{N_y h_{\text{KDE}}} \sum_{i=1}^{N_y} \kappa\left(\frac{z - z_{i,y}}{h_{\text{KDE}}}\right) \quad (3.42)$$

dove h_{KDE} è il parametro di liscio (*bandwidth*), da non confondersi con la trasformazione monotonica h_j della Sezione 3.2.3, e $\kappa(\cdot)$ è la funzione kernel, per la quale si adotta la forma gaussiana standard¹⁸:

$$\kappa(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right) \quad (3.43)$$

Compensazione dello Squilibrio di Classe tramite Scala Logaritmica

Per contrastare lo schiacciamento visivo della classe minoritaria derivante dal forte sbilanciamento cardinale ($P(Y = 0) \gg P(Y = 1)$ o viceversa nei vari benchmark), i valori di densità stimata $\hat{f}_y(z)$ vengono proiettati su scala logaritmica in base 10:

$$\hat{f}_{y,\log}(z) = \log_{10}\left(\hat{f}_y(z) + \epsilon\right) \quad (3.44)$$

dove $\epsilon > 0$ è una costante infinitesima di stabilizzazione numerica. La rappresentazione logaritmica permette di ispezionare visivamente le code delle distribuzioni e le micro-aree di sovrapposizione ad alta incertezza, fornendo supporto analitico alla comprensione dei falsi positivi al variare della soglia decisionale τ .

¹⁸Il simbolo $\kappa(\cdot)$, anziché la più comune notazione $K(\cdot)$, è adottato per evitare ambiguità con la cardinalità K dell'insieme dei seed stocastici $\mathcal{S}$ introdotta nella Sezione 3.3.1.

Il ruolo e l'impatto della standardizzazione dipendente dal sorgente sulle diverse componenti diagnostiche del framework sono sintetizzati nella Tabella 3.10.

Tabella 3.10: Sintesi del ruolo e dell'impatto della standardizzazione dipendente dal sorgente (μ_S, σ_S) sulle componenti diagnostiche del framework.

Strumento	Natura del Requisito	Impatto Tecnico e Motivazione Metodologica
Classificatori (Tree-Based)	Nessuno (Nessuna Trasformazione)	Invarianza Monotonica: gli algoritmi basati su albero operano su ordinamenti relativi dei dati. Nessun impatto sulle metriche di accuratezza.
Proiezioni Latenti (PCA)	Matematicamente Indispensabile	Prevenzione della Dominanza delle Scale: la matrice di covarianza Σ_D richiede unità adimensionali; in caso contrario, le componenti principali catturano solo la scala numerica grezza più ampia.
Stima di Densità (KDE)	Metodologico e Visivo	Preservazione del Shift e Omogeneità: non altera la forma intrinseca della stima 1D, ma rende visibile lo spostamento di media dal sorgente $z = 0$, rende i grafici affiancati comparabili e stabilizza la <i>bandwidth</i> h_{KDE} .

3.4.2 Spiegabilità e Attribuzione delle Feature

Allo scopo di decodificare le logiche decisionali dei classificatori *black-box* e isolare le caratteristiche dello spazio condiviso $\mathcal{X}$ che governano la stima della probabilità a posteriori al variare dei

domini, il framework integra un impianto di interpretabilità basato su SHAP (*SHapley Additive exPlanations*) [20]. Tale approccio, fondato sui rigorosi principi della teoria dei giochi cooperativi [29], consente di quantificare analiticamente il contributo marginale esatto di ciascuna variabile vettoriale alla stima predittiva locale.

Formulazione Teorica dei Valori di Shapley e Proprietà Assiomatiche

Nella teoria dei giochi, il valore di Shapley definisce un metodo per distribuire equamente un "guadagno" totale tra un insieme di giocatori cooperanti. Nel contesto dell'apprendimento supervisionato formalizzato dal framework, il "gioco" corrisponde alla funzione decisionale parametrica $f_{\theta^*}(\mathbf{x})$, e i "giocatori" coincidono con le caratteristiche in ingresso.

Sia $\mathcal{F}$ l'insieme totale delle caratteristiche a disposizione del modello (con $|\mathcal{F}| = d$) e $C \subseteq \mathcal{F}$ un generico sottoinsieme (coalizione) di esse. Definita $v(C)$ la funzione di valore che esprime l'output atteso del modello condizionato al solo sottoinsieme C ¹⁹, il valore di Shapley $\phi_j(\mathbf{x})$ per la j -esima caratteristica del singolo vettore $\mathbf{x} \in \mathcal{X}$ è calcolato come:

$$\phi_j(\mathbf{x}) = \sum_{C \subseteq \mathcal{F} \setminus \{j\}} \frac{|C|!(|\mathcal{F}| - |C| - 1)!}{|\mathcal{F}|!} [v(C \cup \{j\}) - v(C)] \quad (3.45)$$

La robustezza di SHAP e la sua superiorità metodologica rispetto a metriche di impurità alternative risiedono nel soddisfacimento di quattro assiomi matematici fondamentali²⁰:

1. **Efficienza Locale (Additività):** la somma dei valori SHAP di tutte le caratteristiche eguaglia la differenza tra la predizione per lo specifico vettore $\mathbf{x}$ e la predizione media del modello (baseline). Ovvero: $f_{\theta^*}(\mathbf{x}) = \phi_0 + \sum_{j=1}^d \phi_j(\mathbf{x})$.
2. **Simmetria:** se due caratteristiche i e j apportano il medesimo contributo marginale a ogni possibile sottoinsieme C , i loro valori SHAP risultano algebricamente identici ($\phi_i = \phi_j$).

¹⁹In un contesto applicativo di apprendimento automatico, tale funzione coincide con il valore atteso condizionato della stima parametrica, ovvero $v(C) = \mathbb{E}[f_{\theta^*}(\mathbf{x}) \mid \mathbf{x}_C]$.

²⁰L'aderenza a questi assiomi garantisce che l'importanza delle *feature* sia equa e non soggetta a paradossi di attribuzione tipici delle metriche puramente empiriche.

3. **Giocatore Nullo (Missingness):** una caratteristica che non altera mai la predizione, indipendentemente dal sottoinsieme di attributi a cui viene aggiunta, possiede un valore SHAP rigorosamente pari a zero.
4. **Coerenza:** se un modello viene modificato in modo tale che il contributo marginale di una caratteristica aumenti (o rimanga invariato), il relativo valore SHAP non può diminuire.

TreeSHAP e Ottimizzazione Algoritmica

Dal punto di vista algoritmico, l'estrazione dei valori ϕ_j è condotta tramite un'ottimizzazione basata sulla struttura interna degli stimatori. Considerata l'architettura dei classificatori impiegati, basati sul partizionamento ricorsivo dello spazio (e.g., *Random Forest*, *Histogram-based Gradient Boosting*), la metodologia sfrutta l'algoritmo *TreeSHAP* [21]²¹.

Per garantire la stabilità statistica delle stime limitando contestualmente l'overhead computazionale, la spiegazione locale viene calcolata su un sottoinsieme $\mathcal{D}_{\text{sample}}^{(\text{test})} \subset \mathcal{D}^{(\text{test})}$, estratto mediante campionamento stratificato. Tale campionamento preserva inalterato il vincolo di proporzionamento parametrico ρ originario tra il traffico nominale ($Y = 0$) e il traffico malevolo ($Y = 1$).

L'output dell'operazione consiste in matrici ad alta dimensionalità contenenti i valori ϕ_j . Limitando l'analisi al dominio di classificazione binario e alla classe malevola ($Y = 1$), tali valori esprimono l'impatto sul margine non normalizzato del modello (es. *log-odds*) e vengono resi interpretabili attraverso proiezioni a dispersione (*summary dot plot*). In tali costrutti visivi, la posizione sull'asse orizzontale quantifica il valore SHAP (intensità e direzione dell'impatto decisionale), mentre lo spettro cromatico codifica il valore quantitativo grezzo della caratteristica, consentendo di individuare relazioni dirette o inverse tra le metriche trasmissive e l'etichetta di intrusione.

Aggregazione Trasversale e Selezione per l'Analisi Densometrica

I risultati prodotti da SHAP a livello locale per ciascuna configurazione di benchmark sono stati impiegati per alimentare l'indagine densometrica trasversale descritta nella Sezione 3.4.1. A tal

²¹*TreeSHAP* [21] abbatte la complessità computazionale del calcolo esatto dei valori di Shapley da esponenziale $O(2^{|\mathcal{F}|})$ a polinomiale $O(TLD^2)$, dove T è il numero di alberi dell'ensemble, L il numero massimo di foglie e D la profondità massima.

fine, è stata definita una metrica di aggregazione progettata per estrarre le tre dimensioni dello spazio $\mathcal{X}$ globalmente più influenti.

Per ogni generico benchmark di test $D \in \mathcal{B}$, le caratteristiche vengono inizialmente ordinate in senso decrescente in base alla loro importanza media assoluta $I_j^{(D)}$:

$$I_j^{(D)} = \frac{1}{N} \sum_{i=1}^N |\phi_j(\mathbf{x}_i^{(D)})| \quad (3.46)$$

dove $N = |\mathcal{D}_{\text{sample}}^{(\text{test})}|$ rappresenta la cardinalità del sottoinsieme campionato estratto in precedenza, e non l'intero dataset a disposizione.

Sia $\mathcal{T}_D \subset \mathcal{F}$ l'insieme contenente le prime tre caratteristiche classificate nel benchmark D . Per identificare i descrittori maggiormente resilienti e discriminanti ai confini tra domini $\mathcal{D}_S$ e $\mathcal{D}_T$, si definisce un contatore di frequenza globale C_j :

$$C_j = \sum_{D \in \mathcal{B}} \mathbb{I}(j \in \mathcal{T}_D) \quad (3.47)$$

dove $\mathbb{I}(\cdot)$ è la funzione indicatrice che assume valore 1 se la caratteristica j figura nella *top-3* dello scenario D , e 0 altrimenti. Le tre dimensioni vettoriali associate ai valori massimi di C_j al termine di tutte le combinazioni di addestramento e inferenza costituiscono il nucleo stabile informativo utilizzato per tracciare le curve KDE precedentemente illustrate. Questo processo di selezione garantisce che la valutazione del *domain shift* si concentri unicamente sulle proiezioni spaziali che i modelli considerano universalmente più critiche per l'isolamento delle anomalie.

3.4.3 Metriche, Soglia e Significatività dei Confronti

Per valutare analiticamente le prestazioni dei modelli di classificazione formalizzati nella sezione precedente e quantificare l'impatto del *domain shift*, si definisce un framework di valutazione fondato sulla teoria della decisione statistica. In particolare, si distingue la valutazione puntuale delle prestazioni, riferita a una specifica soglia decisionale, dall'analisi del comportamento discriminativo del classificatore al variare della soglia di decisione.

Formalizzazione della Matrice di Confusione

Richiamando la regola di decisione parametrica introdotta nella Sezione 3.2.1, la classe predetta $\hat{y}_i(\tau)$ consente di definire le cardinalità della matrice di confusione, espresse come funzioni della soglia decisionale τ :

$$TP(\tau) = \sum_{i=1}^N \mathbb{I}(y_i = 1 \wedge \hat{y}_i(\tau) = 1) \quad (3.48)$$

$$FP(\tau) = \sum_{i=1}^N \mathbb{I}(y_i = 0 \wedge \hat{y}_i(\tau) = 1) \quad (3.49)$$

$$TN(\tau) = \sum_{i=1}^N \mathbb{I}(y_i = 0 \wedge \hat{y}_i(\tau) = 0) \quad (3.50)$$

$$FN(\tau) = \sum_{i=1}^N \mathbb{I}(y_i = 1 \wedge \hat{y}_i(\tau) = 0) \quad (3.51)$$

Le quattro quantità rappresentano rispettivamente il numero di veri positivi, falsi positivi, veri negativi e falsi negativi prodotti dal classificatore per una determinata soglia decisionale.

Tabella 3.11: Struttura della Matrice di Confusione parametrizzata rispetto alla soglia decisionale τ .

Classe Predetta ($\hat{y}(\tau)$) $\times$ Classe Reale (y)	Positivo ($y = 1$)	Negativo ($y = 0$)
Positivo ($\hat{y} = 1$)	True Positive ($TP(\tau)$)	False Positive ($FP(\tau)$)
Negativo ($\hat{y} = 0$)	False Negative ($FN(\tau)$)	True Negative ($TN(\tau)$)

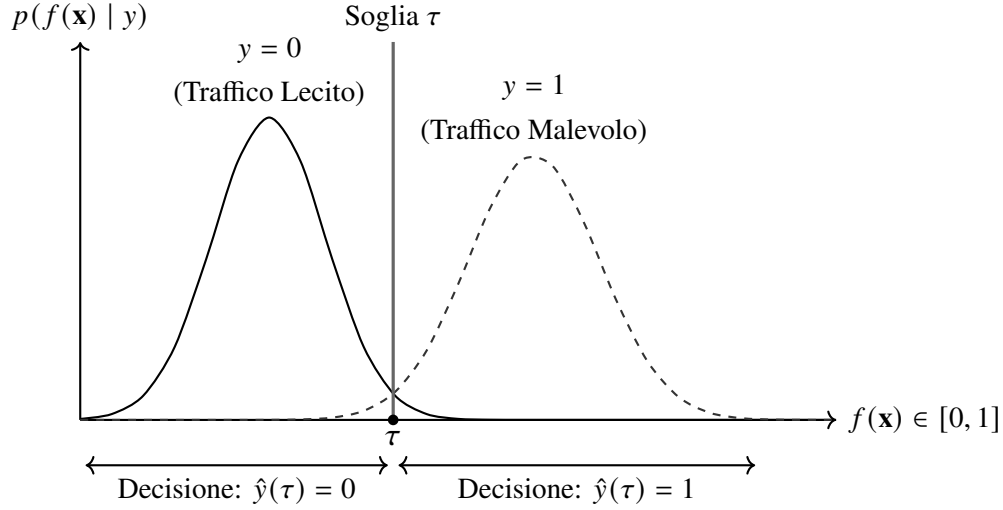


Figura 3.8: Mappatura della probabilità a posteriori $f(\mathbf{x})$ sulla regola decisionale binaria $\hat{y}(\tau)$ in funzione della soglia τ .

Formalizzazione delle Metriche Derivate

Per rendere la valutazione indipendente dalla numerosità del campione di test, si introducono i principali tassi relativi derivati dalla matrice di confusione [9]:

$$TPR(\tau) = \frac{TP(\tau)}{TP(\tau) + FN(\tau)}, \quad FPR(\tau) = \frac{FP(\tau)}{FP(\tau) + TN(\tau)} \quad (3.52)$$

cui corrispondono i rispettivi complementari

$$\begin{aligned} FNR(\tau) &= 1 - TPR(\tau) & TNR(\tau) &= 1 - FPR(\tau) \\ &= \frac{FN(\tau)}{TP(\tau) + FN(\tau)}, & &= \frac{TN(\tau)}{TN(\tau) + FP(\tau)}. \end{aligned} \quad (3.53)$$

L'affidabilità delle segnalazioni generate dal classificatore è invece misurata dalla *Precision*:

$$Precision(\tau) = \frac{TP(\tau)}{TP(\tau) + FP(\tau)} \quad (3.54)$$

Per sintetizzare il compromesso tra capacità di rilevamento degli attacchi e contenimento dei falsi allarmi si utilizza l'*FI-score*, definito come media armonica tra Precision e Recall²²:

²²Nei sistemi NIDS la media armonica è preferita alla media aritmetica poiché penalizza maggiormente modelli caratterizzati da un forte sbilanciamento tra sensibilità agli attacchi e precisione delle segnalazioni.

$$F_1(\tau) = 2 \cdot \frac{Precision(\tau) \cdot TPR(\tau)}{Precision(\tau) + TPR(\tau)} \quad (3.55)$$

Infine, l'*Accuracy* rappresenta la proporzione complessiva di classificazioni corrette:

$$Accuracy(\tau) = \frac{TP(\tau) + TN(\tau)}{N} \quad (3.56)$$

Tale indicatore assume tuttavia un ruolo secondario nei problemi di intrusion detection, poiché non risulta sufficientemente informativo in presenza di distribuzioni fortemente sbilanciate tra traffico nominale e traffico malevolo.

Dinamica della Soglia e Analisi Grafica (ROC e PR AUC)

La valutazione puntuale delle metriche presuppone la scelta di uno specifico valore della soglia decisionale (ad esempio $\tau = 0.5$). Una caratterizzazione più completa del comportamento discriminativo del classificatore richiede invece l'analisi dell'intero intervallo di valori della soglia, $\tau \in [0, 1]$.

A tale scopo si impiegano le curve *Receiver Operating Characteristic* (ROC) e *Precision–Recall* (PR), sintetizzate mediante l'area sottesa alla rispettiva curva (*Area Under the Curve*, AUC):

$$AUC_{ROC} = \int_0^1 TPR d(FPR), \quad AUC_{PR} = \int_0^1 Precision d(TPR) \quad (3.57)$$

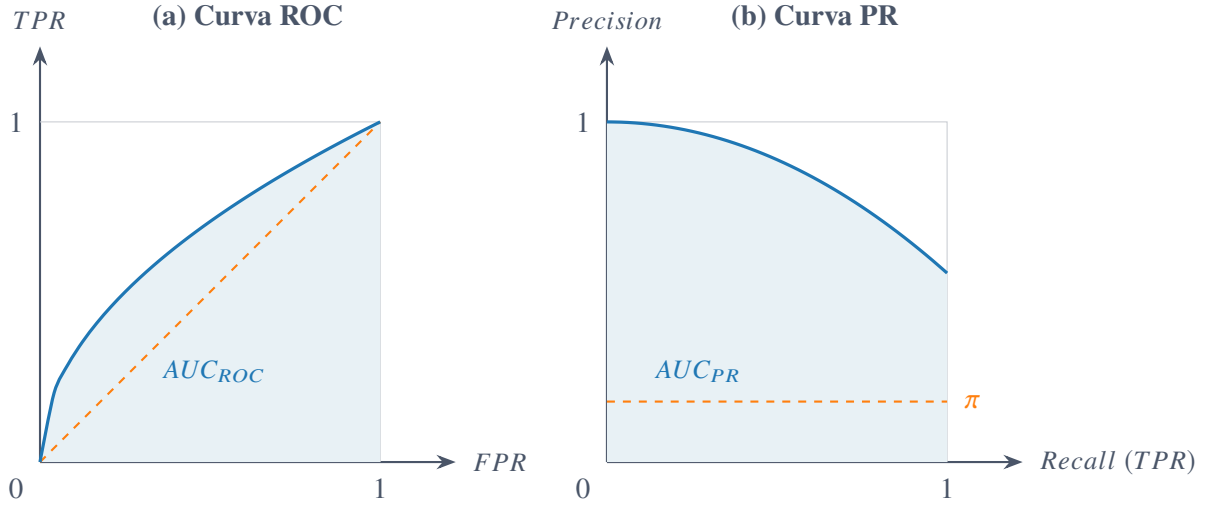


Figura 3.9: Rappresentazione schematica dell'Area Sottesa alla Curva (AUC) per le analisi grafiche ROC (a) e Precision-Recall (b).

L'indice AUC_{ROC} fornisce una misura della capacità discriminativa complessiva del classificatore, mentre AUC_{PR} risulta particolarmente indicato nei sistemi NIDS, dove la forte prevalenza del traffico legittimo rende più significativa la valutazione del compromesso tra Precision e Recall [28].

Degrado Prestazionale e Significatività Statistica (Welch's t-test)

Il trasferimento di un modello tra domini caratterizzati da differenti distribuzioni marginali delle caratteristiche $P(\mathbf{x})$ (*covariate shift*) può determinare una degradazione delle prestazioni di classificazione [24]. Tale variazione dell'efficacia discriminativa di un modello M , valutato sul dominio sorgente $\mathcal{D}_S$ e sul dominio target $\mathcal{D}_T$, può essere espressa come

$$\Delta F_1 = F_1(M, \mathcal{D}_S) - F_1(M, \mathcal{D}_T). \quad (3.58)$$

Per valutare la significatività statistica delle differenze prestazionali tra modelli differenti, oppure tra un modello e una baseline di riferimento, si adotta il test t di Welch, appropriato nel caso di campioni indipendenti con varianze non necessariamente uguali. Date due distribuzioni campionarie della metrica di interesse, caratterizzate da medie $\bar{x}_1, \bar{x}_2$, varianze s_1^2, s_2^2 e numerosità n_1, n_2 , la statistica del test è definita da

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}. \quad (3.59)$$

I corrispondenti gradi di libertà effettivi sono stimati mediante l'approssimazione di Welch–Satterthwaite:

$$\nu \approx \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{(s_1^2/n_1)^2}{n_1-1} + \frac{(s_2^2/n_2)^2}{n_2-1}}. \quad (3.60)$$

L'ipotesi nulla di uguaglianza delle medie,

$$H_0 : \mu_1 = \mu_2,$$

viene respinta qualora il valore p associato alla statistica t risulti inferiore al livello di significatività prefissato α (tipicamente $\alpha = 0.05$), indicando che la differenza prestazionale osservata è statisticamente significativa e non può essere ragionevolmente attribuita al solo errore campionario.

3.5 Sintesi del Capitolo e Raccordo con la Campagna

Sperimentale

L'impalcatura teorica e metodologica formalizzata nel presente capitolo definisce un modello astratto e rigoroso per la valutazione dei sistemi NIDS soggetti a *domain shift*. Attraverso l'allineamento degli spazi delle *feature*, la formalizzazione dei modelli di classificazione (aggregati e specialistici) e la definizione di un protocollo stocastico fondato su valutazioni *multi-seed* e test di significatività, il framework garantisce l'isolamento dei reali contributi algoritmici dal rumore di misura.

Tuttavia, la validità dei vincoli analitici e delle proprietà di invarianza descritte richiede un riscontro quantitativo su scenari operativi concreti. Nel Capitolo 4, il framework teorico viene pertanto istanziato ed eseguito empiricamente. Verranno presentati la caratterizzazione dei dataset di traffico reale, il setup infrastrutturale di addestramento e la discussione critica dei risultati ottenuti, valutando le capacità di generalizzazione e le prestazioni dei modelli al variare dei contesti di rete.

4. Implementazione del Framework Metodologico

Il presente capitolo traduce l'impalcatura teorica e il framework metodologico, definiti nel capitolo precedente, in una configurazione sperimentale concreta e riproducibile. L'obiettivo primario risiede nel dettagliare l'istanziamento software, le scelte architetturelle e i protocolli operativi impiegati per l'addestramento e la valutazione dei sistemi di *Network Intrusion Detection* (NIDS), rimanendo esplicitamente l'analisi dei risultati empirici e delle metriche prestazionali al Capitolo 5. Questa trasposizione materiale garantisce la totale trasparenza tecnologica e fornisce gli strumenti implementativi necessari per quantificare oggettivamente i fenomeni di *domain shift*.

La trattazione si articola secondo una progressione logica suddivisa in sette sezioni principali:

- La **Sezione 4.1** descrive l'ambiente sperimentale, dettagliando l'organizzazione modulare della repository software, lo stack tecnologico adottato e i rigorosi meccanismi di isolamento per la gestione della riproducibilità stocastica.
- La **Sezione 4.2** analizza le fonti di dati originarie, definendo la caratterizzazione del dominio sorgente e dei segmenti di destinazione, per poi formalizzare la parametrizzazione che guida la composizione fisica dei benchmark aggregati e ristretti.
- La **Sezione 4.3** illustra la pipeline di pre-elaborazione, declinando operativamente l'operatore di mappatura per l'allineamento degli spazi delle *feature* e giustificando la scelta architetturelle di omettere la scalatura esplicita globale.
- La **Sezione 4.4** dettaglia l'infrastruttura di classificazione supervisionata, specificando gli algoritmi di base, la configurazione degli iperparametri e le logiche di istanziamento dei modelli aggregati e biclasse.
- La **Sezione 4.5** completa la descrizione del framework operativo definendo l'orchestrazione gerarchica del ciclo di addestramento *multi-seed*, le procedure di inferenza sugli insiemi di collaudo e le logiche costruttive delle matrici di valutazione.

- La **Sezione 4.6** formalizza l’implementazione degli strumenti diagnostici e di spiegabilità, integrando le proiezioni PCA, le stime di densità KDE con ancoraggio al sorgente e l’infrastruttura TreeSHAP per l’interpretazione dei modelli ad albero.
- Infine, la **Sezione 4.7** fornisce la sintesi dell’architettura software end-to-end, illustrando la separazione funzionale tra il flusso computazionale principale di addestramento/valutazione e il ramo diagnostico.

4.1 Ambiente Sperimentale e Riproducibilità

Al fine di garantire la totale trasparenza e la riproducibilità end-to-end degli esperimenti, la presente sezione dettaglia la struttura della repository software, le dipendenze tecnologiche impiegate e i meccanismi di controllo dell’aleatorietà stocastica.

4.1.1 Repository e Organizzazione del Codice

L’intera architettura software sviluppata è stata resa pubblicamente accessibile su piattaforma GitHub¹. Il progetto, distribuito sotto licenza open-source MIT alla versione v4.0.0, è organizzato secondo una struttura modulare concepita per disaccoppiare la logica di pre-elaborazione dei dati, la definizione dei modelli, l’esecuzione della suite sperimentale e la produzione degli artefatti documentali.

La gerarchia fondamentale delle directory, illustrata nella Figura 4.1, si articola nei seguenti moduli principali:

- `src/`: racchiude il core logico dell’applicazione, integrando i moduli di pre-elaborazione (`data_preprocessing.py`), definizione dei classificatori (`models.py`), esecuzione delle procedure di classificazione (`classification.py`) e generazione grafica (`plotting.py`). La sotto-cartella `utils/` gestisce le configurazioni di sistema (`config.py`), l’I/O su disco (`file_utils.py`) e la formalizzazione delle metriche (`metrics.py`).
- `data/`: destinata alla memorizzazione dei dataset nello stato successivo alle fasi di pulizia e trasformazione (`preprocessed/`) unitamente ai relativi metadati.

¹<https://github.com/marchesinia187179/earth-trained-satellite-ids>

- `runs/`: ambiente di persistenza strutturato per tipologia di modello usato per il confronto con la baseline (es. `hybrid_models/`, `nb15_models/`) e scomposizione per seme stocastico. Ogni cartella raccoglie gli artefatti serializzati dei modelli (`models/`), le misurazioni grezze (`results/`) e i prodotti delle analisi di significatività statistica (es. test t di Welch e mappe di calore).
- `doc/`: contiene i sorgenti $\text{\LaTeX}$ e le risorse grafiche della documentazione scientifica.

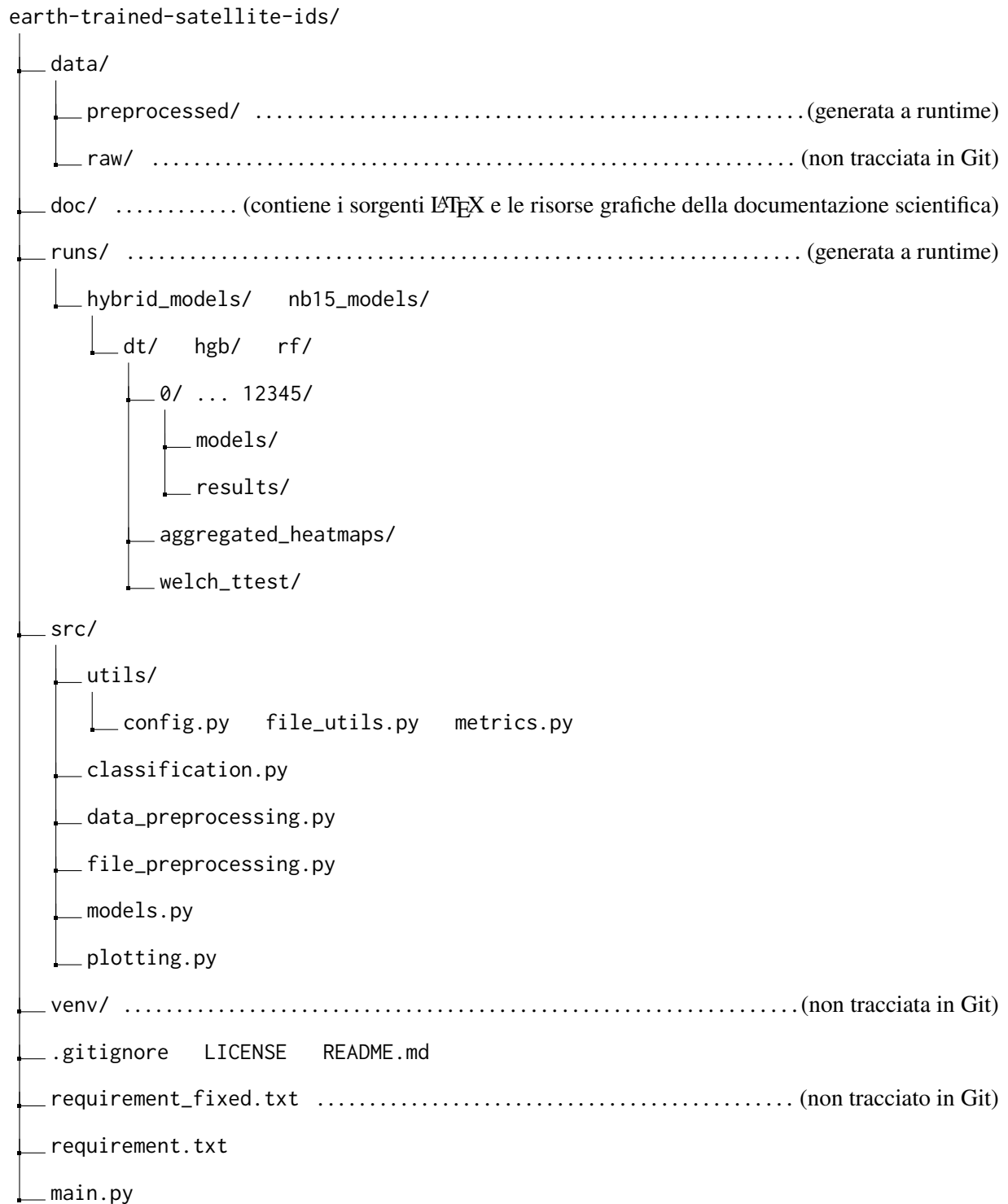


Figura 4.1: Rappresentazione schematica della gerarchia delle directory del progetto e delle risorse generate a runtime.

L'esecuzione dell'infrastruttura non richiede configurazioni manuali complesse. L'avvio del

punto di ingresso principale (`main.py`), previa installazione delle dipendenze dichiarate nel file di testo `requirements.txt`, attiva un'interfaccia a riga di comando (CLI) interattiva. Tale interfaccia guida l'operatore attraverso le fasi di preparazione dei dati, addestramento multi-seed e generazione delle analisi comparative.

4.1.2 Stack Tecnologico e Dipendenze

L'infrastruttura software è stata sviluppata adottando l'interprete Python (v3.14.4) quale ambiente di esecuzione. Per isolare il runtime e prevenire conflitti con librerie di sistema, le dipendenze sono state gestite in un ambiente virtuale dedicato tramite il modulo nativo `venv`.

Le librerie e i pacchetti impiegati per la manipolazione dati, l'addestramento dei modelli e le verifiche statistiche sono riassunti nella Tabella 4.1.

Tabella 4.1: Stack tecnologico, librerie e relative versioni vincolate nell'ambiente di runtime.

Libreria	Versione	Scopo Applicativo
Python ²	3.14.4	Ambiente di esecuzione e interprete base
pandas ³	3.0.3	Manipolazione ed elaborazione di strutture dati tabulari
numpy ⁴	2.5.1	Calcolo scientifico e operazioni vettoriali ad alte prestazioni
scikit-learn ⁵	1.9.0	Pre-elaborazione <i>feature</i> e addestramento classificatori
scipy ⁶	1.18.0	Verifiche di significatività statistica (es. test <i>t</i> di Welch)
shap ⁷	0.52.0	Calcolo dei valori Shapley per l'analisi di <i>explainability</i> (XAI)
joblib ⁸	1.5.3	Serializzazione e caricamento efficiente dei modelli su disco
matplotlib ⁹	3.11.1	Generazione di grafici e diagrammi prestazionali
seaborn ¹⁰	0.13.2	Visualizzazione avanzata e mappe di calore comparative

²<https://www.python.org/>

³<https://pandas.pydata.org/>

⁴<https://numpy.org/>

⁵<https://scikit-learn.org/>

⁶<https://scipy.org/>

⁷<https://github.com/slundberg/shap>

⁸<https://joblib.readthedocs.io/>

⁹<https://matplotlib.org/>

¹⁰<https://seaborn.pydata.org/>

La suite sperimentale è stata eseguita su un calcolatore Apple MacBook Air 2020 (modello A2179), equipaggiato con un processore quad-core Intel Core i5, una GPU Intel Iris Plus Graphics e con 8 GB di memoria RAM unificata¹¹. Per azzerare l’overhead della virtualizzazione e garantire un controllo diretto sulle risorse hardware, l’esecuzione è avvenuta in ambiente nativo *dual-boot* sotto distribuzione Linux Xubuntu.

4.1.3 Gestione della Riproducibilità Stocastica

Il controllo della variabilità stocastica negli algoritmi di apprendimento automatico costituisce un requisito imprescindibile del framework. L’aleatorietà viene gestita separando nettamente la fase di preparazione deterministica del dato dall’addestramento dei classificatori.

La fase di pulizia e trasformazione del dataset originale è vincolata in modo univoco al seme stocastico primario $s_0 = 127001$. Tale vincolo assicura che la generazione dei metadati e il salvataggio dei dati pre-elaborati in `data/preprocessed/` avvengano in maniera deterministica e immutabile.

La campagna di addestramento e validazione si sviluppa invece su un insieme di $K = 10$ semi stocastici distinti:

$$\mathcal{S} = \{0, 1, 7, 42, 101, 123, 999, 1337, 2026, 12345\} \quad (4.1)$$

L’iniezione del controllo stocastico avviene tramite l’assegnazione esplicita del parametro `random_state` nelle routine di `scikit-learn`. Per evitare contaminazioni incrociate (*cross-seed contamination*), il framework garantisce l’isolamento dei processi: ogni esecuzione riferita a un seme $s_k \in \mathcal{S}$ opera in maniera autonoma, leggendo le risorse e scrivendo i propri artefatti esclusivamente nella directory dedicata (es. `runs/hybrid_models/rf/42/`).

¹¹Tale configurazione impone un vincolo operativo sull’impronta mnemonica (*memory footprint*) dell’ambiente di calcolo, richiedendo un’ottimizzazione accurata delle strutture dati `pandas.DataFrame` e la profilazione dei vettori `numpy` durante la fase di allineamento e trasformazione delle feature.

4.2 Caratterizzazione dei Dataset e Composizione dei Benchmark

La presente sezione è dedicata alla traduzione dell'impianto teorico del framework, definito nel capitolo precedente, in una configurazione sperimentale concreta e riproducibile. Affinché i modelli di *Intrusion Detection* possano essere addestrati e valutati rispetto alle sfide imposte dal *domain shift*, è fondamentale istanziare fisicamente il dominio sorgente ($\mathcal{D}_S$) e i domini di destinazione ($\mathcal{D}_T$). Nelle sottosezioni seguenti verranno analizzate le fonti di dati originarie, descrivendo le tassonomie di minaccia native e le relative procedure di pulizia e normalizzazione (Sezioni 4.2.1 e 4.2.2). Infine, la Sezione 4.2.3 formalizzerà l'algoritmo di campionamento e scalatura che ha guidato l'aggregazione di tali fonti, culminando nella generazione dei benchmark fisici standardizzati su cui poggeranno le successive fasi di addestramento e classificazione.

4.2.1 Dominio Sorgente: UNSW-NB15

Il dataset UNSW-NB15 [23]¹² è stato adottato come rappresentazione concreta del dominio sorgente $\mathcal{D}_S$. Il dataset, acquisito nella sua formulazione tabulare in formato CSV, comprende inizialmente un volume complessivo di 257.673 record.

La tassonomia nativa delle minacce, indicata formalmente con $\mathcal{A}_S$, include oltre alla classe di traffico lecito (*Normal*) nove categorie di attacco: *Fuzzers*, *Analysis*, *Backdoors*, *DoS*, *Exploits*, *Generic*, *Reconnaissance*, *Shellcode* e *Worms*.

Al fine di prevenire il degrado prestazionale dei modelli di apprendimento automatico causato da un marcato sbilanciamento delle classi, è stata applicata una procedura di rimozione delle classi fortemente minoritarie (*minority removal*), in accordo con la metodologia validata in letteratura da Azar et al. [2]. In particolare, sono state eliminate quattro categorie di attacco caratterizzate da una frequenza relativa estremamente ridotta nell'insieme originario:

- *Analysis* (1,141% delle istanze totali);
- *Backdoors* (0,996% delle istanze totali);

¹²<https://research.unsw.edu.au/projects/unsw-nb15-dataset>

- *Shellcode* (0,646% delle istanze totali);
- *Worms* (0,074% delle istanze totali).

A seguito dell'eliminazione delle classi minoritarie, la volumetria complessiva del dominio sorgente si attesta a 250.982 record. La distribuzione dettagliata delle istanze per le sei classi residue è riassunta nella Tabella 4.2. Il successivo campionamento e la scalatura in funzione del parametro ρ vengono formalizzati nella Sezione 4.2.3.

Tabella 4.2: Distribuzione delle istanze del dataset UNSW-NB15 a seguito della procedura di *minority removal*.

Classe Nativa	Tipologia	Conteggio Istanze
<i>Normal</i>	Traffico Lecito	93.000
<i>Generic</i>	Attacco Malevolo	58.871
<i>Exploits</i>	Attacco Malevolo	44.525
<i>Fuzzers</i>	Attacco Malevolo	24.246
<i>DoS</i>	Attacco Malevolo	16.353
<i>Reconnaissance</i>	Attacco Malevolo	13.987
Totale		250.982

4.2.2 Domini di Destinazione: Segmento Satellitare/Terrestre STIN

I domini di destinazione $\mathcal{D}_T$ sono stati formalizzati a partire dal dataset STIN (*Satellite-Terrestrial Integrated Network*) [18]¹³, focalizzato sull'identificazione di minacce informatiche in architetture di rete e infrastrutture spaziali integrate. Acquisiti nella formulazione tabulare in formato CSV, a differenza del dominio sorgente $\mathcal{D}_S$, i dataset target considerati contengono esclusivamente traffico malevolo, con una totale assenza di campioni appartenenti alla classe di traffico lecito (*Normal*)¹⁴.

¹³<https://github.com/kun9717/STIN-data-set/>

¹⁴La composizione asimmetrica dei dataset STIN è determinata dalla struttura del testbed di cattura originario, progettato unicamente per isolare ed etichettare flussi di attacco in scenari operativi reali. Di conseguenza, per la costruzione dei benchmark di valutazione biclasse cross-dominio, la componente di traffico benigno ($Y = 0$) viene estratta in modo coerente dal dominio sorgente UNSW-NB15, garantendo l'omogeneità del traffico lecito e preservando il rapporto di sbilanciamento prefissato ($\rho = 10$).

La caratterizzazione del target si articola su due segmenti operativi distinti:

- **Segmento Terrestre (TER20):** Rappresenta il traffico rilevato dai nodi di terra dell'infrastruttura STIN.
- **Segmento Satellitare (SAT20):** Modellizza il traffico intercettato dai vettori spaziali, precisamente quelli satellitari.

Segmento Terrestre (TER20)

Il dataset TER20 comprende originariamente 127.244 record ripartiti su dieci classi native di attacco: *Botnet*, *DDoS*, *Syn_DDoS*, *UDP_DDoS*, *Web Attack*, *Backdoor*, *LDAP_DDoS*, *MSSQL_DDoS*, *NetBIOS_DDoS* e *Portmap_DDoS*.

In linea con la metodologia di accorpamento delle classi minoritarie (*minority merge*) proposta da Azar et al. [2], le categorie con caratteristiche funzionali sovrapponibili e frequenza ridotta sono state raggruppate per preservare la significatività statistica e riflettere la tassonomia delle macro-minacce. Nello specifico:

1. Le classi *Web Attack* e *Backdoor* sono state accorpate alla classe principale *Botnet*;
2. Le sottoclassi di attacco volumetrico *LDAP_DDoS*, *MSSQL_DDoS*, *NetBIOS_DDoS* e *Portmap_DDoS* sono state fuse nella macro-categoria *DDoS*.

A seguito del processo di *minority merge*, la volumetria complessiva rimane invariata a 127.244 record, strutturata nelle quattro macro-classi risultanti riportate nella Tabella 4.3.

Tabella 4.3: Distribuzione delle istanze del dataset TER20 a seguito del processo di *minority merge*.

Macro-Classe Target ($\mathcal{A}_T$)	Classi Native Accorpate	Conteggio Istanze
<i>DDoS</i>	<i>DDoS</i> , <i>LDAP_DDoS</i> , <i>MSSQL_DDoS</i> , <i>NetBIOS_DDoS</i> , <i>Portmap_DDoS</i>	57.292
<i>Botnet</i>	<i>Botnet</i> , <i>Web Attack</i> , <i>Backdoor</i>	40.401
<i>Syn_DDoS</i>	<i>Syn_DDoS</i>	15.713
<i>UDP_DDoS</i>	<i>UDP_DDoS</i>	13.838
Totale		127.244

Segmento Satellitare (SAT20)

Il dataset SAT20 include un totale di 82.320 istanze, focalizzate in modo specifico sulle minacce dirette al segmento spaziale. La tassonomia nativa si articola esclusivamente su due vettori di attacco volumetrico: *UDP_DDoS* e *Syn_DDoS*. Vista l'assenza di classi marginali o fortemente sbilanciate, non è stata applicata alcuna procedura di *minority merge*. La distribuzione finale del segmento SAT20 è dettagliata nella Tabella 4.4.

Tabella 4.4: Distribuzione delle istanze del dataset SAT20.

Classe Target ($\mathcal{A}_T$)	Tipologia Minaccia	Conteggio Istanze
<i>UDP_DDoS</i>	Attacco Volumetrico Satellitare	43.244
<i>Syn_DDoS</i>	Attacco Volumetrico Satellitare	39.076
Totale		82.320

Per entrambi i dataset target, il successivo campionamento e la scalatura in funzione del parametro ρ vengono formalizzati nella Sezione 4.2.3.

4.2.3 Parametrizzazione e Istanziatura dei Benchmark

Al fine di garantire un confronto rigoroso, equo e immune da bias legati alle fluttuazioni nella cardinalità dei dati tra i diversi scenari di test, la costruzione materiale dei benchmark fisici a partire dai domini sorgente e target ($\mathcal{D}_S$ e $\mathcal{D}_T$) è stata sottoposta a un rigido vincolo di bilanciamento matematico. Il parametro di scalatura e proporzione tra classi, definito formalmente come ρ , è stato fissato a un valore numerico pari a 10. Tale configurazione (NORMAL_ANOMALY_RATIO nell'infrastruttura di codice) impone che per ogni istanza appartenente allo spazio delle etichette primarie malevole ($Y = 1$), siano presenti esattamente dieci istanze relative al traffico lecito ($Y = 0$).

Il processo di istanziatura opera mediante un campionamento stratificato senza reinserimento (*without-replacement stratified sampling*). Poiché l'operatore di cardinalità applicato alla componente benevola del dominio sorgente restituisce $N(\mathbf{x} \in \mathcal{D}_S \mid Y = 0) = 93.000$ record, l'algoritmo vincola l'estrazione della componente anomala a esattamente 9.300 istanze complessive per ciascun

benchmark. La partizione dei domini in insiemi di addestramento e collaudo ($\mathcal{D}_k^{(train)}$ e $\mathcal{D}_k^{(test)}$) è mantenuta costante attraverso uno *split* dell'80% per il training e del 20% per il test.

La realizzazione pratica del vincolo di bilanciamento $\rho = 10$ e del campionamento senza reinserimento è affidata alle routine ausiliarie implementate nel modulo `file_preprocessing.py`.

In particolare, la funzione `_get_correct_normal_and_anomaly_data_samples` valuta le cardinalità relative delle componenti lecite e anomale, determinando dinamicamente il volume di campioni da estrarre per rispettare la costante `MLConstants.NORMAL_ANOMALY_RATIO`. Successivamente, la funzione `_safe_stratified_sample` garantisce che il campionamento mantenga inalterata la proporzione originaria tra le partizioni di addestramento e collaudo (*split_type*), prevenendo fenomeni di sbilanciamento tra i sotto-insiemi *train* e *test*.

Codice 4.1: Routine di campionamento per l'istanziatura dei benchmark.

```
1 def _get_correct_normal_and_anomaly_data_samples(normal_data, anomaly_data,
2                                                  normal_samples, anomaly_samples):
3     # Calcola il corretto numero di campioni da estrarre dalle distribuzioni
4     if normal_samples < anomaly_samples * MLConstants.NORMAL_ANOMALY_RATIO:
5         n_anomaly_target = int(
6             normal_samples / MLConstants.NORMAL_ANOMALY_RATIO
7         )
8         anomaly_data_sample = _safe_stratified_sample(
9             anomaly_data, n_anomaly_target
10        )
11        normal_data_sample = normal_data
12    else:
13        n_normal_target = int(
14            anomaly_samples * MLConstants.NORMAL_ANOMALY_RATIO
15        )
16        normal_data_sample = _safe_stratified_sample(
17            normal_data, n_normal_target
18        )
19        anomaly_data_sample = anomaly_data
20
21    return normal_data_sample, anomaly_data_sample
```

Codice 4.2: Routine di bilanciamento stratificato per l'istanziamento dei benchmark.

```
1 def _safe_stratified_sample(data, n_samples):
2     # Se la colonna 'split_type' non è presente,
3     # esegue un campionamento casuale semplice
4     if 'split_type' not in data.columns:
5         return data.sample(n=n_samples, random_state=MLConstants.MAIN_SEED)
6
7     # Suddivide il dataset in partizioni di train e test
8     data_train = data[data['split_type'] == 'train']
9     data_test = data[data['split_type'] == 'test']
10
11     # Calcola la lunghezza totale del dataset
12     total_len = len(data)
13     if total_len == 0 or n_samples == 0:
14         return pd.DataFrame(columns=data.columns)
15
16     train_ratio = len(data_train) / total_len
17     n_train = round(n_samples * train_ratio)
18     n_test = n_samples - n_train
19
20     # Gestione difensiva dei limiti di capienza delle partizioni
21     if n_test > len(data_test):
22         diff = n_test - len(data_test)
23         n_test = len(data_test)
24         n_train += diff
25     elif n_train > len(data_train):
26         diff = n_train - len(data_train)
27         n_train = len(data_train)
28         n_test += diff
29
30     # Campionamento stratificato senza reinserimento
31     sampled_parts = []
32     if n_train > 0 and not data_train.empty:
33         sampled_parts.append(data_train.sample(
34             n=n_train, random_state=MLConstants.MAIN_SEED
35         ))
```

```

36     if n_test > 0 and not data_test.empty:
37         sampled_parts.append(data_test.sample(
38             n=n_test, random_state=MLConstants.MAIN_SEED
39         ))
40
41     if not sampled_parts:
42         return pd.DataFrame(columns=data.columns)
43
44     return concat_and_shuffle(sampled_parts)

```

L'applicazione di questa logica ha permesso di istanziare i dataset aggregati previsti dal framework teorico. In tutte le configurazioni, la componente benevola corrisponde invariabilmente alle distribuzioni stazionarie originarie del dominio sorgente, $P_S(\mathbf{x} \mid Y = 0)$. La differenziazione degli scenari avviene modulando la componente malevola ($Y = 1$) in funzione della famiglia di attacco esplorata ($A \in \mathcal{A}_S$ oppure $A \in \mathcal{A}_T$):

1. **Set di Iniezione di Riferimento (INJECTION):** Costituisce la baseline proposta dall'architettura ($\mathcal{M}_{base}$). Progettato per fornire un'ancora di prestazione basata su *few-shot learning*, questo set combina le minacce sorgente $P_S(\mathbf{x} \mid Y = 1, A \in \mathcal{A}_S)$ (pari a 9.271 istanze) con l'infusione controllata di una porzione altamente sparsa di traffico target $\Delta\mathcal{D}_T$. La cardinalità di tale quota di iniezione è determinata formalmente dalla relazione:

$$|\Delta\mathcal{D}_T| = \lceil N_{\text{mal},T} \times \eta_{\text{inj}} \rceil = \lceil 9.300 \times 0,003 \rceil = 29 \text{ campioni} \quad (4.2)$$

dove $N_{\text{mal},T} = 9.300$ rappresenta la popolazione anomala totale e $\eta_{\text{inj}} = 0,3\%$ la percentuale di iniezione prescritta. L'operatore d'arrotondamento per eccesso ($\lceil \cdot \rceil$) garantisce il rispetto della soglia minima teorica.

2. **Set Nativo Sorgente (NB15):** Rappresenta il NIDS tradizionale ($\mathcal{M}_S$). La componente anomala ammonta a 9.300 istanze appartenenti esclusivamente alla tassonomia nativa sorgente $\mathcal{A}_S$. È impiegato per misurare analiticamente la degradazione prestazionale causata dal *domain shift* in totale assenza di adattamento al target.
3. **Set Ibridi Aggregati (NB15_SAT20, NB15_TER20, NB15_STIN):** Formulati per addestrare i modelli aggregati ibridi ($\mathcal{M}_H^{(T)}$), questi dataset istituiscono il limite teorico di rilevamento

(*upper bound*, paradigma Oracle). La componente anomala di 9.300 record è interamente governata dalle distribuzioni a priori del dominio target $P_T(\mathbf{x} \mid Y = 1, A \in \mathcal{A}_T)$, prelevate rispettivamente dal segmento spaziale (SAT20), terrestre (TER20) o dall’ecosistema olistico congiunto (STIN).

Grazie al vincolo imposto da ρ , i cinque benchmark aggregati presentano un’architettura dimensionale e un partizionamento perfettamente identici, come esposto nella Tabella 4.5. Tale standardizzazione volumetrica neutralizza il rischio di divergenze prestazionali derivanti dalle variazioni della mole campionaria.

Tabella 4.5: Riepilogo delle cardinalità finali per i benchmark aggregati, con partizionamento 80/20 e vincolo $\rho = 10$. Il modello INJECTION funge da baseline di riferimento dell’infrastruttura.

Benchmark	Istanze Normali <i>Normal $Y = 0$</i>	Istanze Anomale <i>$Y = 1$</i>	Totale Istanze	Training Set <i>(80%)</i>	Test Set <i>(20%)</i>
INJECTION (Baseline)	93.000	9.300	102.300	81.840	20.460
NB15	93.000	9.300	102.300	81.840	20.460
NB15_SAT20	93.000	9.300	102.300	81.840	20.460
NB15_TER20	93.000	9.300	102.300	81.840	20.460
NB15_STIN	93.000	9.300	102.300	81.840	20.460

Infine, a partire dai macro-dataset aggregati e scalati finora descritti, la pipeline di pre-elaborazione procede alla derivazione sistematica dei Set Nativi Sorgente Biclasse e dei Set Ibridi Biclasse. Tramite un filtraggio a livello di etichetta secondaria $A = a$, le singole famiglie di minaccia (es. $a \in \mathcal{A}_S$ per NB15 o $a \in \mathcal{A}_T$ per STIN) vengono isolate e combinate esclusivamente con la baseline nativa lecita $P_S(\mathbf{x} \mid Y = 0)$. Questi dataset ristretti risultano indispensabili per l’addestramento dei rilevatori specialistici ($\mathcal{M}_{S,a}$ e $\mathcal{M}_{H,T,a}$), consentendo di valutare la sensibilità del sistema verso singole tipologie di attacco senza incorrere nelle distorsioni intrinseche generate dalla sovrapposizione di distribuzioni anomale eterogenee.

La struttura dettagliata degli 11 benchmark biclasse così generati — 5 per il dominio sorgente NB15, 4 per il segmento terrestre TER20 e 2 per il segmento satellitare SAT20 — è sintetizzata nella Tabella 4.6. Come si evince dai dati di partizionamento, l’applicazione rigorosa del rapporto

$\rho = 10$ garantisce che ciascun rilevatore specialistico ($\mathcal{M}_{S,a}$ o $\mathcal{M}_{H,T,a}$) venga addestrato su una distribuzione uniforme composta da esattamente 93.000 campioni leciti ($Y = 0$) e 9.300 campioni malevoli ($Y = 1$) dedicati alla singola famiglia d'attacco a , mantenendo costante lo *split* 80/20 tra training (81.840 record) e test set (20.460 record).

Tabella 4.6: Prospetto sintetico della cardinalità e del partizionamento per i Set Nativi Sorgente Biclasse e Ibridi Biclasse ($\rho = 10$).

Dominio	Nome Dataset Biclasse (File CSV)	Istanze Normali Normal (Y = 0)	Istanze Attacco Classe a (Y = 1)	Totale Istanze	Train Set (80%)	Test Set (20%)
<i>Set Nativi Sorgente Biclasse</i>						
NB15	Normal_Generic.csv	93.000	9.300	102.300	81.840	20.460
NB15	Normal_Exploits.csv	93.000	9.300	102.300	81.840	20.460
NB15	Normal_Fuzzers.csv	93.000	9.300	102.300	81.840	20.460
NB15	Normal_DoS.csv	93.000	9.300	102.300	81.840	20.460
NB15	Normal_Reconnaissance.csv	93.000	9.300	102.300	81.840	20.460
<i>Set Ibridi Biclasse Terrestri</i>						
TER20	Normal_DDoS.csv	93.000	9.300	102.300	81.840	20.460
TER20	Normal_Botnet.csv	93.000	9.300	102.300	81.840	20.460
TER20	Normal_Syn_DDoS.csv	93.000	9.300	102.300	81.840	20.460
TER20	Normal_UDP_DDoS.csv	93.000	9.300	102.300	81.840	20.460
<i>Set Ibridi Biclasse Satellitari</i>						
SAT20	Normal_UDP_DDoS.csv	93.000	9.300	102.300	81.840	20.460
SAT20	Normal_Syn_DDoS.csv	93.000	9.300	102.300	81.840	20.460

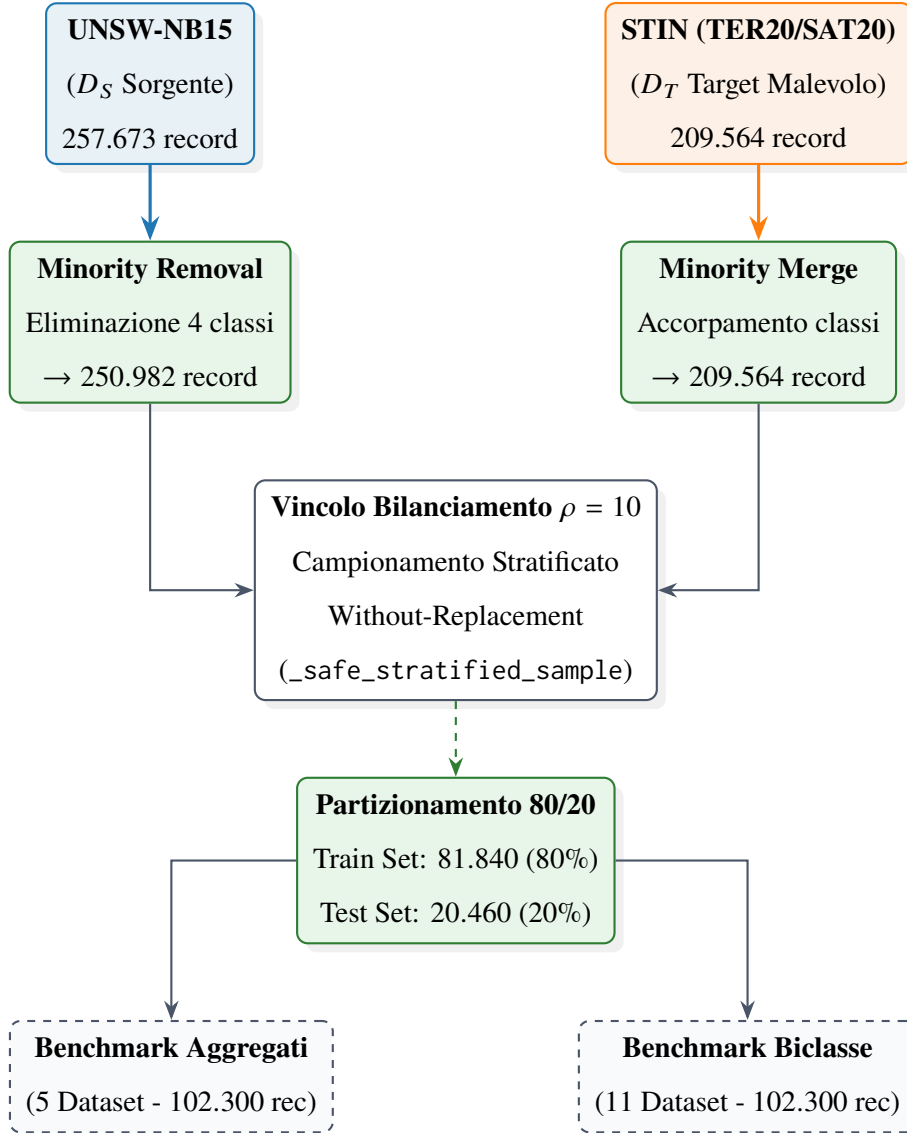


Figura 4.2: Diagramma di flusso del processo di estrazione, pulizia, campionamento vincolato ($\rho = 10$) e partizionamento stratificato 80/20 per la generazione dei benchmark fisici aggregati e biclasse.

4.3 Allineamento delle Feature e Pipeline di Preprocessing

La presente sezione illustra l'istanziatura concreta e l'architettura software dell'operatore di mappatura $\psi_k : \mathcal{X}_k \rightarrow \mathcal{X}_{\text{shared}}$, formalizzato sul piano teorico nella Sezione 3.1.2. L'obiettivo primario della pipeline di preprocessing consiste nel colmare il divario strutturale e semantico esistente tra le rappresentazioni grezze del dominio sorgente ($\mathcal{D}_S$, basato su UNSW-NB15) e dei domini di

destinazione ($\mathcal{D}_T$, articolato sulle catture STIN TER20 e SAT20), proiettando i rispettivi spazi delle feature in uno spazio coordinato condiviso, omogeneo e privo di distorsioni di tracciamento.

L'architettura generale del flusso di elaborazione e la sua ripartizione in stadi sequenziali sono schematizzate nella Figura 4.3. La pipeline si sviluppa lungo tre fasi principali: un filtraggio preliminare per la rigida eliminazione di attributi non trasferibili o ridondanti, l'applicazione delle trasformazioni algebriche e analitiche definite dall'operatore ψ_k , e la strutturazione della tabella coordinata finale composta da $d = 16$ feature predittive continue e 3 variabili di controllo. Nelle sottosezioni seguenti vengono analizzati nel dettaglio i singoli criteri di selezione, la formulazione analitica delle trasformazioni e le proprietà di invarianza del grafo computazionale risultante.

Pipeline di Preprocessing e Armonizzazione

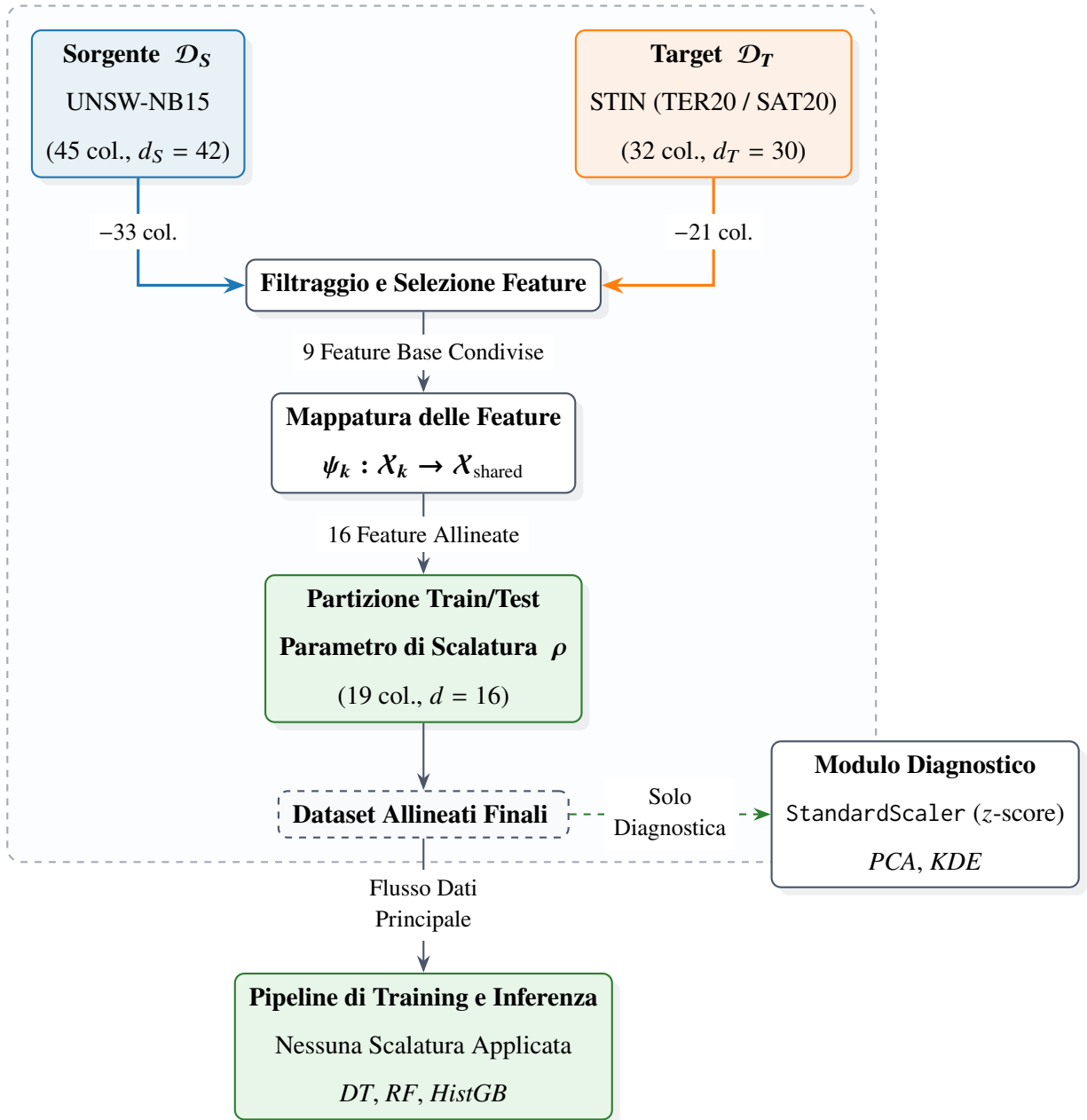


Figura 4.3: Diagramma di flusso della pipeline di preprocessing e armonizzazione tra il dominio sorgente $\mathcal{D}_S$ e i domini target $\mathcal{D}_T$, dall'estrazione delle feature grezze alla generazione dello spazio coordinato finale a 16 dimensioni.

4.3.1 Feature Grezze e Criteri di Filtraggio Applicati

La fase preliminare della pipeline di preprocessing è volta ad allineare gli spazi delle feature originariamente eterogenei del dominio sorgente $\mathcal{D}_S$ (UNSW-NB15) e dei domini di destinazione $\mathcal{D}_T$ (STIN, declinato in TER20 e SAT20). Il dataset sorgente si articola originariamente su un vettore di $d_S = 42$ attributi predittivi grezzi (a cui si aggiungono l'identificatore `id` e la doppia etichettatura nativa `attack_cat` e `label`, per un totale di 45 colonne). Diversamente, le sorgenti di destinazione presentano una struttura nativa derivata da catture di flusso composta da $d_T = 30$ attributi predittivi (unitamente all'identificatore `id` e all'etichetta binaria `label`, per un totale di 32 colonne).

Per garantire l'interoperabilità e la generalizzabilità dei modelli di apprendimento automatico rispetto al *domain shift*, la selezione dello spazio delle feature condiviso d è stata regolata da tre criteri di filtraggio sequenziali:

1. **Rimozione di identificatori univoci e metadati di tracciamento:** Sia nel dominio sorgente che in quelli target, l'attributo identificativo progressivo (`id`) è stato rimosso. Tale eliminazione è indispensabile per prevenire fenomeni di correlazione spuria o *overfitting* legati all'ordinamento sequenziale delle registrazioni di rete.
2. **Criterio di non calcolabilità e assenza di corrispondenza cross-dominio:** Sono stati scartati tutti gli attributi presenti esclusivamente in una delle due sorgenti che non potevano essere matematicamente o logicamente ricostruiti a partire dai dati dell'altra, o le cui metriche sono state scartate in favore di un ricalcolo run-time per garantire la consistenza tra i domini. La classificazione dettagliata delle feature eliminate per ciascun dominio in questa fase è sintetizzata nella Tabella 4.7.

Tabella 4.7: Feature eliminate durante la fase di pre-processing nei domini sorgente e target.

Dominio	Categoria	Feature Rimosse
Sorgente ($\mathcal{D}_S$)	Protocolli Applicativi	service, trans_depth, response_body_len, is_ftp_login, ct_ftp_cmd, ct_flw_http_mthd
	Contatori Aggregati	ct_srv_src, ct_state_ttl, ct_dst_ltm, ct_src_dport_ltm, ct_dst_sport_ltm, ct_dst_src_ltm, ct_src_ltm, ct_srv_dst
	TTL e Perdita Pacchetti	stttl, dtttl, sloss, dloss, is_sm_ips_ports
	Jitter, TCP e Metriche	sjit, djit, sinpkt, dinpkt, tcprtt, synack, ackdat, stcpb, dtcpb
	Metriche Ridondanti	smean, dmean, rate
Target ($\mathcal{D}_T$)	Statistiche Lunghezza	pkt_len_min, pkt_len_max, pkt_len_std, fw_hdr_len
	Flag TCP	bw_psh_flag, fw_urg_flag, bw_urg_flag, fin_cnt, syn_cnt, psh_cnt, urg_cnt, ece_cnt
	Intervalli Inter-arrivo	fl_iat_min, bw_iat_tot, bw_iat_min
	Contatori Burst	fw_byt_blk_avg, fw_pkt_blk_avg, fw_blk_rate_avg, bw_byt_blk_avg, bw_pkt_blk_avg, bw_blk_rate_avg

3. **Criterio di incompatibilità e impossibilità di armonizzazione:** Gli attributi categoriali ad alta cardinalità relativi al protocollo di trasporto (proto) e allo stato della connessione (state) presenti in UNSW-NB15 sono stati scartati. Tale scelta è stata motivata dall'assenza di attributi categoriali equivalenti nei dataset STIN e dall'impossibilità di stabilire una mappatura biunivoca senza introdurre codifiche arbitrarie che avrebbero distorto lo spazio delle feature.

A seguito dell'applicazione di tali criteri di scarto, in entrambi i domini si isola un sottoinsieme condiviso di 9 feature predittive di base. A partire da queste, mediante operazioni algebriche calcolate a tempo di esecuzione (quali somme, rapporti e differenze dirette), vengono derivate 7

ulteriori feature sintetiche (es. `total_bytes`, `pkts_per_sec`, `byte_ratio`). Si ottiene così uno spazio condiviso armonizzato finale composto da $d = 16$ feature predittive continue e quantitative.

Nei dataset pre-processati finali, a questo vettore di 16 dimensioni si affiancano 3 variabili di controllo e destinazione (`class` per la macro/micro tassonomia della minaccia, `label` per la classificazione binaria, e `split_type` per l'indicazione della partizione di addestramento o collaudo), portando la cardinalità complessiva delle tabelle a 19 colonne. La Tabella 4.8 riassume il processo logico di riduzione, derivazione e allineamento dimensionale.

Tabella 4.8: Sintesi del processo di selezione e allineamento dimensionale tra il dominio sorgente ($\mathcal{D}_S$) e i domini di destinazione ($\mathcal{D}_T$).

Dominio / Sorgente	Colonne Totali	Feature Predittive Grezze	Feature Scartate
Sorgente ($\mathcal{D}_S$)	45	$d_S = 42$	33
Target ($\mathcal{D}_T$)	32	$d_T = 30$	21
Spazio Allineato Finale	19	d = 16	–

4.3.2 Implementazione dell'Operatore di Mappatura

Al fine di proiettare gli spazi delle feature originari in uno spazio latente condiviso omogeneo, è stato implementato formalmente l'operatore di mappatura $\psi_k : \mathcal{X}_k \rightarrow \mathcal{X}_{\text{shared}}$, con $k \in \{S, T\}$. Tale operatore agisce sui vettori grezzi applicando trasformazioni algebriche, conversioni di unità di misura e derivazioni analitiche, garantendo la rigorosa coerenza semantica delle grandezze fisiche e di rete.

La Tabella 4.9 formalizza le trasformazioni scalari $\psi_{k,j}$ applicate a ciascuna feature condivisa j , mettendo a confronto il dominio sorgente (UNSW-NB15) e il dominio target (STIN/TER20/SAT20).

Tabella 4.9: Formulazione algebrica dell'operatore di mappatura vettoriale nei domini $\mathcal{D}_S$ e $\mathcal{D}_T$.

Feature Coordinata (x_j)	Dominio Sorgente ($\psi_{S,j}$)	Dominio Target ($\psi_{T,j}$)	Unità di Misura
duration	$dur \cdot 10^6$	fl_dur	μs
src_bytes	sbytes	l_fw_pkt	Byte
dst_bytes	dbytes	l_bw_pkt	Byte
src_pkts	spkts	fw_pk	Adimensionale
dst_pkts	dpkts	$(bw_pkt_s \cdot fl_dur)/10^6$	Adimensionale
src_win_byt	swin	fw_win_byt	Byte
dst_win_byt	dwin	bw_win_byt	Byte
load_s	$(sload + dload)/8$	fl_byt_s	Byte/s
down_up_ratio	$dpkts/(spkts + \epsilon)$	down_up_ratio	Adimensionale
total_bytes		$src_bytes + dst_bytes$	Byte
total_pkts		$src_pkts + dst_pkts$	Adimensionale
src_mean_pkt_size		$src_bytes/(src_pkts + \epsilon)$	Byte
dst_mean_pkt_size		$dst_bytes/(dst_pkts + \epsilon)$	Byte
win_diff		$src_win_byt - dst_win_byt$	Byte
byte_ratio		$dst_bytes/(src_bytes + \epsilon)$	Adimensionale
pkts_per_sec	$total_pkts/(dur + \epsilon)$	$[total_pkts/(duration + \epsilon)] \cdot 10^6$	1/s

Tabella 4.10: Descrizione semantica delle feature coordinate.

Feature Coordinata (x_j)	Descrizione
duration	Durata di esecuzione
src_bytes	Lunghezza totale dei pacchetti in direzione forward
dst_bytes	Lunghezza totale dei pacchetti in direzione backward
src_pkts	Numero totale di pacchetti in direzione forward
dst_pkts	Numero totale di pacchetti in direzione backward
src_win_byt	Lunghezza finestra iniziale in direzione forward
dst_win_byt	Lunghezza finestra iniziale in direzione backward
load_s	Throughput totale
down_up_ratio	Asimmetria dei pacchetti
total_bytes	Bytes totali
total_pkts	Pacchetti totali
src_mean_pkt_size	Dimensione media dei pacchetti in direzione forward
dst_mean_pkt_size	Dimensione media dei pacchetti in direzione backward
win_diff	Differenza di finestra
byte_ratio	Rapporto di byte
pkts_per_sec	Tasso di pacchetti per secondo

Nel processo di mappatura emergono tre aspetti architetturali critici:

- **Conversioni di Scala e Armonizzazione Temporale:** Per allineare le metriche, la durata dei flussi nel dominio sorgente è stata riscalata da secondi a microsecondi (fattore moltiplicativo 10^6). Parallelamente, il throughput nominale (sload, dload), originariamente quantificato in bit/s in UNSW-NB15, è stato convertito in Byte/s (dividendolo per 8) per coincidere con l'unità di misura nativa dei domini target.
- **Derivazione Analitica di Metriche Mancanti:** Nel dominio $\mathcal{D}_T$, il conteggio totale dei pacchetti di destinazione (dst_pkts) non è fornito in forma diretta. Per garantire la ricostruzione fedele dello spazio latente, tale grandezza è stata derivata indirettamente tramite il prodotto analitico tra il tasso di arrivo pacchetti di ritorno (bw_pkt_s) e la durata del flusso (fl_dur), normalizzato per i microsecondi (10^6).
- **Stabilizzazione Numerica:** Durante il calcolo di indici razionali (come down_up_ratio, byte_ratio, le densità di pacchetti e il *packet rate*), è stata introdotta una costante di regolarizzazione $\epsilon = 10^{-6}$ ai denominatori. Questo accorgimento previene singolarità algebriche causate da divisioni per zero indotte da flussi anomali o malformati.

A completamento della trattazione teorica, i Listati 4.3 e 4.4 riportano l'implementazione in linguaggio Python delle funzioni `_align_nb15` e `_align_stin`. Tali funzioni costituiscono la trasposizione operativa diretta degli operatori ψ_S e ψ_T , sfruttando l'esecuzione vettorializzata su strutture dati *DataFrame* della libreria pandas per massimizzare le prestazioni computazionali su dataset ad alta dimensionalità.

Codice 4.3: Trasposizione operativa dell'operatore di mappatura ψ_k tramite vettorializzazione pandas per il dominio UNSW-NB15.

```
1 def _align_nb15(data):
2     required_columns = ['dur', 'sbytes', 'dbytes', 'spkts', 'dpkts', 'swin',
3         'dwin', 'sload', 'dload', 'sinpkt', 'dinpkt', 'attack_cat', 'label']
4     # [Controllo di sicurezza per le required_columns...]
5
6     new_df = pd.DataFrame()
7
8     # Trasposizione operativa dell'operatore di mappatura
9     new_df['duration'] = data['dur'] * 1000000
10    new_df['src_bytes'] = data['sbytes']
11    new_df['dst_bytes'] = data['dbytes']
12    new_df['src_pkts'] = data['spkts']
13    new_df['dst_pkts'] = data['dpkts']
14    new_df['src_win_byt'] = data['swin']
15    new_df['dst_win_byt'] = data['dwin']
16    new_df['load_s'] = ((data['sload'] + data['dload']) / 8)
17    new_df['down_up_ratio'] = data['dpkts'] / (data['spkts'] + 1e-6)
18    new_df['total_bytes'] = new_df['src_bytes'] + new_df['dst_bytes']
19    new_df['total_pkts'] = new_df['src_pkts'] + new_df['dst_pkts']
20    new_df['src_mean_pkt_size'] = \
21        new_df['src_bytes'] / (new_df['src_pkts'] + 1e-6)
22    new_df['dst_mean_pkt_size'] = \
23        new_df['dst_bytes'] / (new_df['dst_pkts'] + 1e-6)
24    new_df['pkts_per_sec'] = \
25        (new_df['src_pkts'] + new_df['dst_pkts']) / (data['dur'] + 1e-6)
26    new_df['win_diff'] = new_df['src_win_byt'] - new_df['dst_win_byt']
27    new_df['byte_ratio'] = new_df['dst_bytes'] / (new_df['src_bytes'] + 1e-6)
28    new_df['class'] = data['attack_cat']
29    new_df['label'] = data['label']
30
31    return new_df
```

Codice 4.4: Trasposizione operativa dell'operatore di mappatura ψ_k tramite vettorializzazione pandas per il dominio STIN.

```
1 def _align_stin(data):
2     required_columns = ['fl_dur', 'l_fw_pkt', 'l_bw_pkt', 'fw_pk',
3                         'bw_pkt_s', 'fw_win_byt', 'bw_win_byt', 'fl_byt_s',
4                         'fl_iat_min', 'down_up_ratio', 'label']
5     # [Controllo di sicurezza per le required_columns...]
6
7     new_df = pd.DataFrame()
8
9     # Trasposizione operativa dell'operatore di mappatura
10    new_df['duration'] = data['fl_dur']
11    new_df['src_bytes'] = data['l_fw_pkt']
12    new_df['dst_bytes'] = data['l_bw_pkt']
13    new_df['src_pkts'] = data['fw_pk']
14    new_df['dst_pkts'] = data['bw_pkt_s'] * data['fl_dur'] / 1000000
15    new_df['src_win_byt'] = data['fw_win_byt']
16    new_df['dst_win_byt'] = data['bw_win_byt']
17    new_df['load_s'] = data['fl_byt_s']
18    new_df['down_up_ratio'] = data['down_up_ratio']
19    new_df['total_bytes'] = new_df['src_bytes'] + new_df['dst_bytes']
20    new_df['total_pkts'] = new_df['src_pkts'] + new_df['dst_pkts']
21    new_df['src_mean_pkt_size'] = \
22        new_df['src_bytes'] / (new_df['src_pkts'] + 1e-6)
23    new_df['dst_mean_pkt_size'] = \
24        new_df['dst_bytes'] / (new_df['dst_pkts'] + 1e-6)
25    new_df['pkts_per_sec'] = ((new_df['src_pkts'] + new_df['dst_pkts']) \
26        / (new_df['duration'] + 1e-6)) * 1000000
27    new_df['win_diff'] = new_df['src_win_byt'] - new_df['dst_win_byt']
28    new_df['byte_ratio'] = new_df['dst_bytes'] / (new_df['src_bytes'] + 1e-6)
29    new_df['class'] = data['label']
30    new_df['label'] = 1
31
32    return new_df
```


4.3.3 Verifica dell'Omissione della Scalatura Esplicita

Un elemento caratterizzante dell'architettura della pipeline di preprocessing risiede nella scelta metodologica di omettere qualsiasi operazione di normalizzazione o re-scalatura globale dei dati (quali la standardizzazione z -score o il ridimensionamento Min-Max) dalle fasi di addestramento e inferenza valutativa cross-dominio.

Tale impostazione si giustifica formalmente sulla base di tre considerazioni teorico-applicative:

1. **Invarianza Monotonica dei Modelli Adottati:** Gli algoritmi di classificazione integrati nella pipeline principale comprendono *Decision Tree*, *Random Forest* e *Histogram-based Gradient Boosting*. Tali modelli determinano i punti di separazione negli iperpiani di decisione (*split points*) valutando esclusivamente le relazioni d'ordine relativo tra i valori ($x_j \leq \theta$). Essendo tali decisioni invarianti rispetto a qualsiasi trasformazione strettamente monotona, la presenza o l'assenza di scalatura lineare non altera la struttura degli alberi generati né la superficie di separazione appresa.
2. **Preservazione della Semantica Fisica e Trasparenza:** Il mantenimento degli attributi nelle rispettive unità di misura fisiche originarie e derivate (microsecondi per le durate, byte per i volumi e valori adimensionali per i rapporti) preserva la leggibilità diretta delle feature. Ciò risulta determinante per l'interpretabilità dei vettori d'attacco e per l'estrazione di spiegazioni mediante attributori di importanza (*SHAP* e *Permutation Feature Importance*).
3. **Prevenzione del Data Leakage e Invarianza nell'Inferenza:** L'eliminazione dei trasformatori di scala nel flusso di dati principale evita la necessità di stimare e congelare parametri statistici (media e deviazione standard) sul dominio sorgente $\mathcal{D}_S$ da proiettare sui domini target $\mathcal{D}_T$. In contesti di *domain shift*, la proiezione di parametri di scala estratti da sorgenti eterogenee rischia di introdurre distorsioni o anomalie numeriche dovute a valori fuori scala (*outliers*) non previsti.

Si evidenzia che l'utilizzo di algoritmi di normalizzazione è stato rigorosamente limitato alle sole procedure di diagnostica ed esplorazione delle distribuzioni, analizzate sul piano teorico nel Capitolo 3. In particolare, la classe `StandardScaler` del modulo `sklearn.preprocessing` è stata impiegata unicamente all'interno del modulo di supporto `plotting.py` per standardizzare le feature

a media nulla e varianza unitaria prima dell'esecuzione dell'Analisi delle Componenti Principali (*PCA*) e della stima della densità di kernel (*KDE*). Tali trasformazioni diagnostiche sono state isolate all'interno delle routine di visualizzazione e non sono mai state integrate nel grafo computazionale di addestramento e inferenza dei classificatori.

4.4 Implementazione dei Classificatori

La presente sezione traduce in architettura software e in moduli operativi l'impianto di classificazione supervisionata introdotto nella Sezione 3.2.3. A valle del processo di armonizzazione e allineamento dello spazio delle feature descritto nel capitolo precedente, l'infrastruttura software è progettata per istanziare, addestrare e tracciare in modo sistematico le diverse famiglie di modelli predittivi. L'obiettivo primario di questo modulo consiste nel definire una pipeline di fitting rigorosa, capace di gestire sia i classificatori basati su dataset aggregati (per la valutazione complessiva del *domain shift*) sia la suite di modelli biclasse (per l'isolamento e l'analisi granulare di minacce mirate).

Nelle sottosezioni seguenti vengono dettagliate le librerie di machine learning adottate, la configurazione degli iperparametri e la struttura dei registri di tracciamento persistenti (Sottosezione 4.4.1). Infine, le Sottosezioni 4.4.2 e 4.4.3 formalizzano le logiche procedurali e i criteri di campionamento stratificato impiegati per la generazione dei rispettivi insiemi di addestramento.

4.4.1 Algoritmi di Base e Iperparametri

L'infrastruttura di classificazione del framework è stata sviluppata in ambiente Python basandosi sulla libreria open-source `scikit-learn` (versione 1.9.0). Coerentemente con le premesse metodologiche discusse nella Sezione 3.2.3, sono state selezionate tre famiglie di algoritmi di apprendimento supervisionato basati su alberi di decisione, caratterizzate da differenti livelli di complessità strutturale e capacità espressiva:

- **Decision Tree (DT):** Classificatore a singolo albero di decisione, implementato tramite la classe `DecisionTreeClassifier`¹⁵.

¹⁵Link alla documentazione del classificatore DT

- **Random Forest (RF):** Architettura d'insieme di tipo *bagging*, implementata mediante la classe `RandomForestClassifier`¹⁶.
- **Histogram-based Gradient Boosting (HGB):** Algoritmo d'insieme di tipo *boosting* ottimizzato per dataset di grandi dimensioni, implementato tramite la classe `HistGradientBoostingClassifier`¹⁷.

Al fine di valutare la capacità intrinseca di generalizzazione cross-dominio delle diverse famiglie di modelli ed evitare fenomeni di adattamento spurio (*overfitting*) delle configurazioni alle specificità statistiche del solo dominio sorgente $\mathcal{D}_S$, non sono state applicate procedure di ricerca ed ottimizzazione automatica degli iperparametri (quali *GridSearchCV* o *RandomizedSearchCV*). Tutti i modelli sono stati pertanto istanziati mantenendo i valori di iperparametri predefiniti (*default*) forniti dall'API di `scikit-learn`.

L'unico parametro esplicitamente vincolato in fase di inizializzazione è il seme di determinismo numerico (`random_state`), il quale viene impostato in modo dinamico iterando su un pool di seed predefiniti (es. `seed = 42`) per garantire la perfetta riproducibilità statistica degli esperimenti. La Tabella 4.11 sintetizza la corrispondenza tra le classi Python adottate e la configurazione degli iperparametri operativi associati.

¹⁶Link alla documentazione del classificatore RF

¹⁷Link alla documentazione del classificatore HGB

Tabella 4.11: Configurazione degli iperparametri di default e classi `scikit-learn` utilizzate per ciascuna famiglia di classificatori.

Algoritmo	Classe <code>scikit-learn</code>	Iperparametri di Default
Decision Tree (DT)	<code>DecisionTreeClassifier</code>	<code>criterion='gini',</code> <code>max_depth=None,</code> <code>min_samples_split=2</code>
Random Forest (RF)	<code>RandomForestClassifier</code>	<code>n_estimators=100,</code> <code>criterion='gini',</code> <code>max_depth=None,</code> <code>max_features='sqrt',</code> <code>min_samples_split=2</code>
Hist. Gradient Boosting	<code>HistGradientBoostingClassifier</code>	<code>learning_rate=0.1,</code> <code>max_iter=100,</code> <code>max_depth=None,</code> <code>max_leaf_nodes=31,</code> <code>l2_regularization=0.0</code>

Per garantire un tracciamento rigoroso del ciclo di vita dei modelli, l'istanziamento e l'addestramento non disperdono lo stato interno degli stimatori. Al termine del fitting di ciascun classificatore su uno specifico dataset di addestramento (sorgente o ibrido) e per ogni seed di esecuzione, le informazioni di configurazione hardware/temporale, l'assetto completo degli iperparametri e le metriche di prestazione in-sample (*training set*) vengono persistite in modo strutturato all'interno del file di registro `models_metadata.csv`.

Tale meccanismo di tracciamento memorizza per ogni modello un record identificativo univoco, la cui struttura logica e i relativi attributi sono riepilogati nella Tabella 4.12.

Tabella 4.12: Struttura e campi del record di tracciamento persistito nel registro `models_metadata.csv`.

Categoria Logica	Campi Registrati
Metadati di Identificazione	<code>id</code> , <code>timestamp</code> , <code>model_name</code> , <code>dataset_type</code> , <code>random_state</code>
Proprietà Struttura Dati	<code>samples</code> , <code>train_split</code> , <code>n_features_in</code> , <code>n_classes</code>
Config. Iperparametrica	<code>n_estimators</code> , <code>max_depth</code> , <code>max_features</code> , <code>criterion</code> , <code>min_samples_split</code> , <code>learning_rate</code> , <code>max_iter</code>
Valutazione In-Sample	Matrice di confusione grezza (TP, TN, FP, FN) e metriche derivate di addestramento (Accuracy, Precision, Recall, F1-Score, ROC-AUC, PR-AUC, TPR, FPR)

L'archiviazione sistematica in `models_metadata.csv` assicura la completa uditabilità dell'infrastruttura di addestramento, fornendo la base informativa per le successive fasi di inferenza e valutazione cross-dominio sui dataset di test target.

4.4.2 Istanziamento dei Modelli Aggregati

La strategia di istanziamento dei classificatori aggregati prevede la generazione di tre distinte classi di modelli per ciascun seed stocastico, corrispondenti alla formalizzazione teorica discussa nella Sezione 3.2.1. Nonostante i dataset integrino l'etichettatura multi-classe (identificando la specifica tipologia di attacco), le architetture aggregate sono addestrate a risolvere un task di rilevamento binario, discriminando il traffico benevolo (Normal) dal generico traffico ostile (Attack).

La composizione dei *training set* per l'addestramento segue precise regole di proporzionalità e campionamento, ancorate al parametro di *Injection Ratio* $\rho_{ben/mal} = 10/1$:

1. **Modello Base ($\mathcal{M}_{base}$):** Costituisce la *baseline* teorica. È addestrato su un dataset ingegnerizzato ad hoc tramite un approccio di *injection* controllata. Mantenendo il rapporto 10/1, i campioni benigni sono estratti integralmente dal dataset UNSW-NB15, mentre la componente malevola è ripartita proporzionalmente: $\frac{299}{300}$ dei campioni anomali richiesti proviene da UNSW-NB15, mentre la frazione residua $\frac{1}{300}$ è popolata con attacchi estratti dal dataset

satellitare integrato (STIN, composto da TER20 e SAT20). Nei registri di esecuzione, tale modello è identificato come `injection_aggregate`.

2. **Modello Sorgente ($\mathcal{M}_S$):** È il modello aggregato nativo, istanziato esclusivamente sui campioni appartenenti al dominio sorgente terrestre ($\mathcal{D}_S$). Si compone di traffico benigno e attacchi estratti unicamente dal dataset UNSW-NB15, denominato `nb15_aggregate`.
3. **Modelli Ibridi ($\mathcal{M}_H^{(T)}$):** Rappresentano la concretizzazione della procedura di *fine-tuning* in ottica cross-dominio. Vengono istanziati combinando tutti i record benigni nativi del dataset sorgente (UNSW-NB15) con la totalità dei campioni malevoli estratti dai domini target armonizzati ($\Delta\mathcal{D}_T^{(\text{train})}$). Anche per l'addestramento di queste architetture viene garantito e forzato il rigoroso rispetto del rapporto di sbilanciamento $\rho_{ben/mal} = 10/1$. Sono state istanziate tre varianti di modelli ibridi aggregati:

- `nb15_stin_aggregate`: Ibrido su dataset STIN complessivo.
- `nb15_ter20_aggregate`: Ibrido sul solo canale target TER20.
- `nb15_sat20_aggregate`: Ibrido sul solo canale target SAT20.

La Figura 4.4 schematizza visivamente il flusso di composizione dei dati appena descritto, evidenziando il rigoroso mantenimento del vincolo proporzionale tra le classi in fase di addestramento.

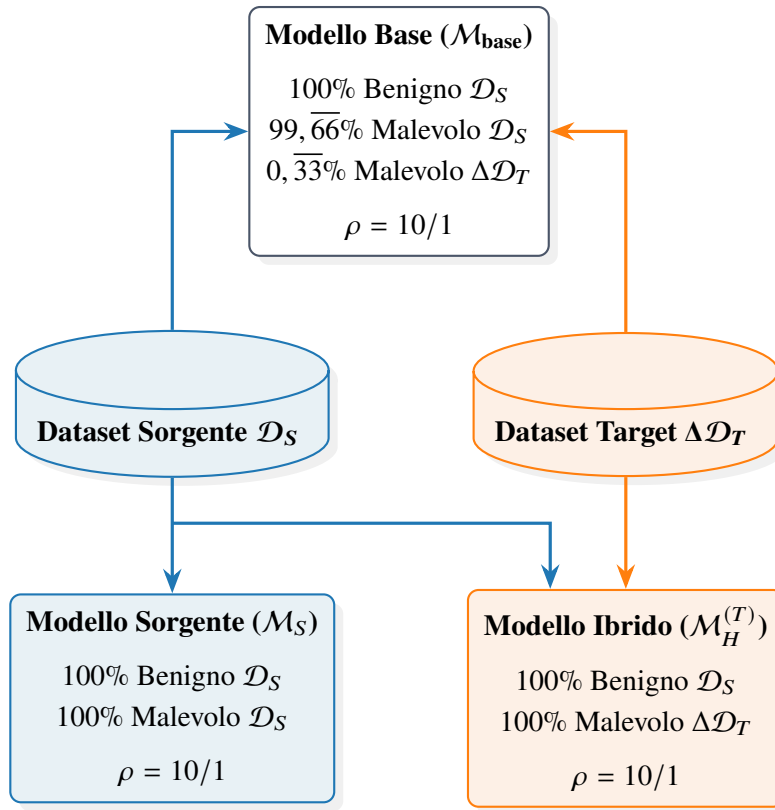


Figura 4.4: Architettura del flusso dati per l’addestramento dei Modelli Aggregati. Le percentuali indicano l’origine logica dei pacchetti che compongono i *training set*, forzati strutturalmente al vincolo $\rho_{\text{ben/mal}} = 10/1$.

Il prelievo dei campioni dalla partizione target ($\Delta\mathcal{D}_T^{(\text{train})}$) è governato da una procedura di campionamento casuale stratificato (*stratified random sampling*)¹⁸. Questa metodologia garantisce che i sotto-insiemi estratti rispettino le proporzioni imposte dal ratio 10/1 e salvaguarda la rigida compartimentazione logica tra *train set* e *test set*, risultando fondamentale per scongiurare qualsiasi fenomeno di dispersione dell’informazione (*Data Leakage*) in fase di validazione.

Dal punto di vista dell’ingegneria del software, la pipeline di istanziazione e addestramento è centralizzata all’interno del modulo `models.py`. La logica architetturale espone un punto di accesso pubblico, la funzione `model_processing()`, la quale orchestra dinamicamente l’esecuzione. Tale funzione delega il calcolo matematico del fitting a un *helper* interno denominato `_train_classifier()`, responsabile dell’inizializzazione dell’algoritmo di ML richiesto e della

¹⁸Implementato avvalendosi della funzione `train_test_split` di `scikit-learn`, valorizzando il parametro `stratify` per garantire la corretta preservazione delle proporzioni relative alle classi originali durante l’estrazione.

valutazione asincrona delle metriche sul *test set* in-sample. Lo script gestisce inoltre il salvataggio dei pesi strutturali (formato `.joblib`¹⁹), garantendo l'aggiornamento automatico del registro `models_metadata.csv` tramite l'assegnazione procedurale del *Naming Standard* in base al dominio analizzato.

4.4.3 Istanziamento dei Modelli Biclasse

Al fine di condurre un'analisi comparativa granulare e valutare le prestazioni analitiche dei classificatori rispetto a minacce mirate, la metodologia prevede l'addestramento di una suite parallela di modelli biclasse, come anticipato nella Sezione 3.2.2. A differenza delle varianti aggregate (discusse nella Sottosezione 4.4.2), ciascun classificatore biclasse è progettato per isolare e discriminare un'unica e specifica famiglia di attacco dal normale traffico di rete (Normal). Tale approccio consente di misurare le metriche di accuratezza del modello dinanzi a pattern di traffico ostile altamente caratterizzati.

Dal punto di vista dell'ingegnerizzazione dei dati, i sotto-insiemi impiegati per il *fitting* di questi modelli vengono strutturati *a priori* durante la macro-fase di pre-elaborazione del file. In questa sede, i record appartenenti a vettori di attacco estranei al contesto di analisi vengono filtrati e scartati, restituendo un dominio di riferimento strettamente binario (es. [Normal, DDoS]). Durante la generazione di questi dataset isolati, l'algoritmo di elaborazione impone rigorosamente il mantenimento del rapporto di sbilanciamento $\rho_{ben/mal} = 10/1$. Contestualmente, viene preservata e fissata la partizione logica dei dati tramite etichettatura statica dei set di *train* e *test*, scongiurando eventuali contaminazioni informative (*Data Leakage*).

In perfetta simmetria con la suddivisione concettuale tra dominio sorgente e domini target ibridizzati, la pipeline ha generato le istanze biclasse riepilogate nella Tabella 4.13.

¹⁹Link alla documentazione ufficiale della libreria `joblib`. Tale formato rappresenta lo standard *de facto* nell'ecosistema Python per la serializzazione efficiente di oggetti che ospitano array NumPy di grandi dimensioni, rendendolo particolarmente ottimizzato per il salvataggio dei modelli basati su alberi di decisione.

Tabella 4.13: Istanze dei modelli biclasse generati per il dominio sorgente e i domini target ibridizzati.

Categoria	Dominio	Istanze Modello Biclasse
Modelli Sorgente	UNSW-NB15	nb15_dos nb15_exploits nb15_fuzzers nb15_generic nb15_reconnaissance
Modelli Ibridi Target	SAT20	nb15_sat20_syn_ddos nb15_sat20_udp_ddos
Modelli Ibridi Target	TER20	nb15_ter20_botnet nb15_ter20_ddos nb15_ter20_syn_ddos nb15_ter20_udp_ddos

L'integrazione di questa logica all'interno del codice sorgente sfrutta elevati livelli di automazione. L'assegnazione della nomenclatura ai file serializzati e ai registri di metadati è gestita in maniera trasparente dalla subroutine `_get_standardized_model_name()` del modulo `models.py`. La funzione elabora il vettore delle classi uniche del dataset in input: verificata l'esatta cardinalità binaria, estrae computazionalmente l'etichetta divergente dalla stringa `Normal` e la concatena in modo canonico alla tipologia del dataset (generando dinamicamente stringhe come `dataset_attackname`). Tale soluzione ingegneristica ha garantito la standardizzazione dei log per tutti i successivi processi di inferenza. Il Listato 4.5 riporta il frammento di codice responsabile di tale elaborazione logica.

Codice 4.5: Estrazione procedurale dell’etichetta di attacco per la generazione dinamica della nomenclatura del modello.

```
1 def _get_standardized_model_name(dataset_type, unique_classes):
2     """
3     Generates a standardized model name
4     based on the unique classes present in the dataset.
5     Supports RF, DT, and HGB models dynamically.
6
7     :param dataset_type: string describing the dataset type
8     :param unique_classes: list or array of unique class labels
9     :return: standardized model name as a string
10    """
11    # Strip whitespace and convert to lowercase for consistency
12    classes = [str(c).strip() for c in unique_classes if str(c).strip()]
13
14    # If there are more than two classes, return a generic name
15    if len(classes) > 2:
16        return f"{dataset_type}_aggregate"
17
18    # Identify the attack class (assuming 'normal' is the benign class)
19    attack_classes = [c for c in classes if c.lower() != 'normal']
20    attack_name = attack_classes[0].lower() if attack_classes else "normal"
21
22    return f"{dataset_type}_{attack_name}"
```

4.5 Protocollo di Addestramento e Valutazione

La presente sezione formalizza e declina in moduli operativi il protocollo sperimentale di addestramento, valutazione ed analisi inferenziale introdotto dal punto di vista teorico nel Capitolo 3. A fronte dell’esigenza di garantire la stabilità stocastica dei modelli e la piena riproducibilità dei risultati, l’infrastruttura software è progettata per gestire in modo sistematico l’intero ciclo di vita degli stimatori: dalla fase di fitting strutturata su molteplici semi stocastici, all’inferenza rigorosa sugli insiemi di collaudo, fino alla validazione statistica delle prestazioni relative. L’obiettivo

primario di questo modulo consiste nel definire una pipeline esecutiva automatizzata e rigorosa, capace di integrare la persistenza degli artefatti, la generazione delle matrici di valutazione e la verifica formale delle ipotesi di significatività.

Nelle sottosezioni seguenti vengono dettagliate le componenti e le logiche procedurali che costituiscono l'architettura del protocollo. In primo luogo, la Sottosezione 4.5.1 descrive l'orchestrazione gerarchica del ciclo di addestramento multi-seed, le modalità di serializzazione degli oggetti predittivi e i meccanismi di tracciamento dei metadati nei registri persistenti. Successivamente, la Sottosezione 4.5.2 illustra il flusso di classificazione e inferenza sugli insiemi di test, la gestione difensiva dei casi limite, la derivazione delle metriche di prestazione e la costruzione delle matrici di valutazione per singolo seed ed aggregate. Infine, la Sottosezione 4.5.3 analizza l'implementazione del test t di Welch, dettagliando la procedura di aggregazione dei dati di *F1-Score* e i criteri di decisione statistica adottati per il confronto tra le diverse configurazioni e il modello di riferimento.

4.5.1 Orchestrazione del Ciclo di Addestramento Multi-Seed

L'esecuzione della campagna di addestramento è stata strutturata secondo un'organizzazione gerarchica basata sulla tipologia di modello (`model_type`) e sul seme stocastico (`seed`). I dataset impiegati nel processo sono univoci e vengono istanziati una sola volta durante la fase di preprocessing, utilizzando il seme deterministico primario $s_0 = 127001$, come descritto nella Sezione 4.1.3. Le successive esecuzioni multi-seed operano pertanto sugli stessi dataset pre-processati, senza introdurre nuove istanziazioni o modifiche alla composizione dei benchmark.

L'orchestrazione globale è demandata al punto di ingresso `main.py`, nel quale il ciclo di costruzione dei modelli viene realizzato iterando sul prodotto cartesiano tra l'insieme delle tipologie di classificatore e l'insieme dei seed definiti dal framework. In termini procedurali, per ciascun `model_type` vengono quindi eseguite in successione le dieci configurazioni associate ai semi dell'Equazione (4.1).

Il flusso operativo dell'infrastruttura di addestramento e la relativa logica di persistenza sono schematizzati nella Figura 4.5.

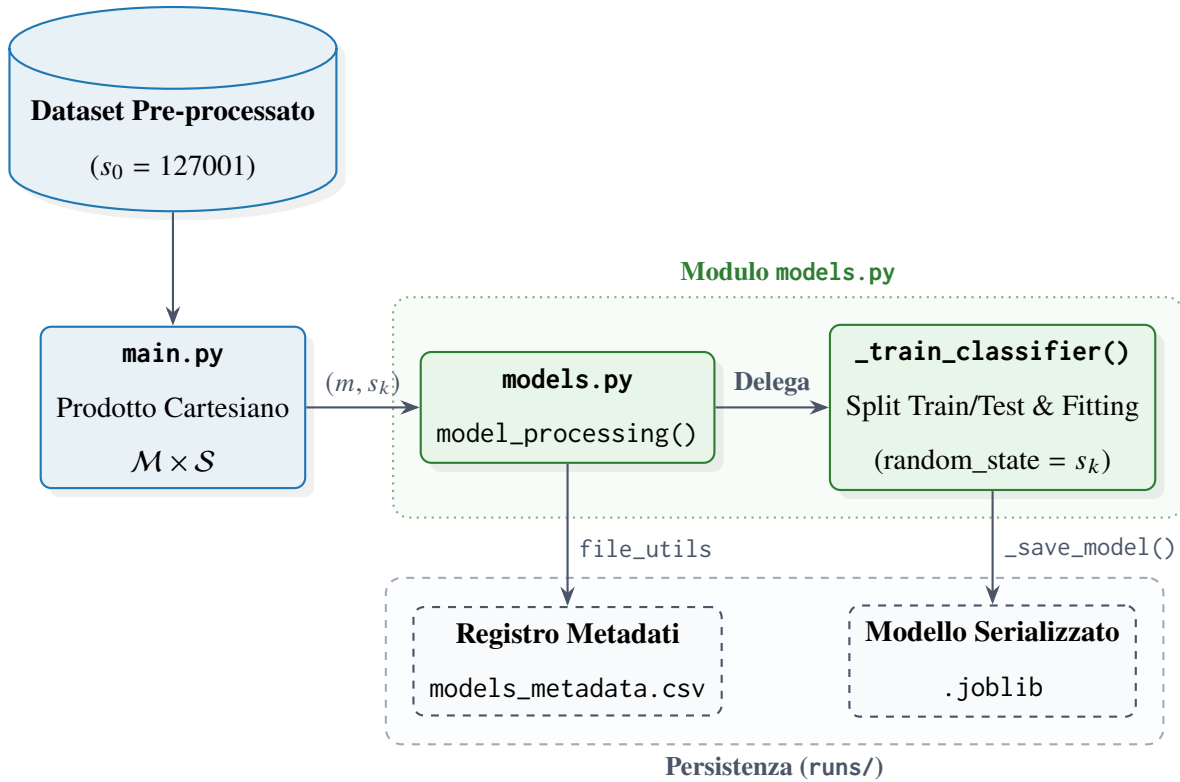


Figura 4.5: Flusso operativo dell’orchestrazione multi-seed e isolamento degli artefatti persistiti.

Tale organizzazione garantisce che ogni combinazione (m, s_k) , con m appartenente all’insieme delle tipologie di modello e $s_k \in \mathcal{S}$, costituisca un’esecuzione logicamente indipendente. Il parametro `random_state` viene trasferito esplicitamente all’istanziamento del classificatore, preservando il controllo della componente stocastica senza alterare i dataset di ingresso. La distinzione tra le diverse configurazioni di algoritmo e seed è quindi mantenuta a livello di file system e di stato interno degli stimatori, in coerenza con il principio di isolamento già adottato nella gestione della riproducibilità.

Dal punto di vista implementativo, la logica di addestramento è centralizzata nel modulo `models.py`. La funzione pubblica `model_processing()` costituisce il punto di ingresso della procedura e riceve il *DataFrame* del benchmark, il relativo `dataset_type`, la tipologia di classificatore e il seed corrente. Tale funzione provvede alla predisposizione della struttura di persistenza associata all’esecuzione, delegando il fitting vero e proprio alla funzione interna `_train_classifier()`. Quest’ultima separa il dataset nelle partizioni train e test, rimuove le colonne non destinate all’apprendimento e istanzia dinamicamente il classificatore richiesto. La centralizzazione della

pipeline consente di applicare la medesima procedura operativa alle diverse famiglie di modelli ultimate nel framework.

L'istanziamento dell'algoritmo mantiene gli iperparametri ai valori predefiniti forniti da `scikit-learn`, mentre il parametro `random_state` viene vincolato al seed dell'esecuzione. Per la *Random Forest*, inoltre, il parametro `n_jobs=-1` abilita l'impiego parallelo dei processori disponibili durante la costruzione dell'ensemble; per il *Decision Tree* e l'*Histogram-based Gradient Boosting* non viene invece specificata una configurazione equivalente nel codice applicativo. La scelta mantiene separata la parallelizzazione interna dell'algoritmo dalla logica di orchestrazione, che continua a processare sequenzialmente le diverse combinazioni di `model_type` e `seed`.

Una volta completato il fitting, lo stato dello stimatore viene congelato mediante serializzazione su disco. La funzione `_save_model()` utilizza la libreria `joblib` per esportare l'oggetto addestrato in formato binario `.joblib`, registrandone contestualmente il percorso nel registro dei modelli. La struttura gerarchica adottata separa la tipologia di modello dal seed e dagli artefatti prodotti, secondo lo schema generale:

```
runs/model_type/seed/models/
```

Un'istanza concreta è rappresentata dal percorso:

```
runs/hybrid_models/rf/42/models/injection_aggregate.joblib
```

nel quale `rf` identifica la famiglia del classificatore, `42` il seed dell'esecuzione e `injection_aggregate.joblib` il modello serializzato associato al benchmark considerato. La nomenclatura dei modelli viene determinata automaticamente sulla base del `dataset_type` e delle classi presenti nel dataset, così da mantenere una corrispondenza univoca tra configurazione sperimentale e artefatto persistito.

Parallelamente alla serializzazione dello stimatore, per ogni configurazione vengono registrati i relativi metadati nel file `models_metadata.csv`. Il meccanismo di aggiornamento è implementato mediante la funzione `update_or_append_csv()` del modulo `file_utils.py`, la quale verifica preventivamente l'eventuale presenza di un record corrispondente alle chiavi di identificazione e, in assenza di una corrispondenza, genera un nuovo identificatore incrementale. La funzione mantiene inoltre il campo `id` come prima colonna del file e allinea lo schema del nuovo record a quello già presente. Per ciascun modello vengono quindi mantenute, oltre alle informazioni identificative, le

principali proprietà strutturali e configurazionali dello stimatore, inclusi il valore di `random_state`, il numero di feature in ingresso, il numero di classi e gli iperparametri specifici dell'algoritmo. Il registro costituisce pertanto la traccia persistente dell'esecuzione e permette di associare ogni artefatto serializzato alla precisa configurazione che ne ha determinato la costruzione.

La procedura adottata garantisce infine una netta separazione tra produzione e successivo utilizzo degli stimatori. Una volta completato il ciclo di fitting e serializzazione per un determinato seed, il relativo modello non viene più modificato nelle fasi successive: la valutazione viene infatti eseguita caricando da disco l'artefatto precedentemente congelato. In questo modo, la fase di classificazione successiva opera esclusivamente in modalità di inferenza sul modello già addestrato, evitando che le procedure di valutazione possano alterarne lo stato interno.

4.5.2 Calcolo delle Metriche e Costruzione della Matrice di Valutazione

La valutazione dei classificatori è stata implementata mediante una procedura comune a tutte le configurazioni sperimentali, così da garantire uniformità nel calcolo delle metriche e nella successiva organizzazione dei risultati. La fase di classificazione opera esclusivamente sugli insiemi di collaudo, identificati tramite il campo `split_type`, mentre le informazioni relative alle feature e alle etichette vengono separate prima dell'esecuzione dell'inferenza. Per ciascun modello precedentemente serializzato, l'artefatto `.joblib` viene ricaricato da disco e utilizzato senza apportare modifiche al suo stato interno.

La procedura è centralizzata nella funzione `classification_processing()` del modulo `classification.py`, che delega il calcolo delle predizioni alla propria funzione interna `_classification()`. Una volta isolato il sottoinsieme associato a `split_type = test`, vengono eliminate dalle feature le colonne `label`, `class` e `split_type`, mentre la variabile `label` viene mantenuta separatamente come vettore delle classi reali. L'inferenza viene quindi effettuata mediante i metodi `predict()` e `predict_proba()`, utilizzando quest'ultimo per estrarre il punteggio associato alla classe positiva. Non viene applicata una soglia decisionale personalizzata: le etichette predette sono pertanto quelle direttamente restituite dall'estimatore secondo il comportamento predefinito del classificatore.

Il calcolo delle metriche è stato incapsulato nella funzione `calculate_metrics()` del modulo `src/utils/metrics.py`, con l'impiego delle funzioni messe a disposizione da `sklearn.metrics`.

In particolare, vengono utilizzate le metriche descritte in Tabella 4.14.

Tabella 4.14: Metriche di valutazione delle prestazioni utilizzate per la caratterizzazione dei modelli di apprendimento automatico.

Funzione Scikit-Learn	Descrizione Operativa
<code>confusion_matrix</code>	Matrice di confusione per il conteggio delle predizioni binarie.
<code>accuracy_score</code>	Proporzione complessiva di predizioni corrette rispetto al totale delle istanze.
<code>precision_score</code>	Frazione di istanze malevole identificate correttamente rispetto a tutte quelle classificate come tali.
<code>recall_score</code>	Sensibilità o tasso di veri positivi (<i>TPR</i>): frazione di anomalie rilevate rispetto al totale reale.
<code>f1_score</code>	Media armonica tra Precision e Recall, indicatore sintetico in presenza di sbilanciamento delle classi.
<code>roc_auc_score</code>	Area sotto la curva ROC (True Positive Rate vs False Positive Rate) al variare della soglia decisionale.
<code>average_precision_score</code>	Area sotto la curva Precision-Recall, pesata sull'incremento della Recall ad ogni soglia.

La matrice di confusione viene esplicitamente calcolata imponendo l'ordine delle classi $[0, 1]$ e successivamente scomposta nei quattro termini fondamentali:

$$\mathbf{C} = \begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}. \quad (4.3)$$

A partire da tali quantità vengono inoltre calcolati direttamente il *True Positive Rate* (TPR), il *False Negative Rate* (FNR), il *True Negative Rate* (TNR) e il *False Positive Rate* (FPR), secondo le relazioni riportate nella Sezione 3.4.3.

Il *Recall* restituito da `recall_score()` coincide, nel caso binario adottato dal framework, con il TPR. Le ulteriori metriche calcolate comprendono *Accuracy*, *Precision*, *F1-Score*, *ROC-AUC* e *PR-AUC*. Le metriche *Accuracy*, *Precision*, *Recall* e *F1-Score* sono ottenute a partire dalle etichette predette `y_pred`, mentre *ROC-AUC* e *PR-AUC* vengono determinate utilizzando i

punteggi `y_scores` associati alla classe positiva. Tutti i valori numerici prodotti dalle funzioni di metrica vengono arrotondati al numero di cifre decimali stabilito nella configurazione globale `MLConstants.DECIMAL_DIGITS`²⁰.

Particolare attenzione è stata riservata ai casi nei quali il vettore delle etichette reali contiene una sola classe. Prima del calcolo delle metriche viene pertanto verificata la presenza di almeno due classi distinte mediante il numero di valori univoci presenti in `y_test`. Qualora tale condizione non sia soddisfatta, le metriche *Accuracy*, *Precision*, *Recall*, *F1-Score*, *ROC-AUC* e *PR-AUC* vengono impostate a `None`. I quattro elementi della matrice di confusione vengono invece sempre ricavati imponendo le classi `[0, 1]`, mentre *TPR*, *FNR*, *TNR* e *FPR* vengono calcolati separatamente e assumono valore `None` esclusivamente quando il denominatore della relativa espressione risulta nullo²¹. La struttura logica di tale gestione difensiva è sintetizzata nel Listing 4.6.

²⁰Nella configurazione sperimentale corrente, il parametro `DECIMAL_DIGITS` è impostato su 4 decimali per preservare la precisione numerica nei calcoli aggregati.

²¹Questa guardia condizionale esplicita sui denominatori ($(TP + FN) > 0$ e $(FP + TN) > 0$) e sul conteggio delle classi univoche previene eccezioni di divisione per zero e avvisi a runtime (*RuntimeWarning/UndefinedMetricWarning*), garantendo la continuità dell'esecuzione e la tracciabilità dei casi degeneri nei report salvati.

Codice 4.6: Gestione difensiva dei casi limite e calcolo delle metriche in `calculate_metrics()`.

```
1 has_classes = len(np.unique(y_test)) > 1
2 # [Controllo di sicurezza sul numero di classi...]
3
4 # Calcolo delle metriche
5 tn, fp, fn, tp = confusion_matrix(y_test, y_pred, labels=[0, 1]).ravel()
6
7 if has_classes:
8     accuracy = round(accuracy_score(y_test, y_pred),
9                       MLConstants.DECIMAL_DIGITS)
10    precision = round(precision_score(y_test, y_pred),
11                      MLConstants.DECIMAL_DIGITS)
12    recall = round(recall_score(y_test, y_pred),
13                  MLConstants.DECIMAL_DIGITS)
14    f1 = round(f1_score(y_test, y_pred),
15              MLConstants.DECIMAL_DIGITS)
16    roc = round(roc_auc_score(y_test, y_scores),
17              MLConstants.DECIMAL_DIGITS)
18    pr = round(average_precision_score(y_test, y_scores),
19             MLConstants.DECIMAL_DIGITS)
20 else:
21     accuracy = precision = recall = f1 = roc = pr = None
22
23 if (tp + fn) > 0:
24     tpr = round(tp / (tp + fn), MLConstants.DECIMAL_DIGITS)
25     fnr = round(fn / (tp + fn), MLConstants.DECIMAL_DIGITS)
26 else:
27     tpr, fnr = None, None
28
29 if (fp + tn) > 0:
30     tnr = round(tn / (fp + tn), MLConstants.DECIMAL_DIGITS)
31     fpr = round(fp / (fp + tn), MLConstants.DECIMAL_DIGITS)
32 else:
33     tnr, fpr = None, None
```

L'insieme completo dei risultati viene quindi organizzato all'interno del file nominato come

`classifications.csv`, generato separatamente per ciascuna combinazione di `model_type` e `seed`. Ogni record contiene i metadati identificativi della classificazione, il tipo di dataset e la relativa composizione delle classi, oltre alle quantità della matrice di confusione e alle metriche derivate. La struttura complessiva del record e la relativa suddivisione funzionale dei campi sono dettagliate nella Tabella 4.15.

Tabella 4.15: Tassonomia e suddivisione concettuale dei campi costituenti il record di tracciamento delle prestazioni.

Campo	Descrizione Operativa
Metadati ed Esecuzione Sperimentale	
id	Identificatore univoco progressivo della singola esecuzione.
timestamp	Marca temporale di completamento della valutazione.
model_name	Architettura o denominazione del modello di apprendimento valutato.
dataset_type	Tipologia di partizione o dominio di test utilizzato.
classes	Cardinalità o identificativi delle classi analizzate.
samples	Numero totale di campioni elaborati nella sessione.
Conteggi della Matrice di Confusione	
TP	Conteggio dei Veri Positivi (<i>True Positives</i>).
TN	Conteggio dei Veri Negativi (<i>True Negatives</i>).
FP	Conteggio dei Falsi Positivi (<i>False Positives</i>).
FN	Conteggio dei Falsi Negativi (<i>False Negatives</i>).
Metriche di Prestazione Sintetiche	
Accuracy	Tasso di accuratezza globale sul totale dei campioni.
Precision	Frazione di predizioni positive risultate corrette.
Recall	Sensibilità o tasso di rilevamento delle anomalie.
F1-Score	Media armonica bilanciata tra Precision e Recall.
ROC-AUC	Area sotto la curva <i>Receiver Operating Characteristic</i> .
PR-AUC	Area sotto la curva <i>Precision-Recall (Average Precision)</i> .
Tassi e Proporzioni Operative	
TPR	<i>True Positive Rate</i> , coincidente con la Recall.
FNR	<i>False Negative Rate</i> , o tasso di mancato allarme.
TNR	<i>True Negative Rate</i> , o specificità del sistema.
FPR	<i>False Positive Rate</i> , o tasso di falso allarme.

La persistenza viene effettuata mediante `update_or_append_csv()`, che individua un'eventuale

voce preesistente sulla base delle chiavi di corrispondenza (`model_name`, `dataset_type` e `classes`) e, in caso contrario, assegna un identificatore al nuovo record.

A partire dai risultati persistiti viene costruita la matrice di valutazione definita teoricamente nel Capitolo 3. Per ciascun seed $s_k \in \mathcal{S}$, tale matrice può essere formalmente rappresentata come:

$$H_k = \left[h_{ij}^{(k)} \right] \in \mathbb{R}^{M \times D}, \quad (4.4)$$

dove M identifica l'insieme dei modelli addestrati, D l'insieme dei dataset di valutazione e ciascun elemento $h_{ij}^{(k)}$ rappresenta il valore della metrica considerata per la coppia costituita dal i -esimo modello e dal j -esimo dataset al seed s_k .

Nell'implementazione concreta, la rappresentazione generale viene articolata in due matrici indipendenti, una relativa al *True Positive Rate* e una relativa al *True Negative Rate*:

$$H_k^{TPR} = \left[TPR_{ij}^{(k)} \right], \quad H_k^{TNR} = \left[TNR_{ij}^{(k)} \right]. \quad (4.5)$$

Le due matrici presentano la medesima organizzazione bidimensionale, con i modelli addestrati disposti sulle righe e i dataset di valutazione sulle colonne. La suddivisione fisica viene invece effettuata a livello di gruppi di modelli: una matrice raccoglie le configurazioni aggregate del dominio sorgente, i modelli biclasse sorgente e il relativo modello *INJECTION*, mentre l'altra raccoglie le configurazioni aggregate ibride, i modelli biclasse ibridi e il medesimo modello di riferimento *INJECTION*.

Per ciascuna matrice, la colonna finale è reserved al valore medio associato alla riga del modello, mentre il modello *INJECTION* viene riportato in una sezione graficamente separata nella parte inferiore della rappresentazione. Gli artefatti relativi alle singole esecuzioni sono salvati nella directory

`runs/<model_group>/<model_type>/<seed>/results/plots/performance/`

attraverso i file `TPR_matrix.png` e `TNR_matrix.png`.

L'intero flusso operativo di valutazione e la successiva aggregazione statistica tra i diversi seed sono schematizzati nella Figura 4.6.

Fase 1: Valutazione e Persistenza per Singolo Seed (s_k)

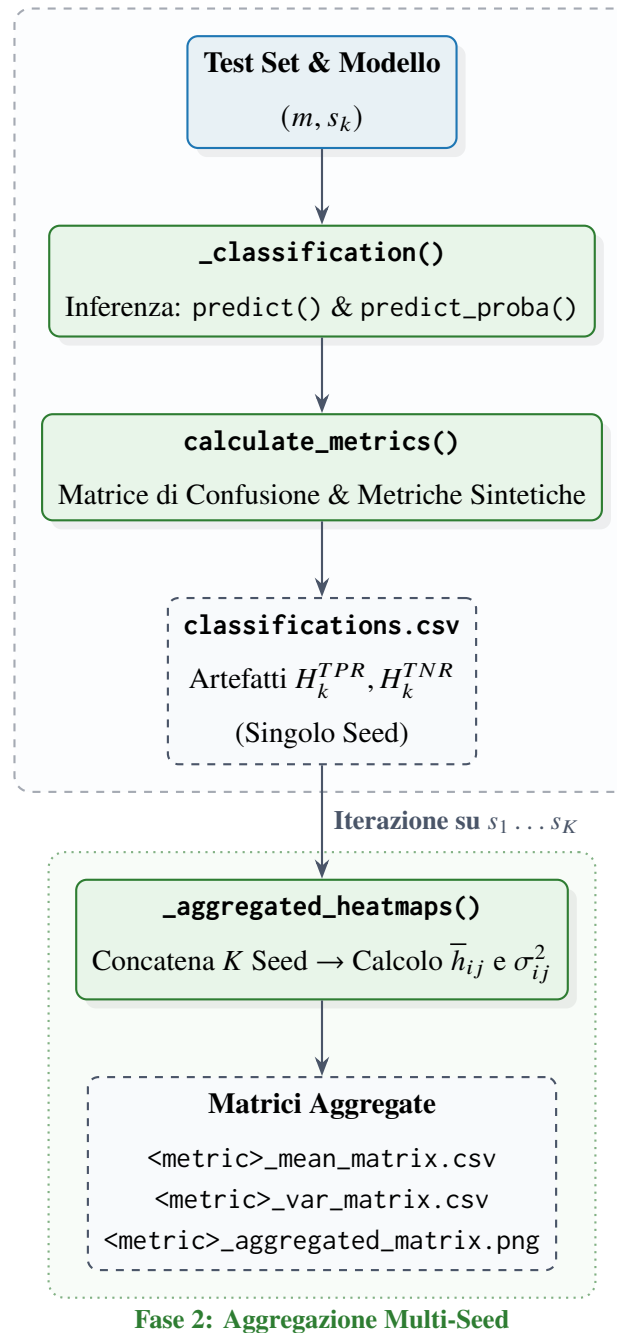


Figura 4.6: Pipeline a due stadi per il calcolo delle metriche di classificazione, tracciamento su file system e aggregazione statistica multi-seed.

Al termine dell'elaborazione dei singoli seed, viene eseguita una seconda fase di aggregazione che considera congiuntamente i risultati prodotti per tutti i valori di s_k . La routine `_aggregated_heatmaps()` legge i file `classifications.csv` presenti nelle directory associa-

te ai diversi seed, li concatena in un unico *DataFrame* e associa esplicitamente a ciascun record il seed di provenienza.

Per ciascuna coppia modello–dataset vengono successivamente determinate le statistiche aggregate della metrica considerata sui diversi seed. In particolare, per TPR e TNR viene prodotta un’unica matrice contenente due valori numerici distinti per ogni coppia modello–dataset: rispettivamente la media

$$\bar{h}_{ij} = \frac{1}{K} \sum_{k=1}^K h_{ij}^{(k)}, \quad (4.6)$$

e la varianza

$$\sigma_{ij}^2 = \frac{1}{K} \sum_{k=1}^K \left(h_{ij}^{(k)} - \bar{h}_{ij} \right)^2. \quad (4.7)$$

La rappresentazione aggregata combina quindi l’informazione relativa al valore medio e alla dispersione osservata tra i seed, senza sostituire i dati relativi alle singole esecuzioni. Per ciascun tipo di classificatore, le matrici aggregate vengono memorizzate sotto la directory

runs/<model_group>/<model_type>/aggregated_heatmaps/

con una sottocartella dedicata alla metrica considerata. Per ciascuna metrica vengono conservati sia i valori tabellari della media e della varianza in formato CSV, sia la rappresentazione grafica finale in formato PNG.

La struttura organizzativa e la composizione tabellare dei file CSV generati per rappresentare le matrici aggregate di media ($\bar{h}_{ij}$) e varianza (σ_{ij}^2) sono descritte nella Tabella 4.16.

Tabella 4.16: Struttura e descrizione dei campi costituenti i file CSV per le matrici delle metriche individuali e aggregate ($M \times D$).

Colonna	Tipo Dati	Descrizione Operativa
model_label	Stringa	Architettura o denominazione del modello valutato (i -esima riga della matrice).
<dataset_1>	Float	Valore (puntuale, medio $\bar{h}_{ij}$ o varianza σ_{ij}^2) della metrica sul primo dataset.
<dataset_2>	Float	Valore (puntuale, medio $\bar{h}_{ij}$ o varianza σ_{ij}^2) della metrica sul secondo dataset.
...	...	Valori associati ai restanti dataset di valutazione di test $j \in \{1, \dots, D\}$.
<dataset_D>	Float	Valore (puntuale, medio $\bar{h}_{ij}$ o varianza σ_{ij}^2) della metrica sull'ultimo dataset.
Average	Float	Media complessiva della metrica calcolata per la riga del modello su tutti i D dataset.

La separazione tra risultati per singolo seed e risultati aggregati consente pertanto di mantenere distinti due livelli di organizzazione implementativa: il primo conserva integralmente la configurazione associata a ciascuna realizzazione stocastica, mentre il secondo sintetizza le osservazioni relative alla stessa coppia modello–dataset lungo l’insieme dei seed. Tale organizzazione costituisce la trasposizione operativa della matrice di valutazione H_k definita nel modello teorico e produce gli artefatti persistenti utilizzati nelle successive procedure di analisi statistica.

4.5.3 Implementazione del Test di Welch

La verifica statistica della variabilità associata alle diverse configurazioni di addestramento è stata implementata mediante il test t di Welch, applicato separatamente a ciascuna tipologia di classificatore e al relativo gruppo di modelli. La procedura è stata progettata per confrontare le prestazioni del modello *INJECTION*, assunto come configurazione di riferimento, con quelle dei restanti modelli

generati durante la campagna multi-seed. L'analisi viene eseguita indipendentemente per i gruppi `nb15_models` e `hybrid_models`, mantenendo pertanto la distinzione architetturale già adottata nella struttura di persistenza dei risultati.

L'esecuzione della procedura è orchestrata dalla funzione `_welch_ttest()` del punto di ingresso `main.py`. Per una specifica tipologia di classificatore, vengono innanzitutto raccolti i file `classifications.csv` prodotti dalle precedenti fasi di valutazione per tutti i seed appartenenti all'insieme $\mathcal{S}$. I risultati vengono quindi forniti alla funzione `aggregate_welch_ttest_feature_scores_by_seed()`, che costituisce il primo livello di aggregazione dei dati destinati all'analisi statistica.

La funzione di aggregazione opera esclusivamente sulla metrica *F1-Score*, il cui utilizzo è definito tramite il parametro di configurazione `MLConstants.WELCH_TTEST_FEATURE`. Per ciascun file di classificazione vengono selezionate le colonne `model_name` e `F1-Score`, mentre il seed viene ricavato dalla directory che contiene il file `classifications.csv`. I dati provenienti dalle diverse esecuzioni vengono successivamente concatenati in un unico *DataFrame*.

Poiché ciascun file di classificazione contiene più osservazioni relative a differenti dataset valutati dallo stesso modello, la procedura effettua un'ulteriore aggregazione mediante un raggruppamento per coppia `model_name`–`seed`. Per ciascuna coppia viene calcolata la media della *F1-Score* sui dataset disponibili. Indicando con $f_{ij}^{(k)}$ il valore della *F1-Score* associato al modello i , al dataset j e al seed s_k , il valore utilizzato nel successivo test statistico è pertanto

$$\overline{F1}_i^{(k)} = \frac{1}{D_i} \sum_{j=1}^{D_i} f_{ij}^{(k)}, \quad (4.8)$$

dove D_i rappresenta il numero di dataset di valutazione considerati per il modello i nella configurazione corrente. Ne consegue che ciascun modello è rappresentato da un campione costituito dai valori medi ottenuti sui $K = 10$ seed, anziché dalla totalità delle singole osservazioni modello–dataset.

Il risultato di tale aggregazione viene trasformato in una struttura matriciale mediante un'operazione di *pivot*. I modelli costituiscono le righe, mentre ciascun seed viene trasformato in una colonna distinta. La struttura risultante presenta quindi una configurazione del tipo

```
id, model_name, seed_0, seed_1, ..., seed_12345
```


e costituisce la matrice di input per il test statistico. L'artefatto viene persistito nella directory

runs/<model_group>/<model_type>/welch_ttest/

con il nome F1-Score_means.csv. Tale file rappresenta esclusivamente la sintesi dei valori medi di *F1-Score* per modello e seed e costituisce una rappresentazione intermedia necessaria alla successiva procedura statistica. L'intero flusso di aggregazione dei dati e la pipeline di calcolo del test statistico sono schematizzati nella Figura 4.7.

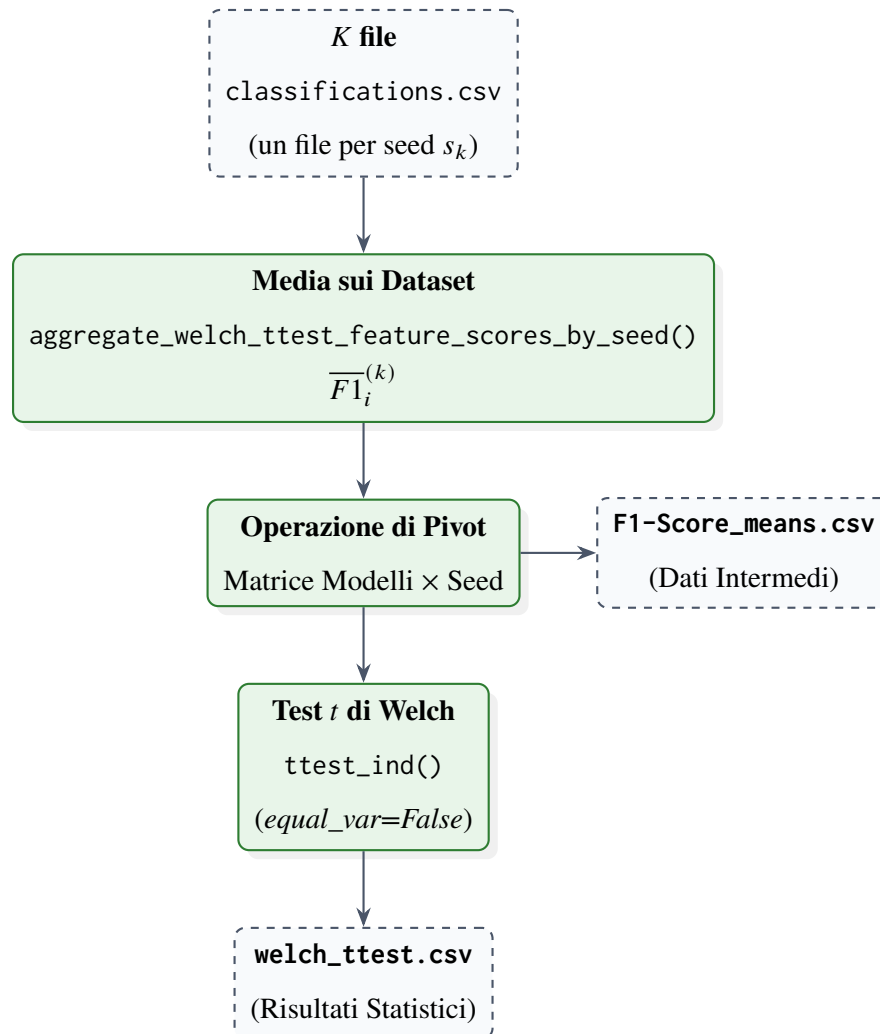


Figura 4.7: Pipeline di trasformazione dei dati per l'esecuzione del Test t di Welch, dal recupero delle metriche sui singoli seed alla persistenza degli esiti statistici.

Il Welch test viene quindi eseguito dalla funzione `calculate_welch_ttest_from_summary()` del modulo `src/utils/metrics.py`, utilizzando la funzione `ttest_ind()` del modulo

`scipy.stats`. L'opzione `equal_var=False` impone esplicitamente l'utilizzo della formulazione di Welch, nella quale non viene assunta l'uguaglianza delle varianze delle due popolazioni campionarie.

Per ogni gruppo di analisi viene individuato il modello di riferimento ricercando nel campo `model_name` la configurazione contenente la stringa `INJECTION`. La struttura dei benchmark garantisce la presenza di una sola configurazione di questo tipo nel *DataFrame* considerato; tale modello viene pertanto utilizzato come riferimento per tutti i successivi confronti. I valori corrispondenti ai dieci seed del modello di riferimento costituiscono il campione

$$\mathcal{F}_{\text{base}} = \left\{ \overline{F1}_{\text{base}}^{(1)}, \overline{F1}_{\text{base}}^{(2)}, \dots, \overline{F1}_{\text{base}}^{(K)} \right\}. \quad (4.9)$$

Per ciascun modello confrontato m , viene analogamente costruito il campione

$$\mathcal{F}_m = \left\{ \overline{F1}_m^{(1)}, \overline{F1}_m^{(2)}, \dots, \overline{F1}_m^{(K)} \right\}. \quad (4.10)$$

Il test inferenziale viene condotto assumendo l'indipendenza statistica delle osservazioni generate dalle diverse iterazioni con seed sintetici distinti ($s_k \in \mathcal{S}$) e senza vincoli sulla varianza delle due popolazioni. L'ipotesi nulla e l'ipotesi alternativa sono pertanto formulate, per ciascun confronto diretto tra $\mathcal{F}_{\text{base}}$ e $\mathcal{F}_m$, come

$$H_0 : \mu_{\text{base}} = \mu_m, \quad H_1 : \mu_{\text{base}} \neq \mu_m, \quad (4.11)$$

dove μ_{base} e μ_m rappresentano i valori medi teorici della *F1-Score* delle rispettive popolazioni campionarie.

La traduzione di tale impianto teorico nella sua controparte implementativa è riportata nel Listing 4.7, il quale illustra la procedura di estrazione dei campioni vettoriali dal *DataFrame* matriciale e l'invocazione del modulo `scipy.stats`. L'utilizzo del parametro `equal_var=False` applica rigorosamente la formulazione di Welch e la valutazione viene eseguita a due code (configurazione predefinita), senza introdurre una direzione privilegiata per l'ipotesi alternativa.

Codice 4.7: Estrazione dei campioni ed esecuzione del Test t di Welch.

```
1 # Extract target statistical scores for the reference model
2 # across all seed columns
3 ref_row = data[data["model_name"] == reference_model]
4 ref_scores = (
5     ref_row.drop(columns=["id", "model_name"]).values.flatten().astype(float)
6 )
7
8 results_list = []
9 # Iterate through each model entry in the summary dataframe for cross-testing
10 for _, row in data.iterrows():
11     current_model = row["model_name"]
12
13     if current_model == reference_model: continue
14
15     # Isolate numerical test values for the compared model
16     current_scores = row.drop(["id", "model_name"]).values.astype(float)
17
18     # Execute Welch's t-test (unequal variances assumed)
19     t_stat, p_value = stats.ttest_ind(
20         ref_scores, current_scores, equal_var=False
21     )
22
23     # Evaluate significance against target confidence threshold
24     is_significant = "YES" if p_value < alpha else "NO"
25
26     results_list.append({
27         "Reference_Model": reference_model,
28         "Compared_Model": current_model,
29         "t_statistic": t_stat,
30         "p_value": p_value,
31         "alpha": alpha,
32         "is_significant": is_significant
33     })
```

Il livello di significatività è stabilito *a priori* al valore di primo tipo

$$\alpha = 0,05, \quad (4.12)$$

acquisito dal parametro di configurazione `MLConstants.WELCH_TTEST_ALFA_VALUE`. La decisione statistica viene determinata direttamente confrontando il *p-value* restituito dal test con tale soglia prefissata. In particolare, la differenza prestazionale tra il modello in esame e la baseline viene marcata come statisticamente significativa quando

$$p < \alpha, \quad (4.13)$$

mentre in caso contrario il confronto viene classificato come non significativo. Non viene applicata alcuna procedura di correzione per confronti multipli ai *p-value* ottenuti dai diversi confronti tra modelli²².

Il confronto del modello *INJECTION* con sé stesso viene esplicitamente escluso dalla procedura, poiché non costituisce un confronto statistico tra configurazioni differenti. Per ciascuno degli altri modelli vengono invece registrati il valore della statistica *t*, il relativo *p-value*, il livello α adottato e l'esito della verifica mediante il campo `is_significant`.

I risultati vengono infine serializzati mediante la funzione `create_csv_from_data()` nel file `welch_ttest.csv`, salvato nella sottocartella di analisi

`runs/<model_group>/<model_type>/welch_ttest/`

La struttura complessiva del registro finale e la descrizione semantica dei singoli campi sono dettagliate nella Tabella 4.17. Tale schematizzazione consente di associare univocamente ogni confronto alla configurazione di riferimento, al modello sottoposto a verifica e ai parametri descrittivi restituiti dal test.

²²L'assenza di correzioni per test multipli (quali Bonferroni o Benjamini-Hochberg) è stata adottata in modo deliberato per preservare la sensibilità esplorativa dell'analisi. In tale contesto, ridurre la probabilità di errori di II tipo (falsi negativi) è stato ritenuto prioritario rispetto al controllo rigido del *Family-Wise Error Rate* (FWER), evitando così di smorzare prematuramente deviazioni prestazionali potenzialmente rilevanti indotte dalle diverse configurazioni di addestramento.

Tabella 4.17: Struttura e descrizione semantica dei campi costituenti il file di registro statistico `welch_ttest.csv`.

Campo	Descrizione Operativa
Reference_Model	Denominazione del modello di riferimento assunto come baseline (variante <i>INJECTION</i>).
Compared_Model	Denominazione del modello di confronto sottoposto a verifica statistica.
t_statistic	Valore numerico della statistica t di Welch calcolato tra i campioni di <i>F1-Score</i> .
p_value	Probabilità osservata di ottenere un valore uguale o più estremo sotto l'ipotesi nulla H_0 .
alpha	Livello di significatività stabilito <i>a priori</i> per la decisione statistica ($\alpha = 0,05$).
is_significant	Indicatore booleano dell'esito della verifica (True se $p < \alpha$, False altrimenti).

In tal modo, la pipeline mantiene formalmente separati il livello di aggregazione delle prestazioni sui singoli seed, memorizzato in `F1-Score_means.csv`, e il livello di diagnostica statistica inferenziale, rappresentato da `welch_ttest.csv`.

4.6 Strumenti di Analisi Diagnostica e Spiegabilità

La presente sezione formalizza l'implementazione degli strumenti di analisi diagnostica e di spiegabilità introdotti dal punto di vista metodologico nel Capitolo 3. L'infrastruttura software integra procedure dedicate alla caratterizzazione geometrica e distributiva dei benchmark, mediante proiezioni *Principal Component Analysis* (PCA) e stime di densità *Kernel Density Estimation* (KDE), nonché strumenti di interpretabilità post-hoc basati sui valori di Shapley per l'analisi del contributo delle singole caratteristiche alle decisioni dei classificatori. Tali componenti sono concepiti esclusivamente come strumenti diagnostici e non intervengono né nella fase di addestramento né nella modifica degli stimatori precedentemente serializzati.

Particolare attenzione viene dedicata alla coerenza del sistema di riferimento utilizzato nelle analisi diagnostiche. In accordo con il principio di standardizzazione dipendente dal sorgente introdotto nella Sezione 3.4.1, le statistiche di posizione e scala vengono ancorate al benchmark sorgente e successivamente riutilizzate per la trasformazione dei benchmark considerati, evitando il ricalcolo locale dei parametri di standardizzazione. La medesima impostazione viene applicata alle rappresentazioni PCA e KDE, con finalità rispettivamente orientate all'ispezione della disposizione dei dati nello spazio delle caratteristiche e all'analisi delle distribuzioni delle variabili selezionate.

Nelle sottosezioni seguenti vengono descritte le componenti che costituiscono l'architettura diagnostica. In primo luogo, la Sottosezione 4.6.1 illustra l'implementazione della standardizzazione diagnostica dipendente dal sorgente e delle procedure PCA e KDE, specificando le modalità di trasformazione dei benchmark e la configurazione delle rappresentazioni grafiche. Successivamente, la Sottosezione 4.6.2 descrive l'integrazione di SHAP nella pipeline di classificazione, il campionamento stratificato del test set e la gestione degli *explainer* per i classificatori basati su alberi. Infine, la Sottosezione 4.6.3 illustra il raccordo tra l'analisi SHAP e la successiva selezione delle caratteristiche per la rappresentazione KDE, evidenziando il carattere supervisionato della procedura e la definizione manuale della configurazione KDE_TOP_FEATURES.

La Tabella 4.18 sintetizza le caratteristiche operative, lo spazio di input e i moduli software associati a ciascuno degli strumenti considerati.

Tabella 4.18: Confronto sinottico tra gli strumenti diagnostici (PCA, KDE) e di spiegabilità (TreeSHAP) integrati nell’architettura software.

Strumento	Obiettivo	Spazio Input	Interpretabilità
PCA	Ispezione geometrica della separabilità globale e variabilità nello spazio 2D	Scalato (Z-Score ancorato al benchmark sorgente S)	Globale (riduzione dimensionale per componenti principali)
KDE	Analisi densometrica univariata del <i>domain shift</i> sulle feature chiave	Scalato (Z-Score ancorato sulle Top-3 feature di S)	Distribuzionale (stima della densità per classe in scala symlog)
TreeSHAP	Spiegabilità post-hoc del contributo delle feature alle predizioni dei classificatori	Originale (matrice delle caratteristiche del test set X_{test})	Locale e Aggregata (valori di Shapley e <i>summary plot</i>)

4.6.1 Standardizzazione Diagnostica, PCA e KDE

L’implementazione degli strumenti diagnostici descritti nella Sezione 3.4.1 è centralizzata nel modulo `plotting.py`. Le due procedure ricevono come input il file di configurazione contenente i percorsi dei benchmark e i relativi identificativi di dataset e producono automaticamente le rappresentazioni grafiche previste dal framework. L’analisi è finalizzata esclusivamente all’ispezione diagnostica della distribuzione dei dati e della loro disposizione nello spazio delle caratteristiche, senza intervenire nel processo di addestramento o nella fase di classificazione.

Per entrambe le procedure viene adottato il benchmark identificato dalla costante `SOURCE_IDENTIFIER`, impostata a `nb15_aggr_scaled`, come riferimento per la standardizzazione diagnostica. Tale scelta implementa il principio teorico introdotto nella Sezione 3.4.1, secondo il quale le statistiche di riferimento devono essere mantenute fisse durante la trasformazione dei diversi benchmark, così da preservare nello spazio trasformato eventuali traslazioni distributive riconducibili al *domain shift*.

L'architettura logica di questo processo di trasformazione dipendente dal sorgente è schematizzata nella Figura 4.8. Come mostrato nel diagramma di flusso, la stima dei parametri di posizione e scala (μ_S, σ_S) , così come la matrice di proiettori vettoriali $\mathbf{W}_S$, viene effettuata in modo atomico durante la fase di *fitting* sul solo benchmark sorgente. Successivamente, tali parametri congelati vengono applicati per via diretta sia a S sia a ciascun benchmark target T_k , garantendo l'invarianza del sistema di riferimento e scongiurando qualunque forma di *data leakage*.

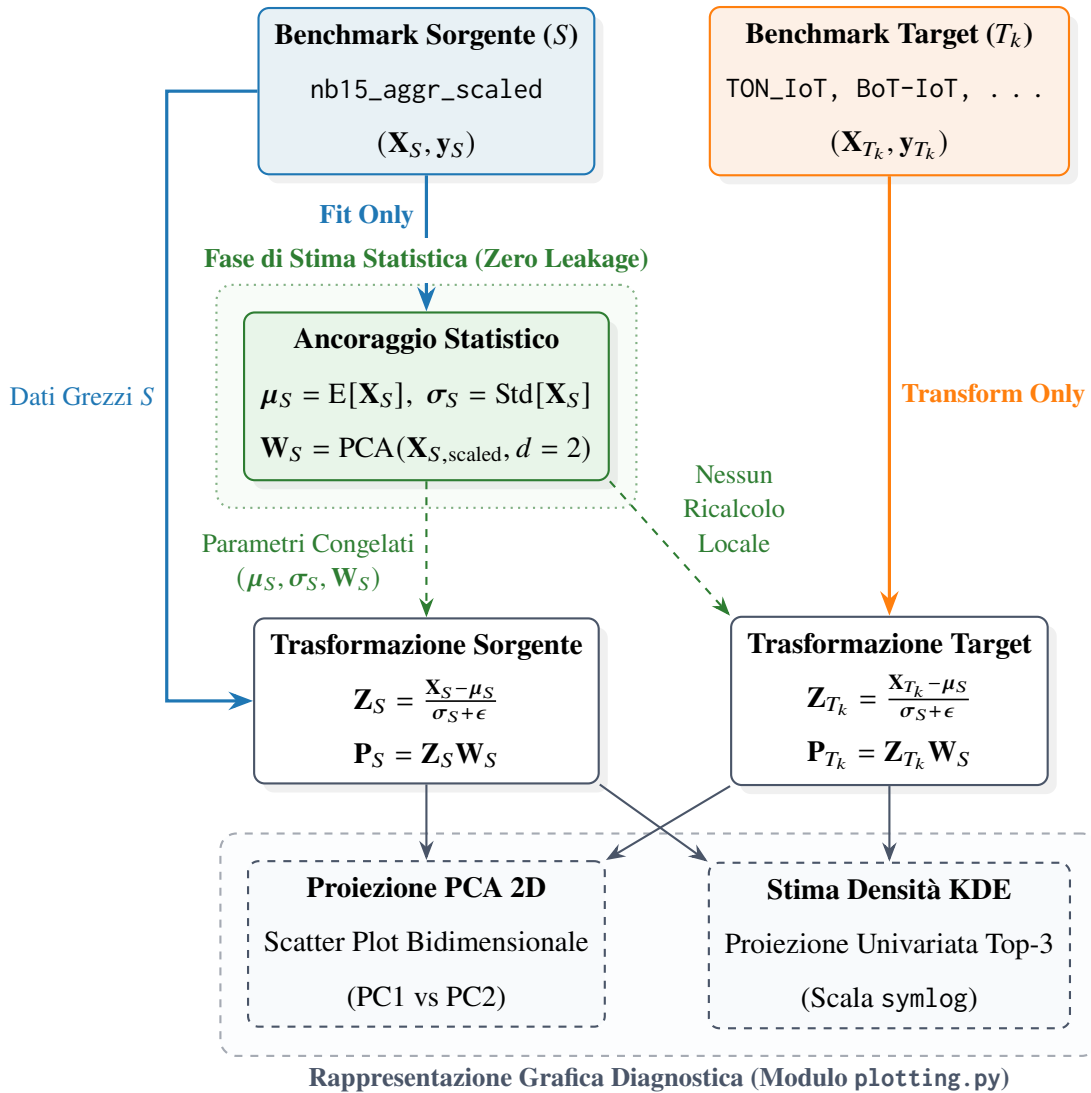


Figura 4.8: Schema concettuale della Standardizzazione Diagnostica Dipendente dal Sorgente: le statistiche di centralità e variabilità (μ_S, σ_S) e gli autovettori della PCA ($\mathbf{W}_S$) vengono stimati esclusivamente sul benchmark sorgente S e congelati. La trasformazione dei benchmark target T_k avviene per applicazione diretta di tali parametri, preservando lo scostamento distributivo reale (*domain shift*).

Proiezione PCA La procedura `save_all_pca_plots()` carica ciascun benchmark indicato nel file di configurazione ed estrae preliminarmente il sottoinsieme associato alla partizione test, identificata tramite la colonna `split_type`. Dalla matrice delle caratteristiche vengono rimosse le colonne `label`, `class` e `split_type`, mentre la variabile `label` viene conservata separatamente

per la successiva caratterizzazione delle classi nel grafico.

Per realizzare la standardizzazione diagnostica viene istanziato uno `StandardScaler` di `scikit-learn`. Nel caso della PCA, il modello di scaling viene adattato esclusivamente al sottoinsieme test del benchmark sorgente. Indicando con $\mathbf{X}_{S,\text{test}}$ tale matrice, l'operazione è implementata mediante:

Codice 4.8: Standardizzazione e adattamento della PCA sul benchmark sorgente.

```
1 scaler = StandardScaler()
2 pca = PCA(
3     n_components=MLConstants.PCA_COMPONENTS,
4     random_state=MLConstants.MAIN_SEED
5 )
6
7 X_source_scaled = scaler.fit_transform(X_source)
8 pca.fit(X_source_scaled)
```

La configurazione `PCA_COMPONENTS=2` vincola la trasformazione alla costruzione di un sottospazio bidimensionale, coerentemente con la finalità esclusivamente visuale dell'analisi. Gli autovettori principali sono quindi determinati una sola volta sul riferimento sorgente e successivamente riutilizzati per tutti i benchmark. Per ciascun dataset, la matrice delle caratteristiche viene trasformata mediante `scaler.transform()` e `pca.transform()`, evitando un nuovo adattamento locale:

Codice 4.9: Applicazione della trasformazione diagnostica ancorata al benchmark sorgente.

```
1 X_scaled = scaler.transform(X_test)
2 X_pca = pca.transform(X_scaled)
```

La procedura garantisce pertanto che tutti i benchmark siano rappresentati all'interno del medesimo sistema di riferimento statistico e geometrico. Tale impostazione è coerente con la formulazione teorica della standardizzazione dipendente dal sorgente, nella quale le statistiche (μ_S, σ_S) vengono mantenute fisse per evitare che una standardizzazione locale annulli le traslazioni distributive che si intendono osservare.

Per ogni benchmark viene infine generato uno scatter plot bidimensionale delle prime due componenti principali. Le osservazioni sono differenziate in funzione dell'etichetta binaria, con la

codifica Normal Traffic per la classe 0 e Attack/Anomaly per la classe 1. Il titolo del grafico riporta inoltre la percentuale complessiva di varianza spiegata dalle due componenti, mentre gli assi riportano separatamente il contributo percentuale di PC1 e PC2. I grafici vengono salvati con risoluzione pari a 300 dpi nella directory

```
data/preprocessed/metadata/plots/pca/
```

in formato png.

Stima KDE delle feature selezionate La procedura `save_all_kde_plots()` realizza invece l'analisi densometrica delle tre caratteristiche definite dalla costante `KDE_TOP_FEATURES`, pari a `pkts_per_sec`, `total_bytes` e `dst_win_byt`. La selezione coincide quindi con il sottoinsieme di caratteristiche individuato a livello metodologico mediante l'aggregazione dei valori SHAP, come previsto nella Sezione 3.4.1.

A differenza della procedura PCA, nella funzione `save_all_kde_plots()` non viene applicato un filtraggio esplicito sulla colonna `split_type`: le caratteristiche selezionate vengono caricate direttamente dai file dei benchmark e utilizzate per la costruzione delle distribuzioni. Il riferimento statistico è nuovamente individuato tramite `SOURCE_IDENTIFIER`; uno `StandardScaler` viene adattato alle sole feature disponibili nel dataset sorgente:

Codice 4.10: Adattamento dello standardizzatore sulle feature utilizzate per la KDE.

```
1 scaler = StandardScaler()
2 scaler.fit(df_source[valid_source_features])
```

Per ciascun benchmark, la trasformazione viene quindi applicata feature per feature utilizzando la media e il fattore di scala memorizzati nello scaler sorgente. La procedura implementa operativamente l'ancoraggio a (μ_S, σ_S) previsto dall'Equazione (3.40), mantenendo invariato il riferimento statistico anche per i benchmark target²³.

²³Il vettore `scaler.scale_` calcolato da `StandardScaler` contiene la deviazione standard per ciascuna caratteristica, comprensiva del fattore di tolleranza $\epsilon > 0$ introdotto automaticamente dalla libreria per garantire la stabilità numerica ed evitare divisioni per zero in presenza di varianza nulla.

Codice 4.11: Trasformazione diagnostica feature-per-feature ancorata ai parametri dello scaler sorgente.

```
1 for feature in valid_features:
2     if feature in scaler.feature_names_in_:
3         idx = list(scaler.feature_names_in_).index(feature)
4         mu_s = scaler.mean_[idx]
5         sigma_s = scaler.scale_[idx]
6         X_test_scaled[feature] = (X_test_raw[feature] - mu_s) / sigma_s
```

La stima della densità viene effettuata mediante `seaborn.kdeplot()`, con separazione delle curve sulla base della classe binaria e con normalizzazione indipendente delle densità (`common_norm=False`). La regione sottostante le curve viene riempita mediante `fill=True`, mentre il parametro `cut=0` limita la rappresentazione all'intervallo definito dai dati osservati. Non viene specificato esplicitamente alcun parametro di *bandwidth*: la larghezza del kernel è quindi determinata mediante il comportamento predefinito della procedura KDE utilizzata da `seaborn`.

Poiché l'implementazione deve consentire l'ispezione di distribuzioni caratterizzate da scale molto differenti e da possibili forti sbilanciamenti nella densità, l'asse verticale dei grafici viene rappresentato mediante una scala *symlog*, con parametro `linthresh=1e-5`²⁴. Questa configurazione è distinta dalla formulazione teorica della proiezione logaritmica della densità riportata nella Sezione 3.4.1: nel codice non viene calcolato esplicitamente $\log_{10}(\hat{f}_y + \epsilon)$, ma viene modificata direttamente la scala dell'asse mediante le funzionalità grafiche di `Matplotlib`.

²⁴La scala *symlog* (Symmetric Logarithmic) applica una trasformazione logaritmica all'asse y mantenendo un intervallo puramente lineare all'interno della soglia $[-\text{linthresh}, +\text{linthresh}]$. Ciò previene divergenze verso $-\infty$ per densità prossime o pari a zero.

Codice 4.12: Generazione delle stime KDE ed impostazione della scala logaritmica simmetrica su asse y.

```
1 sns.kdeplot(  
2     data=X_test_scaled, x=feature, hue=labels, fill=True, alpha=0.3,  
3     common_norm=False, linewidth=2,  
4     palette={'Normal Traffic': '#1f77b4', 'Attack/Anomaly': '#ff7f0e'},  
5     ax=ax, warn_singular=False, cut=0  
6 )  
7  
8 ax.set_yscale('symlog', linthresh=1e-5)
```

Per ciascun benchmark viene prodotto un pannello contenente una KDE per ogni feature disponibile tra quelle richieste. Le tre caratteristiche selezionate vengono pertanto visualizzate in forma affiancata, con asse orizzontale espresso in termini di *Z-Score* e asse verticale etichettato come densità. I grafici vengono salvati a 300 dpi nella directory

data/preprocessed/metadata/plots/kde/

con nomenclatura determinata dall'identificativo del benchmark e dal nome del dataset.

Nel complesso, le due procedure costituiscono istanze distinte del medesimo principio di standardizzazione diagnostica dipendente dal sorgente: la PCA utilizza il riferimento sorgente per costruire una trasformazione geometrica comune a tutti i benchmark, mentre la KDE utilizza il medesimo riferimento per rendere confrontabili, su scala adimensionale, le distribuzioni delle tre caratteristiche selezionate. La standardizzazione rimane in entrambi i casi confinata alla componente diagnostico-visiva del framework e non modifica i dati impiegati dall'addestramento dei classificatori, in accordo con la separazione metodologica stabilita nel Capitolo 3.

A completamento dell'analisi implementativa, la Tabella 4.19 riassume in modo sinottico le principali differenze operative e concettuali che intercorrono tra la procedura di proiezione PCA e la stima della densità KDE. Sebbene entrambe le funzioni condividano la medesima filosofia di standardizzazione ancorata al benchmark sorgente *S*, esse operano su granularità differenti: la PCA offre una vista spazio-geometrica ad ampio spettro sull'intero set di caratteristiche, mentre la KDE consente un'ispezione analitica e mirata sul comportamento distributivo delle singole variabili a più alto impatto decisionale.

Tabella 4.19: Confronto sinottico tra le procedure diagnostiche di proiezione PCA e stima della densità KDE nel modulo `plotting.py`.

Dimensione di Confronto	Procedura <code>save_all_pca_plots()</code>	Procedura <code>save_all_kde_plots()</code>
Obiettivo Diagnostico	Ispezione geometrica della separabilità globale e variabilità nello spazio 2D	Analisi densometrica univariata del <i>domain shift</i> sulle feature chiave
Partizione Input	Filtraggio esplicito della sola partizione di test	Dataset completo senza filtraggio sulla colonna <code>split_type</code>
Feature Considerate	Tutto lo spazio vettoriale (escluse colonne target e meta-dati)	Sottoinsieme ridotto Top-3 SHAP (<code>pkts_per_sec</code> , <code>total_bytes</code> , <code>dst_win_byt</code>)
Ancoraggio Statistico	Fit di <code>StandardScaler</code> e <code>PCA(n_components=2)</code> su $\mathbf{X}_{S,\text{test}}$	Fit di <code>StandardScaler</code> sulle sole 3 feature del sorgente S
Trasformazione Target	Applica <code>scaler.transform()</code> e <code>pca.transform()</code> congelati	Mappatura vettoriale ancorata: $Z_D = (X_D - \mu_S) / (\sigma_S + \epsilon)$
Spazio di Rappresentazione	Spazio proiettato bidimensionale (PC1 vs PC2) con % varianza spiegata	Asse orizzontale in Z-Score adimensionale; asse verticale in scala <code>symlog</code>
Output Grafico	Scatter plot 2D con distinzione cromatica binaria per classe	Pannelli affiancati di stime KDE riempite (<code>fill=True</code>)

4.6.2 Implementazione di SHAP e TreeSHAP

L'analisi di spiegabilità viene integrata nella pipeline di classificazione mediante la funzione `save_shap_plot()`, invocata da `classification_processing()`. La procedura riceve il modello precedentemente addestrato, la matrice delle caratteristiche del test set, le relative etichette e i metadati necessari alla generazione e alla persistenza della rappresentazione grafica. L'ana-

lisi rimane pertanto confinata alla fase diagnostica e non modifica il modello precedentemente serializzato.

Prima della costruzione delle spiegazioni viene effettuato un campionamento stratificato e deterministico del test set (ancorato al seed di esecuzione), utilizzato per definire il dataset di background dell'algoritmo TreeSHAP. La dimensione massima del campione è fissata dalla costante `SHAP_MAX_SAMPLES=500`²⁵. Le quote assegnate alle due classi sono calcolate mediante il parametro `NORMAL_ANOMALY_RATIO`, secondo:

$$N_{\text{anom}} = \left\lfloor \frac{\text{SHAP_MAX_SAMPLES}}{\text{NORMAL_ANOMALY_RATIO}} \right\rfloor, \quad N_{\text{norm}} = \text{SHAP_MAX_SAMPLES} - N_{\text{anom}}. \quad (4.14)$$

Nel codice il valore effettivamente assegnato a `NORMAL_ANOMALY_RATIO` è pari a 10 (corrispondente al rapporto di sbilanciamento $\rho = 10 : 1$), mentre il campionamento delle singole classi viene effettuato mediante `pandas.Series.sample()` con `random_state=127001`. Gli indici estratti per il traffico nominale e per quello anomalo vengono successivamente concatenati e utilizzati per selezionare le corrispondenti righe di `X_test`. Nel caso in cui non sia disponibile alcun campione valido, viene applicato un campionamento casuale del test set, anch'esso limitato a `SHAP_MAX_SAMPLES` e controllato dallo stesso seed.

Nel Listato 4.13 è riportata la porzione di codice responsabile dell'estrazione stratificata, che bilancia i vincoli prestazionali con la necessità di mantenere una corretta rappresentatività di entrambe le classi.

²⁵Il sotto-campionamento deterministico del dataset di background costituisce una misura essenziale per contenere la complessità computazionale del calcolo dei valori di Shapley. Riducendo la cardinalità del campione di riferimento a una quota rappresentativa, si previene un incremento insostenibile dei tempi di esecuzione e dell'occupazione di memoria durante l'algoritmo TreeSHAP, preservando al contempo la convergenza stocastica delle stime empiriche e la stabilità delle spiegazioni rese.

Codice 4.13: Campionamento stratificato del test set per l'analisi SHAP.

```
1 # Stratified sampling: Align labels with features
2 y_series = pd.Series(
3     y_test, index=X_test.index if hasattr(X_test, 'index') else None
4 )
5
6 # Calculate class quotas: e.g., 10:1 ratio on maximum samples
7 n_anomaly = MLConstants.SHAP_MAX_SAMPLES // MLConstants.NORMAL_ANOMALY_RATIO
8 n_normal = MLConstants.SHAP_MAX_SAMPLES - n_anomaly
9
10 # Extract sampled indices for both classes, capping at available instances
11 idx_normal = pd.Index([])
12 if sum(y_series == 0) > 0:
13     idx_normal = y_series[y_series == 0].sample(
14         n=min(n_normal, sum(y_series == 0)),
15         random_state=MLConstants.MAIN_SEED
16     ).index
17 idx_anomaly = pd.Index([])
18 if sum(y_series == 1) > 0:
19     idx_anomaly = y_series[y_series == 1].sample(
20         n=min(n_anomaly, sum(y_series == 1)),
21         random_state=MLConstants.MAIN_SEED
22     ).index
23
24 # Union of stratified indices
25 sampled_idx = idx_normal.append(idx_anomaly)
26
27 # Filter the original test set using the stratified indices
28 # (or fallback to standard random sampling)
29 if not sampled_idx.empty:
30     X_test_sampled = X_test.loc[sampled_idx]
31 else:
32     X_test_sampled = X_test.sample(
33         n=min(MLConstants.SHAP_MAX_SAMPLES, len(X_test)),
34         random_state=MLConstants.MAIN_SEED
35     )
```


La costruzione dello spiegatore viene effettuata dinamicamente in funzione del classificatore ricevuto. L'implementazione tenta inizialmente di utilizzare `shap.TreeExplainer`, configurato con `feature_perturbation="tree_path_dependent"`²⁶ e `model_output="raw"`, opzioni orientate all'analisi di modelli ad albero quali `RandomForestClassifier` e `DecisionTreeClassifier`. Nel caso in cui tale configurazione non sia compatibile con il modello, viene eseguito un secondo tentativo mediante `shap.TreeExplainer(model)`. Qualora anche questa inizializzazione fallisca, viene utilizzato il costrutto generico `shap.Explainer(model, X_test_sampled)`. Tale meccanismo consente di mantenere un'unica interfaccia di generazione delle spiegazioni anche in presenza delle differenti modalità di supporto offerte dalla libreria SHAP ai classificatori considerati, come evidenziato dalla struttura condizionale `try-except` del Listato 4.14.

Codice 4.14: Inizializzazione dinamica dello spiegatore SHAP con catena di fallback.

```
1 # Initialize Explainer dynamically to support RF, DT, and HGB
2 try:
3     # First attempt: standard TreeExplainer optimized for RF and DT
4     explainer = shap.TreeExplainer(
5         model,
6         feature_perturbation="tree_path_dependent",
7         model_output="raw"
8     )
9 except Exception as explainer_err:
10     # Fallback attempt: generic Explainer
11     # or simplified TreeExplainer (crucial for HGB compatibility)
12     try:
13         explainer = shap.TreeExplainer(model)
14     except Exception:
15         explainer = shap.Explainer(model, X_test_sampled)
```

L'intera architettura della pipeline, articolata nelle sue quattro fasi di elaborazione (dal campionamento iniziale alla persistenza dell'artefatto), è schematizzata in Figura 4.9.

²⁶La modalità `tree_path_dependent` calcola le aspettative condizionali seguendo la struttura trasversale dei nodi degli alberi senza richiedere un dataset di *background* esplicito (a differenza dell'approccio *interventional*). Ciò riduce drasticamente il costo computazionale e gestisce la correlazione tra le variabili.

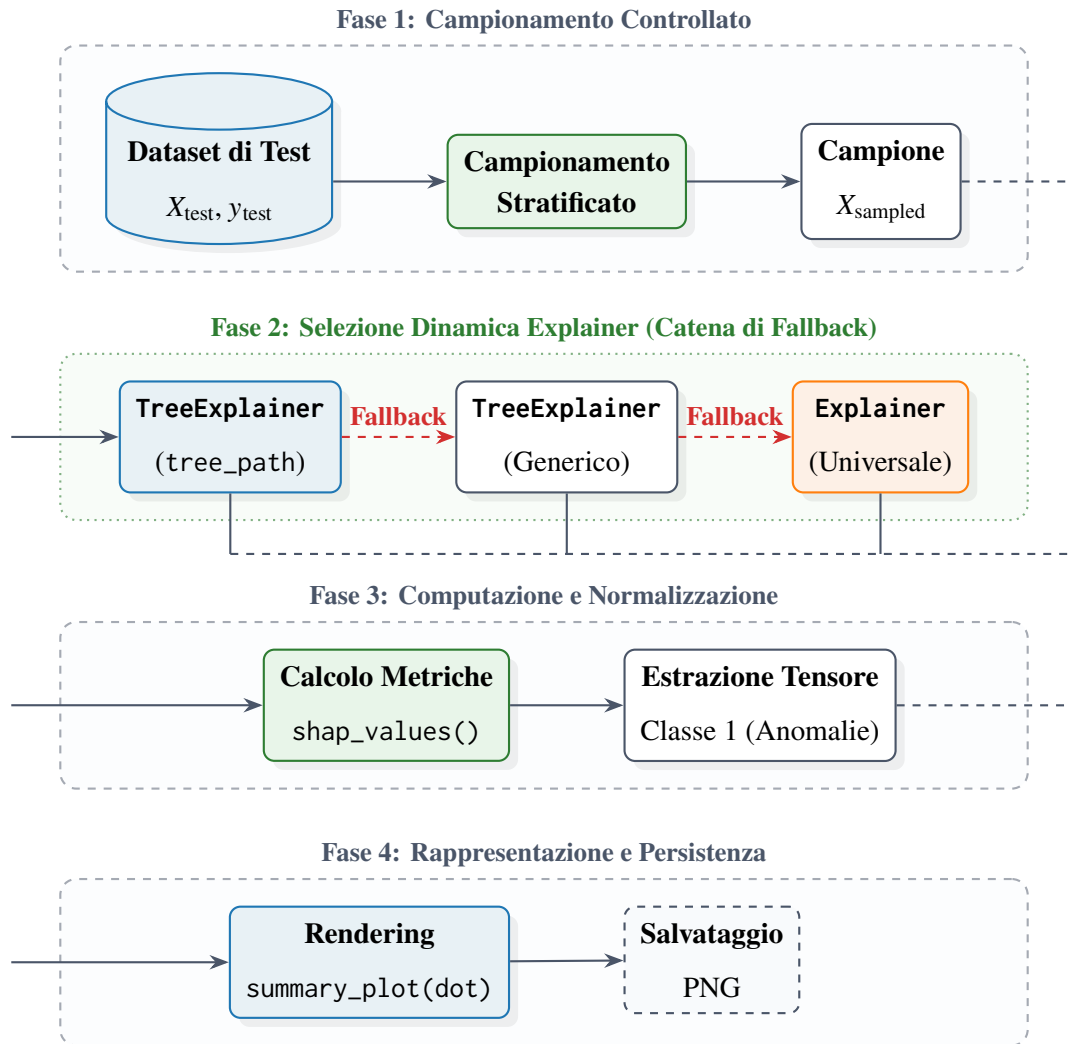


Figura 4.9: Architettura della pipeline di spiegabilità SHAP per i modelli di classificazione. Il diagramma illustra le quattro fasi sequenziali disposte in blocchi verticalmente allineati con connessioni di flusso circolare (effetto Pacman) sul margine destro/sinistro tra le diverse fasi.

I valori di Shapley vengono quindi calcolati sul campione selezionato mediante il metodo `shap_values()`. L'implementazione disabilita esplicitamente il controllo di additività mediante `check_additivity=False`²⁷, quando tale argomento è supportato dallo specifico explainer. In caso contrario, viene eseguita una seconda chiamata senza tale parametro. La procedura prevede inoltre una normalizzazione della struttura restituita dalla libreria, necessaria a gestire le differenti

²⁷La disabilitazione del controllo di additività previene l'interruzione improvvisa dell'esecuzione dovuta a microscopiche discrepanze numeriche dovute alla precisione in virgola mobile (*floating-point precision*) tra la somma dei valori SHAP e l'output teorico in modelli ad albero complessi.

rappresentazioni possibili dei valori SHAP. In particolare, quando viene restituita una lista di matrici corrispondenti alle due classi, viene selezionata la componente relativa alla classe positiva, ossia Attack/Anomaly. Analogamente, nel caso di un array tridimensionale, viene selezionata la componente associata all'indice di classe 1, mentre una matrice già bidimensionale viene utilizzata direttamente.

La spiegazione risultante è infine visualizzata mediante `shap.summary_plot()` utilizzando la configurazione `plot_type="dot"`. Tale rappresentazione associa a ciascuna caratteristica i valori SHAP osservati sui campioni selezionati, permettendo di rappresentare simultaneamente l'ampiezza e il verso del contributo delle singole variabili alla predizione. Il numero massimo di caratteristiche visualizzate viene impostato dinamicamente al numero di colonne della matrice `X_test_sampled`, mediante il parametro `max_display=n_features`. La dimensione verticale della figura viene inoltre adattata al numero complessivo di feature, secondo una dimensione minima di 6 pollici e un incremento proporzionale pari a 0.4 pollici per caratteristica.

Il Listato 4.15 illustra la sequenza logica che, partendo dall'estrazione tensoriale tramite l'oggetto `explainer`, normalizza la struttura dati e ne adatta il rendering in funzione della dimensione del *feature space*.

Codice 4.15: Calcolo, normalizzazione dimensionale e rendering dei valori SHAP.

```
1 # Compute SHAP values with additivity check handling
2 try:
3     shap_values = explainer.shap_values(
4         X_test_sampled, check_additivity=False
5     )
6 except TypeError:
7     shap_values = explainer.shap_values(X_test_sampled)
8
9 # Handle output disparities (e.g. Explainer objects vs raw ndarrays)
10 if hasattr(shap_values, "values"):
11     shap_values_raw = shap_values.values
12 else:
13     shap_values_raw = shap_values
14
15 # Extract positive class (Anomaly) values based on structural dimensions
16 if isinstance(shap_values_raw, list) and len(shap_values_raw) == 2:
17     shap_values_raw = shap_values_raw[1]
18 elif hasattr(shap_values_raw, 'shape') and len(shap_values_raw.shape) == 3:
19     shap_values_raw = shap_values_raw[:, :, 1]
20 elif hasattr(shap_values_raw, 'shape') and len(shap_values_raw.shape) == 2:
21     pass
22
23 # Generate and configure the dynamic plot
24 n_features = X_test_sampled.shape[1]
25 plt.figure(figsize=(8, max(6, n_features * 0.4)))
26
27 # Force the "dot" view to show individual feature impacts
28 shap.summary_plot(
29     shap_values_raw,
30     X_test_sampled,
31     show=False,
32     max_display=n_features,
33     plot_type="dot"
34 )
```

Il grafico SHAP prodotto viene salvato con risoluzione pari a 300 dpi nella directory shap e organizzata per modello. Il nome del file png viene costruito combinando il tipo di dataset, il nome del dataset. La procedura consente pertanto di mantenere separati gli artefatti di spiegabilità relativi ai diversi modelli e benchmark analizzati.

A completamento della descrizione implementativa, la Tabella 4.20 sintetizza i parametri principali definiti all'interno del modulo, esplicitando le configurazioni adottate e le rispettive motivazioni ingegneristiche.

Tabella 4.20: Sintesi dei parametri di configurazione e delle scelte ingegneristiche della pipeline SHAP.

Parametro = Impostazione	Scopo Ingegneristico e Funzionale
SHAP_MAX_SAMPLES = 500	Contenimento del costo computazionale e della complessità temporale $O(N)$ nel calcolo dei valori SHAP.
NORMAL_ANOMALY_RATIO = 10	Preservazione del rapporto di sbilanciamento nativo ($\rho = 10 : 1$) tra la classe nominale e quella anomala.
feature_perturbation = "tree_path_dependent"	Calcolo rapido delle aspettative condizionali lungo i percorsi degli alberi senza dipendenza da un dataset di <i>background</i> esplicito.
plot_type = "dot"	Visualizzazione ad alta densità che mostra simultaneamente ampiezza, verso e dispersione dei contributi di ciascuna caratteristica.
check_additivity = False	Prevenzione dell'interruzione della pipeline causata da microscopiche discrepanze numeriche di precisione in virgola mobile (<i>floating-point</i>).

Nel complesso, l'implementazione realizza una pipeline di spiegabilità indipendente dalla fase di addestramento: il modello già addestrato viene utilizzato come input per TreeExplainer o, quando necessario, per l'explainer generico di shap, mentre le spiegazioni vengono calcolate su un campione controllato del test set. Questa separazione mantiene coerente l'analisi con il

principio metodologico secondo cui SHAP viene utilizzato come strumento diagnostico post-hoc per analizzare il contributo delle caratteristiche alle decisioni dei classificatori ad albero.

4.6.3 Pipeline di Aggregazione SHAP–KDE

Il raccordo tra l'analisi di spiegabilità SHAP e la successiva analisi densometrica KDE non è implementato mediante una procedura automatizzata all'interno del framework software²⁸. La selezione delle caratteristiche destinate alla rappresentazione KDE viene infatti effettuata attraverso un passaggio di analisi manuale dei grafici SHAP prodotti dalla pipeline diagnostica.

Al fine di formalizzare la natura ibrida di tale procedura, la Figura 4.10 illustra l'architettura logica del raccordo *Human-in-the-Loop*. Il diagramma evidenzia la scomposizione del flusso di lavoro in tre fasi sequenziali: una prima fase di automazione diagnostica, una fase intermedia di supervisione umana in cui avviene l'estrazione quantitativa del parametro empirico, e una successiva fase di esecuzione densometrica automatizzata.

²⁸La scelta di non automatizzare tale raccordo previene la propagazione non controllata di rumore transitorio o di instabilità di *ranking* tra i differenti *seed* sperimentali.

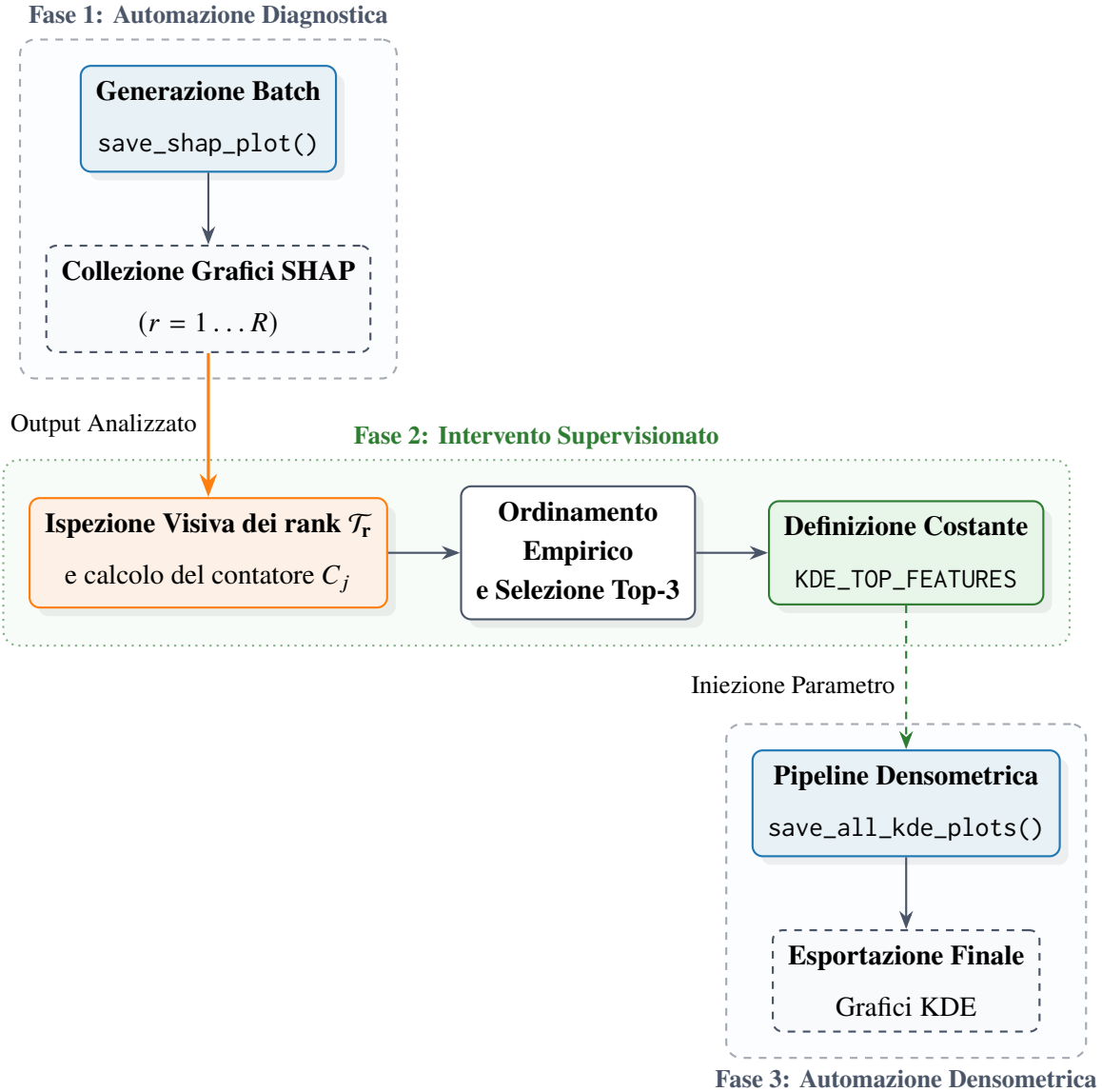


Figura 4.10: Diagramma architetturale del raccordo *Human-in-the-Loop* tra l'analisi diagnostica (SHAP) e la valutazione densometrica (KDE). L'intervento manuale (Fase 2) funge da disaccoppiamento metodologico tra le due pipeline automatizzate.

Per ciascun grafico SHAP generato vengono considerate le prime tre caratteristiche riportate nel `summary_plot`. Poiché la rappresentazione `plot_type="dot"` ordina le caratteristiche sulla base della loro rilevanza globale, le prime tre righe del grafico costituiscono il sottoinsieme di interesse per la procedura di conteggio. L'analisi viene ripetuta considerando tutti i grafici SHAP prodotti per le combinazioni di modelli, benchmark e seed previste dal protocollo sperimentale.

A partire da tale ispezione viene costruito manualmente un contatore per ciascuna caratteristica j , definito come:

$$C_j = \sum_{r=1}^R \mathbb{I}(j \in \mathcal{T}_r), \quad (4.15)$$

dove R rappresenta il numero complessivo di grafici SHAP analizzati, $\mathcal{T}_r$ denota l'insieme delle tre caratteristiche collocate nelle prime posizioni del grafico r e $\mathbb{I}(\cdot)$ è la funzione indicatrice. Di conseguenza, C_j esprime il numero di occorrenze con cui la caratteristica j compare tra le prime tre posizioni nelle spiegazioni considerate. I risultati del conteggio empirico effettuato sulle spiegazioni diagnostiche sono sintetizzati nella Tabella 4.21.

Tabella 4.21: Distribuzione delle frequenze di occorrenza C_j delle caratteristiche registrate nelle prime tre posizioni dei grafici SHAP.

Caratteristica (j)	Occorrenze (C_j)	Frequenza Relativa (C_j/R)
pkts_per_sec	C_1	$\sim 18.89\%$
total_bytes	C_2	$\sim 17.04\%$
dst_win_byt	C_3	$\sim 11.67\%$

La frequenza così ottenuta viene utilizzata come criterio empirico di selezione delle caratteristiche da sottoporre alla successiva stima KDE. Le tre caratteristiche caratterizzate dai valori di C_j più elevati sono risultate essere pkts_per_sec, total_bytes e dst_win_byt. La selezione viene quindi trasferita manualmente nella configurazione del framework mediante la costante:

Codice 4.16: Configurazione delle feature selezionate per l'analisi KDE.

```
1 KDE_TOP_FEATURES = ['pkts_per_sec', 'total_bytes', 'dst_win_byt']
```

La funzione `save_all_kde_plots()` utilizza successivamente tale lista come insieme di caratteristiche da analizzare, senza ricalcolare automaticamente il conteggio C_j né eseguire una nuova selezione sulla base dei grafici SHAP. Il passaggio SHAP-KDE deve pertanto essere inteso come una procedura di selezione supervisionata manualmente dall'analista: SHAP fornisce il criterio diagnostico per individuare le caratteristiche più ricorrenti nelle posizioni di maggiore rilevanza, mentre la KDE costituisce lo strumento successivo per l'ispezione delle rispettive distribuzioni.

Nel protocollo adottato non si è verificata alcuna situazione di parità tra caratteristiche nei valori del contatore C_j ²⁹ tali da richiedere una regola di spareggio. La lista finale KDE_TOP_FEATURES rappresenta pertanto la selezione conclusiva utilizzata nella fase di analisi densometrica descritta nella Sezione 4.6.1.

4.7 Sintesi della Pipeline Implementativa

La pipeline implementativa descritta nel presente capitolo costituisce la traduzione operativa del framework metodologico definito nel Capitolo 3. L'architettura è organizzata in moduli logicamente distinti, corrispondenti alle principali fasi del ciclo di elaborazione: acquisizione e armonizzazione dei dati, costruzione dei benchmark, addestramento multi-seed, inferenza, valutazione e analisi diagnostica. Tale decomposizione consente di mantenere separati il flusso computazionale principale e le procedure di analisi, preservando la modularità dell'infrastruttura software e la riproducibilità delle singole fasi. Inoltre, la struttura disaccoppiata garantisce un'elevata estensibilità dell'architettura software: l'integrazione di nuovi dataset target o l'inclusione di ulteriori algoritmi di classificazione può essere effettuata in modo trasparente, agendo unicamente sulle definizioni contenute nel file di configurazione centralizzato (`config.py`), senza dover modificare la logica di elaborazione dei singoli moduli computazionali.

La Tabella 4.22 riassume la mappa dei moduli Python sviluppati, i rispettivi file sorgente, le responsabilità primarie e i flussi di Input/Output che attraversano l'architettura software.

²⁹In caso di parità nel contatore C_j , la regola formale di spareggio avrebbe previsto un ordinamento secondario decrescente basato sull'impatto medio assoluto di Shapley $|\bar{\phi}_j| = \frac{1}{R} \sum_{r=1}^R |\phi_{j,r}|$, calcolato esclusivamente sulle esecuzioni che hanno generato l'ex aequo.

Tabella 4.22: Riepilogo dell'architettura software end-to-end: moduli Python, file sorgente, responsabilità e flussi di Input/Output.

File Sorgente	Responsabilità Principale	Input / Output
Punto di Ingresso		
main.py	Gestione CLI interattiva e orchestrazione del ciclo esecutivo multi-seed (s_0 e S).	In: Comandi CLI, parametri Out: Trigger pipeline esecutive
Allineamento Feature		
src/data_preprocessing.py	Mappatura ψ_k , derivazione metriche e allineamento allo spazio coordinato ($d = 16$).	In: Dataframe parziali Out: Dataframe coordinati (16 feature)
Core Preprocessing		
src/file_preprocessing.py	Pulizia, campionamento stratificato ($\rho = 10$), bilanciamento e split 80/20 train/test.	In: Dataframe coordinati Out: Benchmark (data/preprocessed/)
Gestione Modelli		
src/models.py	Istanziamento, fitting dei classificatori (DT, RF, HGB), naming e serializzazione pesi.	In: Dataset pre-processati Out: Pesi .joblib, update models_metadata.csv
Inferenza & Valutazione		
src/classification.py	Inferenza sui test set, generazione matrici di confusione e calcolo metriche.	In: Modelli .joblib e test set Out: CSV risultati in runs/.../results/

Diagnostica & Visualizzazione		
src/plotting.py	Mappe di calore, diagnostica spaziale (PCA, KDE con StandardScaler) e XAI (TreeSHAP).	In: Matrici, log e dataset Out: Grafici e diagrammi (PNG/PDF)
Configurazione		
src/utils/config.py	Centralizzazione costanti (MLConstants, $\rho = 10$, semi stocastici, path).	In: N.D. Out: Parametri globali di runtime
I/O & Persistenza		
src/utils/file_utils.py	Gestione I/O su disco, salvataggio/caricamento sicuro di artefatti e registri CSV.	In: Oggetti Python, file dati Out: Persistenza strutturata su disco
Metriche Spiegabili		
src/utils/metrics.py	Formalizzazione e calcolo sintetico delle metriche (Accuracy, F1, ROC-AUC, ecc.).	In: Vettori etichette y e $\hat{y}$ Out: Dizionari/Dataframe di metriche

Il processo ha origine dai domini sorgente e target, i cui dati vengono sottoposti alle procedure di filtraggio e allineamento descritte nella Sezione 4.3. L'operatore ψ_k riconduce le rappresentazioni eterogenee a uno spazio condiviso di caratteristiche, sul quale vengono successivamente applicati i criteri di composizione e partizionamento necessari alla costruzione dei benchmark sperimentali. I dataset risultanti costituiscono l'input del modulo di classificazione, senza introdurre alcuna ulteriore scalatura esplicita nel flusso principale, in accordo con quanto discusso nella Sezione 4.3.3.

La fase di apprendimento viene quindi eseguita mediante l'istanziatura dei classificatori *Decision Tree*, *Random Forest* e *Histogram-based Gradient Boosting*, ripetuta sui semi stocastici definiti dal protocollo. Per ogni combinazione tra algoritmo, benchmark e seme vengono prodotti gli artefatti relativi al modello e ai relativi metadati, mantenendo l'isolamento delle diverse esecuzioni nella struttura gerarchica della directory runs/.

I modelli così ottenuti vengono successivamente utilizzati nella fase di inferenza sugli insiemi

di test, generando predizioni e punteggi di probabilità SHAP. Le predizioni e i relativi punteggi alimentano il calcolo delle metriche e la costruzione delle matrici di valutazione, mentre l'aggregazione delle osservazioni sui diversi semi fornisce gli ingressi per la successiva procedura di confronto statistico. Gli artefatti prodotti lungo tale percorso vengono persistiti secondo la struttura organizzativa descritta nella Sezione 4.1.1, mantenendo distinti modelli, risultati e prodotti delle analisi.

Parallelamente al flusso principale opera il modulo diagnostico. La standardizzazione utilizzata per PCA e KDE viene mantenuta separata dal grafo computazionale di training e inferenza. Le caratteristiche individuate mediante l'ispezione dei grafici SHAP vengono quindi utilizzate per configurare l'analisi KDE, secondo la procedura supervisionata descritta nella Sezione 4.6.3.

La composizione complessiva degli stadi implementativi è sintetizzata nella Figura 4.11, che evidenzia, con il relativo dettaglio parametrico, il flusso principale della pipeline e il relativo ramo diagnostico.

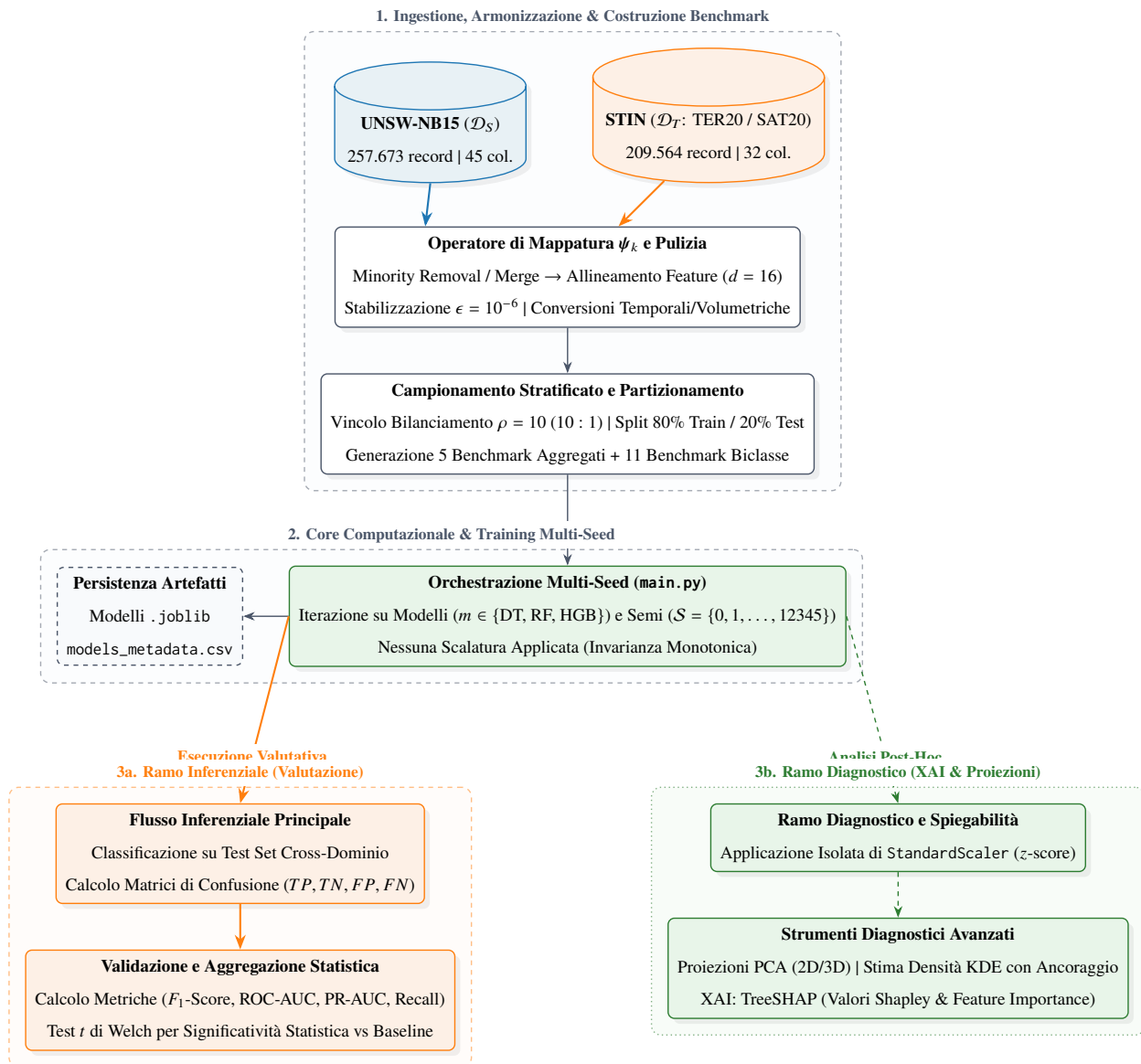


Figura 4.11: Architettura software end-to-end e flusso di esecuzione unificato: dall’ingestione dei dataset grezzi, passando per l’armonizzazione delle feature e il ciclo di addestramento multi-seed, fino alla biforcazione tra ramo inferenziale (valutazione prestazionale) e ramo diagnostico (PCA, KDE e TreeSHAP).

La Figura 4.11 evidenzia pertanto la separazione funzionale tra il percorso principale, destinato alla costruzione e alla valutazione degli stimatori, e il ramo diagnostico, dedicato alla caratterizzazione geometrica, distributiva e interpretativa dei dati e dei modelli. La persistenza degli artefatti costituisce il punto comune tra le diverse fasi, consentendo di mantenere disponibili modelli,

metriche, matrici e rappresentazioni grafiche secondo una struttura riproducibile.

La descrizione implementativa si conclude con la definizione di tale catena end-to-end. Gli output generati dalla pipeline costituiscono quindi la base documentale e quantitativa sulla quale viene sviluppata, nel Capitolo 5, l'analisi dei risultati sperimentali.

5. Risultati Sperimentali e Analisi

Il presente capitolo illustra e analizza i risultati sperimentali ottenuti mediante il framework metodologico e la pipeline implementativa descritti nei capitoli precedenti. L'analisi considera le prestazioni dei classificatori sui differenti benchmark, la variabilità introdotta dalla componente stocastica, la significatività statistica delle differenze osservate e le evidenze diagnostiche ottenute mediante PCA, SHAP e KDE.

5.1 Organizzazione dell'Analisi dei Risultati

5.1.1 Criteri di Presentazione e Confronto

5.1.2 Configurazioni Sperimentali e Criteri di Aggregazione

5.2 Risultati dei Modelli Aggregati

5.2.1 Prestazioni del Modello Nativo Sorgente

5.2.2 Degradazione delle Prestazioni in Presenza di Domain Shift

5.2.3 Prestazioni dei Modelli Ibridi Aggregati

5.2.4 Confronto tra Baseline INJECTION, Modello Sorgente e Oracle

5.3 Risultati dei Modelli Biclasse

5.3.1 Rilevazione delle Singole Famiglie di Attacco nel Dominio Sorgente

5.3.2 Rilevazione delle Singole Famiglie di Attacco nei Domini Target

5.3.3 Confronto tra Modelli Specialistici Sorgente e Ibridi

5.4 Analisi della Variabilità Multi-Seed

5.4.1 Distribuzione delle Prestazioni sui Seed

5.4.2 Media e Varianza delle Metriche di Prestazione

5.4.3 Stabilità delle Prestazioni tra le Configurazioni

5.5 Analisi della Significatività Statistica

5.5.1 Confronto della *F1-Score* con la Baseline INJECTION

5.5.2 Risultati del Test di Welch

Ringraziamenti

Qui i ringraziamenti, se lo desideri.

Bibliografia

- [1] Sajid Alam e Wang-Cheol Song. «Intent-Based Network Resource Orchestration in Space-Air-Ground Integrated Networks: A Graph Neural Networks and Deep Reinforcement Learning Approach». In: *IEEE Access* 12 (2024), pp. 185057–185077. DOI: 10.1109/ACCESS.2024.3507829. URL: <https://ieeexplore.ieee.org/document/10770209>.
- [2] Ahmad Taher Azar et al. «Deep Learning Based Hybrid Intrusion Detection Systems to Protect Satellite Networks». In: *Journal of Network and Systems Management* 31.4 (2023), p. 82. ISSN: 1573-7705. DOI: 10.1007/s10922-023-09767-8. URL: <https://link.springer.com/article/10.1007/s10922-023-09767-8>.
- [3] Leo Breiman. «Random Forests». In: *Machine Learning* 45.1 (2001), pp. 5–32. ISSN: 1573-0565. DOI: 10.1023/A:1010933404324. URL: <https://link.springer.com/article/10.1023/A:1010933404324#citeas>.
- [4] Leo Breiman et al. *Classification and Regression Trees*. Belmont, CA: Wadsworth International Group, 1984. ISBN: 978-0534980535.
- [5] Tianqi Chen e Carlos Guestrin. «XGBoost: A Scalable Tree Boosting System». In: *CoRR* abs/1603.02754 (2016). DOI: 10.1145/2939672.2939785. URL: <http://arxiv.org/abs/1603.02754>.
- [6] Hal Daumé III. «Frustratingly Easy Domain Adaptation». In: *CoRR* abs/0907.1815 (2009). DOI: 10.48550/arXiv.0907.1815. URL: <https://arxiv.org/abs/0907.1815>.
- [7] Janez Demšar. «Statistical Comparisons of Classifiers over Multiple Data Sets». In: *Journal of Machine Learning Research* 7 (2006), pp. 1–30. DOI: 10.5555/1248547.1248548. URL: https://www.researchgate.net/publication/220320196_Statistical_Comparisons_of_Classifiers_over_Multiple_Data_Sets.
- [8] Abolfazl Farahani et al. «A Brief Review of Domain Adaptation». In: *Advances in Data Science and Information Engineering*. Springer, 2021, pp. 877–894. DOI: 10.1007/978-3-030-71704-9_59. URL: <https://arxiv.org/abs/2010.03978>.

- [9] Tom Fawcett. «An introduction to ROC analysis». In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874. DOI: 10.1016/j.patrec.2005.10.010. URL: https://www.researchgate.net/publication/222511520_Introduction_to_ROC_analysis.
- [10] Jerome H. Friedman. «Greedy Function Approximation: A Gradient Boosting Machine». In: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232. DOI: 10.1214/aos/1013203451. URL: <https://projecteuclid.org/journals/annals-of-statistics/volume-29/issue-5/Greedy-function-approximation-A-gradient-boosting-machine/10.1214/aos/1013203451.full>.
- [11] Trevor Hastie, Robert Tibshirani e Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd. Springer, 2009. ISBN: 978-0387848570.
- [12] Hanan Hindy et al. «A Taxonomy of Network Threats and the Effect of Current Datasets on Intrusion Detection Systems». In: *IEEE Access* 8 (2020), pp. 104650–104675. ISSN: 2169-3536. DOI: 10.1109/access.2020.3000179. URL: <https://ieeexplore.ieee.org/document/9108270>.
- [13] Harold Hotelling. «Analysis of a Complex of Statistical Variables into Principal Components». In: *Journal of Educational Psychology* 24.6 (1933), pp. 417–441. DOI: 10.1037/h0071325. URL: <https://psycnet.apa.org/record/1934-00645-001>.
- [14] Guolin Ke et al. «LightGBM: a highly efficient gradient boosting decision tree». In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. Curran Associates Inc., 2017, pp. 3149–3157. DOI: 10.5555/3294996.3295074. URL: <https://dl.acm.org/doi/10.5555/3294996.3295074>.
- [15] Ansam Khraisat et al. «Survey of intrusion detection systems: techniques, datasets and challenges». In: *Cybersecurity* 2.1 (2019), p. 20. ISSN: 2523-3246. DOI: 10.1186/s42400-019-0038-7. URL: <https://link.springer.com/article/10.1186/s42400-019-0038-7>.
- [16] Ilhan Firat Kilincer, Fatih Ertam e Abdulkadir Sengur. «Machine learning methods for cyber security intrusion detection: Datasets and comparative study». In: *Computer Networks* 188 (2021), p. 107840. ISSN: 1389-1286. DOI: 10.1016/j.comnet.2021.107840. URL: <https://www.sciencedirect.com/science/article/pii/S1389128621000141?via%3Dihub>.

- [17] Oltjon Kodheli et al. «Satellite Communications in the New Space Era: A Survey and Future Challenges». In: *IEEE Communications Surveys & Tutorials* 23.1 (2021), pp. 70–109. DOI: 10.1109/COMST.2020.3028247. URL: <https://ieeexplore.ieee.org/document/9210567>.
- [18] Kun Li et al. «Distributed Network Intrusion Detection System in Satellite-Terrestrial Integrated Networks Using Federated Learning». In: *IEEE Access* 8 (2020), pp. 214852–214865. DOI: 10.1109/ACCESS.2020.3041641. URL: <https://ieeexplore.ieee.org/document/9274426>.
- [19] Suzanne Lightman, Theresa Suloway e Joseph Brule. *Satellite Ground Segment: Applying the Cybersecurity Framework to Satellite Command and Control*. NIST Interagency Report NIST IR 8401. National Institute of Standards e Technology, 2022. DOI: 10.6028/NIST.IR.8401. URL: <https://nvlpubs.nist.gov/nistpubs/ir/2022/NIST.IR.8401.pdf>.
- [20] Scott M Lundberg e Su-In Lee. «A unified approach to interpreting model predictions». In: *CoRR* abs/1705.07874 (2017). DOI: 10.48550/arXiv.1705.07874. URL: <https://arxiv.org/abs/1705.07874>.
- [21] Scott M Lundberg et al. «Explainable AI for Trees: From Local Explanations to Global Understanding». In: *CoRR* abs/1905.04610 (2019). DOI: 10.48550/arXiv.1905.04610. URL: <https://arxiv.org/abs/1905.04610>.
- [22] Saeid Motiian et al. «Unified Deep Supervised Domain Adaptation and Generalization». In: *CoRR* abs/1709.10190 (2017). DOI: 10.48550/arXiv.1709.10190. URL: <https://arxiv.org/abs/1709.10190>.
- [23] Nour Moustafa e Jill Slay. «UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)». In: *2015 Military Communications and Information Systems Conference (MilCIS)*. IEEE, 2015, pp. 1–6. DOI: 10.1109/MilCIS.2015.7348942. URL: <https://ieeexplore.ieee.org/document/7348942>.

- [24] Sinno Jialin Pan e Qiang Yang. «A Survey on Transfer Learning». In: *IEEE Transactions on Knowledge and Data Engineering* 22.10 (2010), pp. 1345–1359. DOI: 10.1109/TKDE.2009.191. URL: <https://ieeexplore.ieee.org/document/5288526>.
- [25] Emanuel Parzen. «On Estimation of a Probability Density Function and Mode». In: *The Annals of Mathematical Statistics* 33.3 (1962), pp. 1065–1076. DOI: 10.1214/aoms/1177704472. URL: <https://projecteuclid.org/journals/annals-of-mathematical-statistics/volume-33/issue-3/On-Estimation-of-a-Probability-Density-Function-and-Mode/10.1214/aoms/1177704472.full>.
- [26] Karl Pearson. «LIII. On Lines and Planes of Closest Fit to Systems of Points in Space». In: *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*. Series 6 2.11 (1901), pp. 559–572. DOI: 10.1080/14786440109462720. URL: <https://www.tandfonline.com/doi/abs/10.1080/14786440109462720>.
- [27] Murray Rosenblatt. «Remarks on Some Nonparametric Estimates of a Density Function». In: *The Annals of Mathematical Statistics* 27.3 (1956), pp. 832–837. DOI: 10.1214/aoms/1177728190. URL: <https://projecteuclid.org/journals/annals-of-mathematical-statistics/volume-27/issue-3/Remarks-on-Some-Nonparametric-Estimates-of-a-Density-Function/10.1214/aoms/1177728190.full>.
- [28] Takaya Saito e Marc Rehmsmeier. «The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets». In: *PLOS ONE* 10.3 (2015), pp. 1–21. DOI: 10.1371/journal.pone.0118432. URL: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0118432>.
- [29] Lloyd S Shapley. «A value for n-person games». In: *Contributions to the Theory of Games* 2.28 (1953), pp. 307–317. DOI: 10.1515/9781400881970-018. URL: <https://www.scirp.org/reference/referencespapers?referenceid=2126587>.
- [30] Iman Sharafaldin, Arash Habibi Lashkari e Ali A. Ghorbani. «Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization». In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. SciTePress, 2018, pp. 108–116. DOI: 10.5220/0006639801080116. URL: <https://api.semanticscholar.org/CorpusID:4707749>.

- [31] Robin Sommer e Vern Paxson. «Outside the Closed World: On Using Machine Learning for Network Intrusion Detection». In: *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. 2010, pp. 305–316. DOI: 10.1109/SP.2010.25. URL: <https://ieeexplore.ieee.org/document/5504793>.
- [32] William Stallings. *Cryptography and Network Security: Principles and Practice*. 8th. Pearson, 2020. ISBN: 978-1-292-43748-4.
- [33] Andrew S. Tanenbaum, Nick Feamster e David Wetherall. *Computer Networks*. 6th. Pearson, 2021. ISBN: 978-1-292-37406-2.
- [34] Mahbod Tavallae et al. «A detailed analysis of the KDD CUP 99 data set». In: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*. IEEE, 2009, pp. 1–6. DOI: 10.1109/CISDA.2009.5356528. URL: <https://ieeexplore.ieee.org/document/5356528>.
- [35] Vladimir N. Vapnik. *Statistical Learning Theory*. New York: John Wiley & Sons, 1998. ISBN: 978-0-471-03003-4.
- [36] Mengke Wan et al. «A Federated Learning-Based Intrusion Detection System for Satellite-Terrestrial Integrated Networks». In: *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2025, pp. 1–5. DOI: 10.1109/ICASSP49660.2025.10890330. URL: <https://ieeexplore.ieee.org/document/10890330>.
- [37] B. L. Welch. «The Generalization of "Student's" Problem when Several Different Population Variances are Involved». In: *Biometrika* 34.1-2 (1947), pp. 28–35. DOI: 10.1093/biomet/34.1-2.28. URL: <https://academic.oup.com/biomet/article-abstract/34/1-2/28/210174?redirectedFrom=fulltext#no-access-message>.
- [38] Garrett Wilson e Diane J. Cook. «A Survey of Unsupervised Deep Domain Adaptation». In: *ACM Transactions on Intelligent Systems and Technology* 11.5 (2020), pp. 1–46. DOI: 10.1145/3400066. URL: <https://arxiv.org/abs/1812.02849>.